

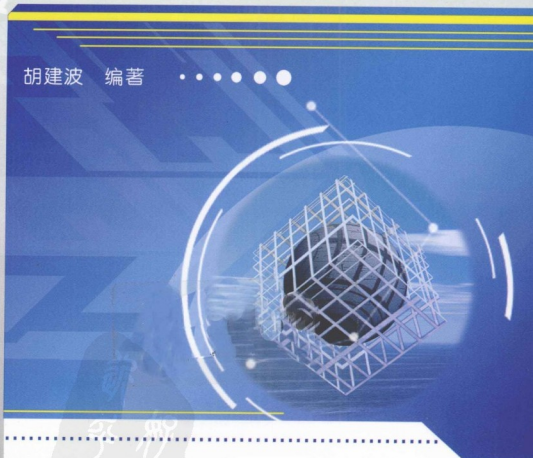
微机原理与 接口技术实验

——基于 Proteus 仿真

胡建波 编著



Computer



PDG



机械工业出版社
CHINA MACHINE PRESS

- ISBN 978-7-111-35065-1
- 策划：陈玉芝/封面设计：张静

Computer

相关图书链接

序号	书号	书名	定价
1	16041	单片机基本原理及应用系统	27
2	27588	单片机控制系统调试与维修	26
3	31564	单片机原理及应用	27
4	32863	单片机实用技术	25



地址：北京市百万庄大街22号
 电话服务
 社服务中心：(010)88361066
 销售一部：(010)68326294
 销售二部：(010)88379649
 读者购书热线：(010)88379203

邮政编码：100037
 网络服务
 门户网：<http://www.cmpbook.com>
 教材网：<http://www.cmpedu.com>
 封面无防伪标均为盗版

上架指导：计算机 / 辅助设计 / 其他

ISBN 978-7-111-35065-1



9 787111 350651 >

定价：28.00元

前言

“微机原理与接口技术”是一门实践性很强的课程，学习时必须理论联系实际，亲自动手做实验，才能达到预期效果。在 Proteus 仿真软件出现前，传统的实验教学一般都要在实验箱上完成，学生只有在上实验课时才能动手进行实验操作，不仅灵活性差，硬件电路不便改动，而且也不利于提高创新能力。Proteus 从 7.5 版开始提供了 VSM (Virtual System Modeling) for 8086 模块，为实验教学提供了理想的实验平台。随着计算机技术的发展，利用虚拟软件进行电路设计与仿真已经成为现代电子技术系统设计的必然趋势。Proteus 仿真软件丰富的元器件模型、多处理器的支持、多样的虚拟仪器、强大的图表分析功能和第三方集成开发环境，已成为电类实践教学与科研的巨大资源。

本书共分为 5 章。第 1 章介绍了 emu8086 的使用并重点练习 8086 指令系统。emu8086 具有直观的操作窗口，有利于理解 8086 CPU 的内部结构及操作数的寻址方式。其中的指令练习多是以任务驱动方式给出一些实例，使同学们在做中学，在学中做。当读得多了，写得多了以后，所用的指令就会逐渐地被深入理解。第 2 章介绍了汇编语言程序设计实验，目的是使学生掌握从汇编语言源程序到可执行程序的步骤。第 3 章介绍了 Proteus 操作基础，主要讲述了 Proteus 的使用及第三方程序开发编译工具。第 4 章介绍了接口技术实验，对本章中的每种接口都给出了多个实例，如 LED 控制循环环灯、LED 数码管可调时电子钟、LED 点阵显示汉字、LCD 液晶显示器显示汉字等。在学习时先不要改动每个实例的硬件仿真电路，应把程序成功调试通过，对接口电路有了初步的理解后，再试着改变或设计电路及编制程序，进一步提高程序的编写及硬件接口电路的设计能力。第 5 章介绍了高级 C 语言程序设计，C 语言更接近于工程实际，具有丰富的函数接口，可极大地减少编程工作量，是现代机电类工程师进行嵌入式开发必备的编程语言工具。本章内容还可作为课程设计或毕业设计的参考文献。

因仿真软件中部分元器件符号与现行国家标准图形符号不一致，而为了便于学习，书中均采用仿真软件中的元器件符号。

由于编者水平有限，加之时间仓促，书中难免会有错误和不妥之处，恳请广大读者批评指正，可通过 E-mail 与作者联系：hjb372@sohu.com。

编者



本书是基于 Proteus 软件平台的“微机原理与接口技术”仿真实验教材,书中提供了完整的实验参考程序和模拟仿真电路,特别适合学生学习使用,有助于提高学生的自主创新能力,从而使学生不再是被动地做实验,而是实验的设计者。本书的主要内容包括:emu8086 汇编指令应用、汇编语言程序设计实验、Proteus 操作基础、接口技术实验和高级 C 语言程序设计。

本书可作为职业院校计算机应用、工业自动化、电子与通信等相关专业的实训教材,也可作为工程技术人员的参考用书。

图书在版编目(CIP)数据

微机原理与接口技术实验:基于 Proteus 仿真/胡建波编著. —北京:机械工业出版社,2011.8

ISBN 978-7-111-35065-1

I. ①微… II. ①胡… III. ①微型计算机-理论-高等职业教育-教材 ②微型计算机-接口-高等职业教育-教材 IV. ①TP36

中国版本图书馆 CIP 数据核字(2011)第 115684 号

机械工业出版社(北京市百万庄大街 22 号 邮政编码 100037)

策划编辑:陈玉芝 责任编辑:林运鑫

版式设计:霍永明 责任校对:闫玥红

封面设计:张静 责任印制:杨曦

北京中兴印刷有限公司印刷

2011 年 8 月第 1 版第 1 次印刷

184mm×260mm·13.75 印张·334 千字

0 001—3 000 册

标准书号:ISBN 978-7-111-35065-1

定价:28.00 元

凡购本书,如有缺页、倒页、脱页,由本社发行部调换

电话服务

网络服务

社服务中心:(010)88361066

门户网:<http://www.cmpbook.com>

销售一部:(010)68326294

教材网:<http://www.cmpedu.com>

销售二部:(010)88379649

读者购书热线:(010)88379203

封面无防伪标均为盗版

目 录

前言

第 1 章 emu8086 汇编指令应用 1

1.1 emu8086 汇编软件 1

1.2 emu8086 汇编指令应用 5

第 2 章 汇编语言程序设计实验 15

2.1 汇编语言源程序到可执行程序的生成过程 15

2.2 汇编语言程序设计实例 19

第 3 章 Proteus 操作基础 28

3.1 Proteus ISIS 操作界面 28

3.1.1 ISIS 菜单栏 29

3.1.2 ISIS 命令工具条 29

3.1.3 ISIS 模式选择工具条 29

3.1.4 ISIS 旋转、镜像控制按钮 31

3.1.5 ISIS 仿真控制按钮 31

3.2 Proteus ISIS 电路原理图设计 31

3.2.1 ISIS 鼠标使用规则 31

3.2.2 原理图设计 31

3.3 源程序编辑编译环境 36

3.3.1 编译器 36

3.3.2 环境变量设置 37

第 4 章 接口技术实验 38

4.1 74LS273 输出口控制 LED 40

4.1.1 74LS273 输出口控制循环彩灯 40

4.1.2 单个移动点亮 LED 43

4.1.3 逐个递增长点亮 LED 44

4.1.4 LED 霓虹灯 44

4.1.5 LED 模拟交通灯 46

4.2 74LS273 输出口控制 LED 数码管 47

4.2.1 74LS273 输出口控制 1 位共阴极 LED 数码管 47

4.2.2 74LS273 输出口控制 2 位共阴极 LED 数码管 49

4.2.3 74LS273 输出口控制 8 位 LED 数码管滚动显示单个数字 51

4.2.4 74LS273 输出口控制 8 位共阴极 LED 数码管动态扫描 53

4.3 74LS244 简单输入接口 54

4.3.1 用 74LS244 作输入口读取开关状态 54

4.3.2 独立式按键控制 LED 左右移动 55

4.3.3 独立式按键控制 LED 数码管增减 1 57

4.3.4 LED 数码管显示行列式键盘键值 59

4.4 点阵 LED 显示汉字 63

4.4.1 16 × 16 点阵 LED 显示汉字 63

4.4.2 16 × 16 点阵 LED 移动显示多个汉字 65

4.4.3 16 × 32 点阵 LED 移动显示多个汉字 68

4.4.4 32 × 32 点阵 LED 移动显示多个汉字 71

4.5 液晶显示器 75

4.5.1 LM032L 字符液晶显示器 75

4.5.2 12864 图形液晶显示器 79

4.5.3 PG160128 图形液晶显示器 87

4.6 8255A 可编程接口芯片 95

4.6.1 8255A 可编程接口芯片 PC 口应用 95

4.6.2 8255A 可编程接口芯片 PA 口作输出口 96

4.6.3 8255A 可编程接口芯片 PA 口作输出口控制一位共阳极 LED 数码管 98

4.6.4 8255A 可编程接口芯片 PA 口方式 0 作输入口、PB 口方式 0 作输出口 99

4.6.5 8255A 可编程接口芯片作按键检测及多位 LED 数码管显示接口 100

4.6.6 8255A 可编程接口芯片 PB 口方式 1 作输出口 103

4.6.7 8255A 可编程接口芯片 PB 口方式 1 作输入口 105

4.6.8 8255A 可编程接口芯片 PA 口方式 1

作输出口	108	转换结果	146
4.6.9 8255A 可编程接口芯片 PA 口方式 2 检测开关状态	111	4.10.2 延时方式读取 ADC0809 模/数 转换结果	148
4.7 不可屏蔽中断	114	4.10.3 中断方式读取 ADC0809 模/数 转换结果	149
4.7.1 不可屏蔽中断控制 8 位 LED 交替闪烁	114	4.10.4 中断方式读取 ADC0809 多路 模/数转换结果	150
4.7.2 不可屏蔽中断控制 8 位 LED 向左移动	116	4.10.5 数据线方式选择 ADC0809 模/数 转换通道	153
4.7.3 不可屏蔽中断控制 1 位 LED 数码管递减	117	4.10.6 ADC0809 多路模拟量采集	154
4.7.4 不可屏蔽中断控制 2 位 LED 数码管	119	4.11 DAC0832 数/模转换器	159
4.8 8251A 可编程串行接口芯片	121	4.11.1 DAC0832 单缓冲方式输出 三角波	159
4.8.1 8251A 可编程串行接口芯片发送 接口及编程	121	4.11.2 DAC0832 双缓冲工作方式	160
4.8.2 基于虚拟串行接口的 8251A 收发 程序测试	123	4.11.3 DAC0832 直通工作方式	162
4.8.3 串并转换接口	124	4.12 电动机控制	164
4.8.4 并串转换接口	125	4.12.1 用 DAC0808 数/模转换器 实现电动机调速	164
4.9 8253A 可编程定时/计数器	128	4.12.2 直流电动机正反转控制	166
4.9.1 用虚拟示波器观察 8253A 的 输出波形	128	4.12.3 步进电动机正反转控制	168
4.9.2 用 8253A 定时/计数器控制 8 位 LED 循环移动	129	4.13 8279 键盘显示接口	171
4.9.3 用 8253A 定时/计数器控制 1 位 LED 数码管数字递增	131	4.13.1 利用 8279 键盘显示接口 显示键号	171
4.9.4 用 8253A 定时/计数器设计 跑表	133	4.13.2 利用 8279 键盘显示接口 显示时钟	175
4.9.5 用 8253A 定时/计数器设计可调 时电子钟	137	4.14 存储体扩展	178
4.9.6 用 8253A 定时/计数器作方波 发生器	144	4.15 8259A 可编程中断控制器	179
4.10 ADC0809 模/数转换器	146	第 5 章 高级 C 语言程序设计	184
4.10.1 查询方式读取 ADC0809 模/数 转换结果	146	5.1 开关检测与状态显示	184
		5.2 LCD1602 电子钟	187
		5.3 LCM12864 万年历	194
		5.4 LCD2002 显示 ADC0809 八路模拟 转换结果	205
		参考文献	210

第 1 章 emu8086 汇编指令应用

1.1 emu8086 汇编软件

1. 实验任务

在屏幕第 10 行 20 列上显示“HELLO WORLD!”。

2. 实验目的

通过实例熟练掌握 emu8086 汇编软件实验环境，观察和分析在不同的寻址方式下存储单元逻辑地址的表示以及指令的执行结果。

3. 字符显示原理

在 25×80 彩色字符模式下，从第 0 行 0 列到第 24 行 79 列共有 2000 个字符位置。对于屏幕上的每个字符，内存的显示缓冲区中都有相应的存储单元与之相对应。在内存的 B8000H~BFFFFH 共 32KB 的区域是 25×80 彩色字符模式的显示缓冲区。如果向这个地址范围的单元中写入数据，写入的字符就可立即显示在屏幕上。屏幕上显示的每个字符用两个字节表示，第一个字节为字符的 ASCII 码，第二个字节为字符的属性。字符的属性代表了字符的显示特性，如颜色、闪烁、高亮反相显示等。

(1) 彩色字符显示属性 彩色字符的背景色有 8 种颜色，前景色有 16 种颜色，其字符属性格式如图 1-1 所示。前景色由 4 位 (D₀~D₃) 组成，背景色由 3 位 (D₄~D₆) 组成；最高位 BL 表示闪烁，RGB 为红、绿、蓝，I 表示亮度。由表 1-1 可知，02H 表示黑底绿字；1EH 表示蓝底黄字，应避免前景色、背景色一致，否则字符看不出来。

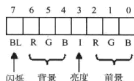


图 1-1 字符属性格式

表 1-1 字符显示属性

IRGB	颜色	IRGB	颜色
0000	黑	1000	灰
0001	蓝	1001	浅蓝
0010	绿	1010	浅绿
0011	青	1011	浅青
0100	红	1100	浅红
0101	品红	1101	浅品红
0110	棕	1110	黄
0111	灰白	1111	白

(2) 字符显示位置 对于 25×80 彩色字符模式，一屏字符需占用 4000B，因此 32KB 的显示缓冲区分为 8 页，每页 4KB。B8000H~B8F9FH (4000B) 为第 0 页的内容。由于一

行有 80 个字符, 共占用 160B (A0H), 因此显存单元和显示器中行的对应关系为: 000H ~ 09FH 单元对应显示器上的第 0 行; 0A0H ~ 13FH 单元对应显示器上的第 1 行; 140H ~ 1DFH 单元对应显示器上的第 2 行。

内存中每两个字节对应一个字符, 高字节为字符的属性, 低字节为字符的 ASCII 码。偶地址单元存放字符的 ASCII 码, 奇地址单元存放字符的属性。字符的位置计算公式为: 位置 = 行号 $\times$ 160 + 列号 $\times$ 2。

为了简化程序设计, 本例用顺序程序设计方法, 根据上述任务要求写出相应的汇编指令。在上机时可先不看注释前面的指令, 自己先试着编写一下再作对比。

4. 参考程序

```

MOV  AX, 0B800H      ;立即数 0B800H 送到寄存器 AX
MOV  DS, AX          ;寄存器 AX 内容送到寄存器 DS
MOV  ES, AX          ;寄存器 AX 内容送到寄存器 ES
MOV  AL, 'H'         ;字符 'H' 送到寄存器 AL
MOV  AH, 1EH         ;颜色属性值 1EH (蓝色背景黄色字体) 送到寄存器 AH
                        ;第 10 行 20 列显存地址偏移量:  $10 \times 160 + 20 \times 2 = 1640 = 0668H$ 
MOV  [0668H], AX     ;寄存器 AX 内容送到数据段直接地址 0668H

MOV  AL, 'E'         ;字符 'E' 送到寄存器 AL
MOV  AH, 02H         ;02H (黑底绿字) 送到寄存器 AH
MOV  ES:[066AH], AX  ;寄存器 AX 内容送附加段直接地址 066AH

MOV  BX, 066CH       ;066CH 送到寄存器 BX
MOV  AL, 'L'         ;字符 'L' 送到寄存器 AL
MOV  AH, 02H         ;02H (黑底绿字) 送到寄存器 AH
MOV  [BX], AX        ;AX 寄存器内容送到数据段 BX 间接寻址

MOV  AL, 'L'         ;字符 'L' 送到寄存器 AL
MOV  2[BX], AX       ;AX 寄存器内容送到数据段 BX + 2 相对间接寻址

MOV  SI, 0004H       ;立即数 0004H 送到寄存器 SI
MOV  AL, 'O'         ;字符 'O' 送到寄存器 AL
MOV  ES:[BX + SI], AX ;寄存器 AX 内容送到附加段 BX + SI 基址变址寻址

MOV  AL, 'W'         ;字符 'W' 送到寄存器 AL
MOV  ES:4[BX + SI], AX ;寄存器 AX 内容送到附加段 4 + BX + SI 相对基址变址寻址
ADD  SI, 2           ;SI + 2  $\rightarrow$  SI
MOV  AL, 'O'         ;字符 'O' 送到寄存器 AL

```

```

MOV ES:4[BX+SI], AX ;寄存器 AX 内容送到附加段 4 + BX + SI 相对基址变址寻址

MOV BP, 0678H ;立即数 0678H 送到寄存器 BP
MOV AL, 'R' ;字符'R'送到寄存器 AL
MOV DS:[BP], AX ;寄存器 AX 内容送到数据段 BP 间接寻址
MOV AL, 'D' ;字符'D'送到寄存器 AL
MOV ES:2[BP], AX ;寄存器 AX 内容送到附加段 BP 相对间接寻址
MOV AL, '!' ;字符'!'送到寄存器 AL
MOV ES:4[BP], AX ;寄存器 AX 内容送到附加段 BP 相对间接寻址
HLT

```

5. 实验步骤

1) 直接双击桌面上的 emu8086 快捷图标进入 emu8086 汇编语言编辑窗口, 图 1-2 所示为其编辑窗口菜单, 选择“new”图标, 在弹出的窗口中选择“empty workspace”, 新建一个空的工作区。

2) 输入上述源程序。特别强调的是可先输入两三条指令直接单击图 1-2 中“emulate”图标进入调试环境, 单步执行, 观察指令执行时 CPU 寄存器的变化情况。由于刚开始对汇编指令格式掌握还不太牢固, 最好不要一次将全部指令输入完毕, 否则不易发现错误。

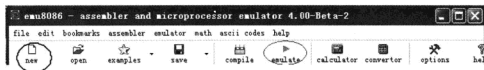


图 1-2 emu8086 编辑窗口菜单

3) 待程序全部输入完, 可存盘为“hello.asm”, 单击图 1-2 中“compile”图标进行编译, 若没有错误, 编译通过弹出可执行文件保存对话框, 存盘为“hello.exe”。在新弹出的图 1-3 所示对话框中单击“external”图标选择“run”全速执行。需要说明的是, 步骤 2) 中的 emulate 调试环境中也有 run 功能但速度较慢。



图 1-3 emu8086 外部命令窗口

4) 调试。单击“emulate”图标(见图 1-2)进入调试窗口(见图 1-4), 单步执行并观察 CPU 寄存器值、指令的物理地址、逻辑地址、机器码等变化; 打开堆栈窗口(见图 1-5)、附加段窗口(见图 1-6)、debug 命令窗口(见图 1-7)、flags 标志位窗口(见图 1-8), 单步执行观察其数据的变化, 并分析每一条指令执行后的结果是否与要求一致。

6. 练习

1) 设当程序执行到最后一条指令“MOV ES: 4 [BP], AX”时, debug 状态如下:

```
0100: 004C 26          ES:
AX = 0221  BX = 066C  CX = 0000  DX = 0000  SP = FFFE  BP = 0678  SI = 0006  DI = 0000
DS = B800  ES = B800  SS = 0100  CS = 0100  IP = 004D  NV UP EI PL NZ NA PE NC
0100: 004D 894604      MOV [BP] + 04H, AX      ES: 067C = 0700H
```

附加段状态如下:

```
B800: 0668 48 1E 45 02 4C 02 4C 02 -4F 02 00 07 57 02 4F 02
B800: 0678 52 02 44 02 00 07 00 07 -00 07 00 07 00 07 00 07
```

从上述指令 debug 状态中可知当前指令使用了段超越, 使用附加段寄存器 ES 作为目的操作数的段基址, 附加段基址为_____ ; 当前指令的 IP 值为_____, 物理地址为_____ ; 当前指令长度为_____ B (不含 ES), 其中机器码中的 04 表示目的操作数的_____, BP 寄存器的值为_____, 两者的和为目的操作数的有效地址, 目的操作数的逻辑地址为_____, 物理地址为_____。

2) 在屏幕的第 4 行 25 列以红色字体黄色背景显示: “ONE WORLD! ONE DREAM!”。

```
MOV AX, 0B800H          ;显示缓冲区首地址
MOV ES, AX              ;附加段基址
MOV SI, OFFSET ASC       ;取 ASC 有效地址送 SI 寄存器
MOV BX, 4 * 160 + 25 * 2 ;第 4 行 25 列显存地址
MOV AH, 0ECH             ;0ECH 表示黄底红字
MOV CX, 21               ;ASC 字符串长度
LP: MOV AL, [SI]          ;取字符
    MOV ES: [BX], AX      ;送显示缓冲区
    INC SI                ;字符串地址加 1
    ADD BX, 2              ;待显示字符串 ASC 的下一列显存地址
    LOOP LP               ;循环
HLT
ASC DB 'ONE WORLD! ONE DREAM!'
```

1.2 emu8086 汇编指令应用

本节继续通过一些实例加深对 emu8086 汇编指令的理解与应用, 初步掌握汇编语言程序的编写方法。

例 1-1 在屏幕的第 8 行 20 列开始显示如图 1-9 所示一个宽 40、高 10 行的红色窗口。

1) 参考程序。



图 1-9 红色窗口

○因图书为黑白印刷, 在图中显示为深灰色, 仿真软件中显示为红色。

```

WINWIDTH  = 40                                ;窗口宽度
HEIGHT     = 10
WINTOP     = 8                                ;窗口左上角行号
WINLEFT    = 20                                ;窗口左上角列号
WINBOTTOM  = WINTOP + HEIGHT - 1              ;窗口右下角行号
WINRIGHT   = WINLEFT + WINWIDTH - 1          ;窗口右下角列号
COLOR      = 47H                              ;47H 红屏白字

        MOV AX,0B800H                        ;显示缓冲区首址
        MOV DS,AX
        MOV BX,WINTOP * 160                  ;BX 用作行指针
LPO:     MOV SI,WINLEFT * 2                  ;SI 用作列指针
LP:      MOV BYTE PTR 1[BX + SI],COLOR      ;颜色属性送到显示缓冲区
        CMP SI,WINRIGHT * 2                ;是否到窗口最后一列
        JE  NEXTLINE
        INC SI
        INC SI
        JMP LP
NEXTLINE: CMP BX,WINBOTTOM * 160            ;是否到窗口最后一行
        JE  EXITBOTTOM
        ADD BX,160
        JMP LPO
EXITBOTTOM: HLT

```

2) 练习。单步执行上述程序, 查看 debug 状态, 指令“MOV BX, WINTOP * 160”的逻辑地址、机器码、反汇编指令分别是____、____、____, 机器码中后两个字节是____; LP 标号的逻辑地址是____; 当程序执行到 JE NEXTLINE 指令时, 其指令逻辑地址、机器码、反汇编指令分别是____、____、____, 该指令转移的条件标志是____; JMP LP 指令其机器码中的相对量是____; JE EXITBOTTOM 指令其机器码中的相对量是____; JMP LPO 指令其机器码中的相对量是____。

例 1-2 在屏幕的第 8 行 20 列开始显示如图 1-10 所示一个高 10 行的红色直角三角形窗口。

```

HEIGHT     = 10                                ;窗口左上角行号
WINTOP     = 8                                ;窗口左上角列号
WINLEFT    = 20                                ;窗口右下角行号
WINBOTTOM  = WINTOP + HEIGHT - 1              ;窗口右下角列号
COLOR      = 47H                              ;47H 红屏白字

```

```

MOV AX,0B800H                        ;显示缓冲区首址
MOV DS,AX

```

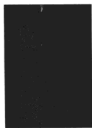


图 1-10 红色直角三角形窗口 (一)


```

MOV BX, WINTOP * 160          ;BX 用作行指针
MOV CX, WINLEFT * 2          ;列尾
LP0:  MOV SI, WINLEFT * 2      ;SI 用作列指针
LP:   MOV BYTE PTR 1[BX + SI], COLOR ;颜色属性送到显示缓冲区
      CMP SI, CX              ;是否到窗口最后一列
      JE NEXTLINE
      INC SI
      INC SI
      JMP LP
NEXTLINE: CMP BX, WINBOTTOM * 160 ;是否到窗口最后一行
      JE EXITBOTTOM
      ADD BX, 160
      INC CX
      INC CX
      JMP LP0
EXITBOTTOM: HLT

```

例 1-3 在屏幕的第 8 行 20 列开始显示如图 1-11 所示一个高 10 行的红色直角三角形窗口。

```

WINWIDTH   = 10          ;窗口宽度
HEIGHT     = 10
WINTOP      = 8          ;窗口左上角行号
WINLEFT     = 20         ;窗口左上角列号
WINBOTTOM   = WINTOP + HEIGHT - 1 ;窗口右下角行号
WINRIGHT    = WINLEFT + WINWIDTH ;窗口右下角列号
COLOR       = 47H        ;47H 红屏白字
DATA        SEGMENT
ASC         DB 30H,31H,32H,33H,34H,35H,36H,37H,38H,39H
DATA        ENDS
CODE        SEGMENT
START:      MOV AX, DATA
            MOV DS, AX          ;加载数据段基址
            MOV AX, 0B800H      ;显示缓冲区首址
            MOV ES, AX
            MOV BX, WINTOP * 160 ;BX 用作行指针
            MOV CX, WINLEFT * 2  ;列尾
LP0:        MOV DI, 0           ;字符指针
            MOV SI, WINLEFT * 2 ;SI 用作列指针
LP:         MOV AL, ASC[DI]
            MOV AH, COLOR

```



图 1-11 红色直角三角形窗口 (二)

```

MOV ES:[BX + SI],AX      ;ASC 码及颜色属性送显示缓冲区
CMP SI,CX                ;是否到窗口最后一列
JE NEXTLINE
INC SI
INC SI
INC DI                    ;下一字符
JMP LP
NEXTLINE: CMP BX,WINBOTTOM * 160 ;是否到窗口最后一行
JE EXITBOTTOM
ADD BX,160
INC CX
INC CX
JMP LPO
EXITBOTTOM: HLT
CODE      ENDS
END      START

```

例 1-4 在屏幕的第 8 行 20 列开始显示如图 1-12 所示一个高 10 行的红色倒立直角三角形窗口。

```

WINWIDTH   = 10          ;窗口宽度
HEIGHT      = 10
WINTOP      = 8           ;窗口左上角行号
WINLEFT     = 20          ;窗口左上角列号
WINBOTTOM   = WINTOP + HEIGHT - 1 ;窗口右下角行号
WINRIGHT    = WINLEFT + WINWIDTH - 1 ;窗口右下角列号
COLOR       = 47H        ;47H 红屏白字
DATA        SEGMENT
ASC         DB 30H,31H,32H,33H,34H,35H,36H,37H,38H,39H
DATA        ENDS
CODE        SEGMENT
ST:         MOV AX,DATA      ;加载数据段基址
            MOV DS,AX
            MOV AX,0B800H    ;显示缓冲区首址
            MOV ES,AX
            MOV BX,WINTOP * 160 ;BX 用作行指针
            MOV CX,WINLEFT * 2 ;列头指针
LPO:        MOV SI,WINLEFT * 2 ;SI 用作列指针
            MOV DI,0
LP:         CMP SI,CX
            JB  NEXT_COL     ;小于列头,转 NEXT_COL

```

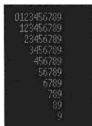


图 1-12 红色倒立
直角三角形窗口

```

        MOV AL,ASC[DI]
        MOV AH,COLOR
        MOV ES:[BX+SI],AX           ;颜色属性送到显示缓冲区
        CMP SI,WINRIGHT*2          ;是否到窗口最后一列
        JE NEXTLINE
NEXT_COL:INC SI
        INC SI
        INC DI
        JMP LP
NEXTLINE:CMP BX,WINBOTTOM*160       ;是否到窗口最后一行
        JE EXITBOTTOM
        ADD BX,160
        INC CX
        INC CX
        JMP LPO
EXITBOTTOM:HLT
CODE ENDS
END ST

```

例 1-5 在屏幕的第 3 行 20 列 ($160 \times 3 + 20 \times 2 = 520$) 循环显示 0~9。

1) 参考程序。

```

CODE SEGMENT
        ASSUME CS:CODE,DS:DATA
START: MOV AX,DATA
        MOV DS,AX
        MOV AX,0B800H           ;数据段基址
        MOV ES,AX               ;显示缓冲区首地址
LP0:    MOV SI,0                 ;0~9
        MOV CX,10
LP:     MOV AL,VAR[SI]           ;ASCII 码
        MOV ES:[520],AL        ;ASCII 码送到显示缓冲区
        CALL DEL                ;延时
        INC SI
        MOV AH,0BH              ;Ctrl + C 退出
        INT 21H
        LOOP LP
        JMP LPO
DEL:    PROC                    ;延时
        PUSH BX

```

```

PUSH CX
MOV BX,0400H
LP2: MOV CX,4000H
LP1: PUSHF
POPF
LOOP LP1
DEC BX
JNZ LP2
POP CX
POP BX
RET
DEL ENDP
CODE ENDS
DATA SEGMENT
VAR DB '0123456789'
DATA ENDS
END START

```

2) 调试说明。在 emu8086 中输入上述程序，存盘为“num.asm”，调试时，子程序中的“MOV BX, 0400H”和“LP2: MOV CX, 4000H”两条指令中的立即数可以先赋值 1，调试通过后再改为原值，或者先用一个空的（仅有 RET 指令）延时子程序。上述程序调试通过后编译生成 num.exe，在新弹出的对话框中单击“external”图标选择“run”，观察程序执行效果。

3) 练习。单步执行上述程序，查看 debug 和 stack 状态，指令 CALL DEL 的逻辑地址、机器码、反汇编指令分别是_____、_____、_____，指令执行前 SP 的值是_____，指令 CALL DEL 执行后 SP 的值是_____，栈顶内容是_____，子程序 DEL 的逻辑地址是_____。

RET 指令的 IP 地址是_____，RET 指令执行前 SP 的值是_____，栈顶内容是_____，RET 指令执行后将栈顶内容送入_____并将 SP 值加_____，程序返回主程序继续运行。

INT 21H 指令的逻辑地址、机器码、反汇编指令分别是_____、_____、_____，指令执行前 SP 的值是_____，执行指令 INT 21H 进入软中断服务程序后 SP 的值是_____，栈顶的 3 个字内容是（由高到低）_____、_____、_____，分别是进入中断前的_____标志，程序返回指令的_____、_____。

例 1-6 如图 1-13 所示，设外部有一个 16 位输出端口，地址为 4，用其低 12 位分别控制东南西北四个方向的交通灯。

1) 任务分析。由图 1-13 可知，0、1、2 位分

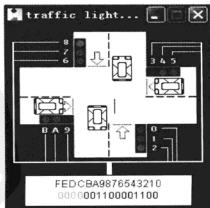


图 1-13 交通灯

别控制路口南边的红、黄、绿3个交通灯；3、4、5位分别控制路口东边的红、黄、绿3个交通灯；6、7、8位分别控制路口北边的红、黄、绿3个交通灯；9、A、B位分别控制路口西边的红、黄、绿3个交通灯；当某位为1时，对应于该位交通灯点亮，为0时熄灭。设交通信号有如下4个状态：

- ① 状态1为南北绿、东西红；延时20s；控制字为：0000_001_100_001_100B。
- ② 状态2为南北黄、东西红黄；延时5s；控制字为：0000_011_010_011_010B。
- ③ 状态3为南北红、东西绿；延时20s；控制字为：0000_100_001_100_001B。
- ④ 状态4为南北红黄、东西绿；延时5s；控制字为：0000_010_011_010_011B。

2) 参考程序。

#START = TRAFFIC_LIGHTS.EXE# ;模拟交通灯

NAME "TRAFFIC"

```
AGAIN: MOV SI, OFFSET S1 ;状态1
        MOV AX, [SI]
        OUT 4, AX
        CALL DEL20s ;延时20s
        ADD SI, 2 ;状态2
        MOV AX, [SI]
        OUT 4, AX
        CALL DEL05s ;延时5s
        ADD SI, 2 ;状态3
        MOV AX, [SI]
        OUT 4, AX
        CALL DEL20s ;延时20s
        ADD SI, 2 ;状态4
        MOV AX, [SI]
        OUT 4, AX
        CALL DEL05s ;延时5s
        JMP AGAIN ;返回状态1
```

```
; FEDC_BA98_7654_3210
S1 DW 0000_0011_0000_1100B
S2 DW 0000_0110_1001_1010B
S3 DW 0000_1000_0110_0001B
S4 DW 0000_0100_1101_0011B

DEL20s PROC ;延时20s 子程序
        MOV CX, 0131H ;01312D00H = 20,000,000
        MOV DX, 2D00H
        MOV AH, 86H
        INT 15H
```

```

RET
DEL20s ENDP
DEL05s PROC                                ;延时 5s 子程序
    MOV     CX, 4CH                        ;004C4B40H = 5,000,000
    MOV     DX, 4B40H
    MOV     AH, 86H
    INT     15H
    RET
DEL05S ENDP

```

例 1-7 在屏幕的第 10 行 10 列显示“12: 55: 33”格式的电子钟。

1) 参考程序。

```

DATA SEGMENT
TIME DB '23:59:50'
DATA ENDS
STACK SEGMENT STACK
    DW 100 DUP(?)
STACK ENDS
CODE SEGMENT
    ASSUME CS:CODE,DS:DATA
START:MOV AX,DATA                        ;数据段基址
    MOV DS,AX
TODISP_INIT:MOV AX,0B800H                ;彩色显示缓冲区首址
    MOV ES,AX
    MOV CX,8                            ;LOOP 循环次数,即 TIME 字符个数
    MOV BX,0                            ;数组变量 TIME 下标
    MOV DI,654H                          ;待显示字符显存有效地址
TODISP:MOV AL,TIME[BX]                  ;取数组变量 TIME 的一个字节
    MOV ES:[DI],AL                      ;送显存
    INC BX                               ;TIME 下标加 1
    ADD DI,2                             ;指向下一个显存位置
    LOOP TODISP
LP:    INC TIME+7                        ;时间值秒个位加 1
    CMP TIME+7,3AH                      ;秒个位是否等于 10
    JNE EXIT                             ;秒个位不等于 10 退出
    MOV TIME+7,30H                      ;秒个位回 0
    INC TIME+6                           ;秒十位加 1
    CMP TIME+6,36H                      ;秒十位是否等于 6

```

```

JNE EXIT                ;秒十位不等于6,退出
MOV TIME+6,30H          ;秒十位回0
INC TIME+4              ;分个位加1
CMP TIME+4,3AH          ;分个位是否等于10
JNE EXIT                ;分个位不等于10,退出
MOV TIME+4,30H          ;分个位回0
INC TIME+3              ;分十位加1
CMP TIME+3,36H          ;分十位是否等于6
JNE EXIT                ;分十位不等于6,退出
MOV TIME+3,30H          ;分十位回0
INC TIME+1              ;时个位加1
CMP TIME+1,34H          ;时个位是否等于4
JNE EXIT                ;时个位不等于4,退出
MOV TIME+1,30H          ;时个位回0
INC TIME+0              ;时十位加1
CMP TIME+0,33H          ;时十位是否等于3
JNE EXIT                ;时十位不等于3,退出
MOV TIME+0,30H          ;时十位回0
EXIT: CALL DELAY 1s     ;延时
      JMP TODISP_INIT
DELAY1S PROC NEAR      ;延时子程序
      PUSH CX
      PUSH BX
      MOV BX,100
LP2:  MOV CX,4690H
LP1:  PUSHF
      POPF
      LOOP LP1
      DEC BX
      JNZ LP2
      POP BX
      POP CX
      RET
DELAY1S ENDP
CODE ENDS
END START

```

2) 调试说明。在 emu8086 中输入上述程序,存盘为“time.asm”,调试时,子程序中

的“MOV BX, 100”和“LP2: MOV CX, 4690H”两条指令中的立即数可以先赋值 1，调试通过后再改为原值，或者先用一个空的（仅有 RET 指令）延时子程序。上述程序调试通过后编译生成 time.exe，在新弹出的对话框中单击“external”图标选择“run”，观察程序执行效果。

3) 练习。设当程序执行到 EXIT: CALL DELAY1S 子程序调用指令时，其 debug 状态如下：

```
AX=B830  BX=0008  CX=0000  DX=0000  SP=00C8  BP=0000  SI=0000  DI=0664
DS=0710  ES=B800  SS=0711  CS=071E  IP=007E  NV UP EI NG NZ AC PO CY
071E; 007E E80200          CALL 00083H
```

从上述状态中可知，这条调用指令的逻辑地址是_____，机器码是_____，下一条指令 JMP TODISP_ INIT 指令的 IP 地址是_____，当前堆栈段栈顶指针 SP 的值是_____，程序转向的目的地址 IP 是_____。

第2章 汇编语言程序设计实验

2.1 汇编语言源程序到可执行程序的生成过程

本节通过一个实例说明由汇编语言源程序 (*.ASM) 到可执行程序 (*.EXE) 操作过程。

1. 实验任务

在屏幕上输出: “hello, world!”。

2. 实验目的

- 1) 学习汇编语言的编译过程。
- 2) 学习常用 debug 调试命令的使用。

3. 参考程序

```
;HELLO.ASM
DATA SEGMENT                                ;数据段定义开始
MSG DB 'hello,world! ',0DH,0AH, '$'         ;0DH,0AH 表示回车、换行
DATA ENDS                                    ;数据段定义结束
CODE SEGMENT                                ;代码段定义开始
    ASSUME CS:CODE,DS:DATA                  ;ASSUME 伪指令指定段寄存器与对应
                                              ;段名
START: MOV AX,DATA                          ;段名即表示段基址的高16位
    MOV DS,AX                               ;将数据段的段地址装入 DS 寄存器
    MOV DX,OFFSET MSG                      ;取得显示字符串的有效地址
    MOV AH,09H                             ;DOS 调用 09 号功能,显示输出
    INT 21H
    MOV AH,4CH                             ;DOS 调用 4C 号功能,返回操作系统
    INT 21H
CODE ENDS
END START
```

4. 汇编器 MASM 和连接器 LINK 的使用

假设编译器存放在“C:\MASM50”文件夹下,为了方便地将程序存放在不同的目录下,首先设定 WINDOWS 环境变量,编辑 PATH 环境变量,在其前面加上“C:\MASM50”。

设上述源程序保存在“C:\8086\hello”目录下,打开【开始】→【程序】→【附件】→【命令提示符】进入 DOS 目录。

1) 执行“cd c:\8086\hello”进入“\hello”目录(见图 2-1),执行 edit hello.asm 编辑源程序或用记事本等其他编辑软件。

```

c:\ 命令提示符
Microsoft Windows [XP 版本 5.1.2600]
(C) 版权所有 1985-2001 Microsoft Corp.

C:\Documents and Settings\Administrator\ed c:\8086\hello

C:\8086\hello>edit hello.asm

```

图 2-1 EDIT 编辑命令

2) 执行 `masm hello.asm` 命令对源程序进行汇编, 确定源程序没有编译错误 (见图 2-2), 若有错误, 则应修改源程序错误后重新编译。若在汇编时输入列表文件名, 除生成 `.obj` 目标文件外还会产生 `.lst` 列表文件。

3) 执行 `link hello.obj` 命令对生成的 `hello.obj` 进行连接。确定没有错误 (见图 2-3), 若有错误, 则应修改源程序错误后重新编译和连接。

```

c:\ 命令提示符

C:\8086\hello>masm hello.asm
Microsoft (R) Macro Assembler Version 5.00
Copyright (C) Microsoft Corp 1981-1985, 1987. All rights reserved.

Object filename [hello.OBJ]:
Source listing [NUL.LST]:
Cross-reference [NUL.CRF]:

    50770 + 45031B Bytes symbol space free

    0 Warning Errors
    0 Severe Errors

C:\8086\hello>

```

图 2-2 汇编后提示信息

```

c:\ 命令提示符

C:\8086\hello>link hello.obj
Microsoft (R) Overlay Linker Version 3.60
Copyright (C) Microsoft Corp 1983-1987. All rights reserved.

Run File [HELLO.EXE]:
List File [NUL.MAP]:
Libraries [LIB]:
LINK : warning L4021: no stack segment

C:\8086\hello>

```

图 2-3 连接后提示信息

5. 调试

多数情况下源程序编译通过后可能还隐藏着其他一些非语法或结构性错误, 例如, 在输入程序时漏掉一行指令, 宏汇编程序是不

```

C:\8086\hello>debug hello.exe

```

图 2-4 启动 debug 并加载 exe 文件

会发现并报错的。因此, 可以用 DOS 操作系统所带的 debug 调试程序对可执行文件 hello.exe 进行调试, 在进入 debug 时应加上文件的扩展名 (*.exe), 如图 2-4 所示。

1) 反汇编: U 命令。反汇编命令的最左边一列是程序的逻辑地址 (见图 2-5), 从上边的状态中可知当前程序第一条指令 CS 段值为 13DAH, IP 偏移量为 0000H。这里要说明的是: debug 中的所有数据均为 16 进制数; 反汇编命令输出的中间部分为对应指令的机器码; 最右边部分为反汇编指令。

```

-U
13DA:0000 B8D913      MOV     AX,13D9
13DA:0003 8ED8        MOV     DS,AX
13DA:0005 B00000      MOV     DX,0000
13DA:0008 B409        MOV     AH,09
13DA:000A CD21        INT     21
13DA:000C B44C        MOV     AH,4C
13DA:000E CD21        INT     21
13DA:0010 8BE5        MOV     SP,BP
13DA:0012 5D          POP     BP
13DA:0013 C3          RET
13DA:0014 833E560720     CMP     WORD PTR [07561, +20]
13DA:0019 720A        JB      0025
13DA:001B B81C04        MOV     AX,041C
13DA:001E 50          PUSH    AX
13DA:001F E86244        CALL   4484

```

图 2-5 反汇编窗口 (一)

U 命令显示的第一条指令是源程序中标号为 START 的指令 MOV AX, DATA, 在 debug 下 DATA 已经变为 13D9H, 则说明系统已将数据段分配到 13D90H 单元开始的存储区。本程序较短, 线条下面的反汇编代码是系统内存区中存放的随机无效指令, 如果程序较长可连续使用 U 命令; 或用 “U[起始地址] [结束地址]” 显示指定偏移地址的反汇编代码 (见图 2-6)。

```

-U 0 E
13DA:0000 B8D913      MOV     AX,13D9
13DA:0003 8ED8        MOV     DS,AX
13DA:0005 B00000      MOV     DX,0000
13DA:0008 B409        MOV     AH,09
13DA:000A CD21        INT     21
13DA:000C B44C        MOV     AH,4C
13DA:000E CD21        INT     21

```

图 2-6 反汇编窗口 (二)

2) 查看和修改寄存器: R 命令 (见图 2-7)。

```

R
AX=0C1E BX=0000 CX=0050 DX=0000 SP=0020 BP=0000 SI=0000 DI=0000
DS=0C1E ES=0C08 SS=0C10 CS=0C1C IP=0005  NU UP EI PL NZ NA PO NC
0C1C:0005 B00000      MOV     DX,0000
R AX
AX=0C1E
11234
R
AX=1234 BX=0000 CX=0050 DX=0000 SP=0020 BP=0000 SI=0000 DI=0000
DS=0C1E ES=0C08 SS=0C10 CS=0C1C IP=0005  NU UP EI PL NZ NA PO NC
0C1C:0005 B00000      MOV     DX,0000

```

图 2-7 R 命令窗口

R 命令有两种用法：直接键入 R 将显示 CPU 所有寄存器和标志位；在 R 后跟寄存器名，则显示寄存器的内容，在冒号后可键入新值，再用 R 命令就可看到修改后的内容了。R 命令的最后一行是代码段当前指令的反汇编。所谓反汇编是将二进制的机器指令显示成汇编指令。该行由三部分组成：最左边 0C1C:0005 表示该指令所在单元的逻辑地址，即 CS:IP；中间 BA0000 表示该指令的机器码；最右边显示的是汇编指令 MOV DX, 0000。R 命令的第二行右边显示的是 CPU 标志寄存器 FLAG 各标志位的状态，见表 2-1。

表 2-1 debug 下 FLAG 标志位的状态

标 志 位								状 态
OF	DF	IF	SF	ZF	AF	PF	CF	
OV	DN	EI	NG	ZR	AC	PE	CY	表示各标志位位置
NV	UP	DI	PL	NZ	NA	PO	NC	表示各标志位复位

3) 单步执行：T 命令。在执行 T 命令前应确认当前指令的 IP 地址，可用 U 命令查看当前的反汇编指令地址或用 R 命令查看寄存器状态 IP。

用 -R IP 可查看和修改当前 IP 的值，如要修改在“:”后输入新值，若不需要修改直接按回车。从图 2-8 所示状态可知，执行 T 命令前 DS 寄存器的值为 13C9，执行 T 命令后 DS 寄存器的值更新为 13D9。



图 2-8 T 命令窗口

4) 查看存储单元内容：D 命令。在用 D 命令查看存储单元内容之前应先加载段基址（见图 2-9），这里先用 T 命令单步执行前两条指令，用以加载数据段基址，然后用 D 命令显示内存区数据段的内容。

图 2-9 中分别用 D 命令显示了数据段、代码段、堆栈段、数据段的内存区数据，D 命令后直接跟偏移量则默认显示数据段的内容，如不跟结束地址，则显示 8 行。D 命令状态中最左边部分显示的是对应段的逻辑地址，中间部分显示的是对应段的内存区数据，最右边部分是对应单元的 ASCII 字符。如本程序中数据段定义了“hello, world!”，其对应单元的 ASCII 码分别为 68、65 等，其偏移地址分别是 0000、0001，一般把程序中 DB 伪指令前面的标识符称为变量，如参考程序中 MSG 为字节变量，其有效地址是 0000。

5) 程序执行：G 命令。G 命令可连续地执行指令一直到所给出的断点为止。图 2-10 中，地址 000CH 设置了程序断点，程序执行到此指令，暂停并显示寄存器的状态。

```

-D ds:0 f 60 65 6C 6C 6F 2C 77 6F-72 6C 64 21 0D 0A 24 00 hello,world!.!
13D9:0000 -D cs:0 f B8 D9 13 8E D8 BA 00 00-B4 09 CD 21 B4 4C CD 21 .....!.!.!
13DA:0000 -D ss:0 1f 68 65 6C 6C 6F 2C 77 6F-72 6C 64 21 0D 0A 24 00 hello,world!..$.
13D9:0010 -D 0 B8 D9 13 8E D8 BA 00 00-B4 09 CD 21 B4 4C CD 21 .....!.!.!
13D9:0000 68 65 6C 6C 6F 2C 77 6F-72 6C 64 21 0D 0A 24 00 hello,world!..$.
13D9:0010 B8 D9 13 8E D8 BA 00 00-B4 09 CD 21 B4 4C CD 21 .....!.!.!
13D9:0020 8B E5 5D C3 83 3E 56 07-20 72 0A B8 1C 04 50 E8 ..1..>0. f....P
13D9:0030 62 44 83 C4 02 B8 FF FF-50 B8 05 00 50 8D 86 7A bD.....P...P..2
13D9:0040 FE 50 E8 4B 10 83 C4 06-8B 1E 56 07 D1 E3 D1 E3 .P.K.....0....
13D9:0050 01 3A 21 8B 16 3C 21 89-87 BE 22 89 97 C0 22 80 .:~.<~".....
13D9:0060 3E 45 07 00 74 0A FF 36-56 07 E8 21 FC 83 C4 02 >E..t..60..!...
13D9:0070 FF 06 56 07 5E 8B E5 5D-C3 90 55 8B EC 81 EC 90 ..0.^..1..0....

```

图 2-9 D 命令窗口

```

-G c
hello,world!
AX=0924 BX=0000 CX=0020 DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=13D9 ES=13C9 SS=13D9 CS=13DA IP=000C NU UP EI PL NZ NA PO NC
13DA:000C B44C MOV AH,4C

```

图 2-10 G 命令窗口

6) 退出 debug; Q 命令。

6. 执行

在 DOS 命令行输入生成的可执行文件名即可执行该文件, 如图 2-11 所示。

```

c:\ 命令提示符
C:\>cd \8086\hello>hello
hello,world!
C:\>cd \8086\hello>

```

图 2-11 执行窗口

2.2 汇编语言程序设计实例

本节通过若干实例可进一步提高编写汇编语言程序的能力, 下述程序均用 MASM5.0 编译器进行编译链接。实验时应首先建立工作目录 (非中文), 以记事本或 emu8086 编辑下述的每一个源程序。以记事本编辑存盘时保存类型应选择“所有文件”, 文件名应保存为“*.asm”; 以 emu8086 编辑存盘时应注意其保存路径。由于 debug 调试指令都是命令行方式, 人机界面不直观, 下述程序段均可在 emu8086 中编辑调试, 调试通过后再用 MASM5.0 重新编译链接生成可执行文件, 以 DOS 命令行方式执行并观察程序运行效果。

例 2-1 在屏幕的第 8 行 20 列开始显示如图

2-12所示宽 40、高 5 行的红色窗口, 并在窗口的第 2 行 5 列显示: “hello world!”。

```
hello world!
```

图 2-12 窗口内显示字体

参考程序:

```

WINWIDTH   = 40                                ;窗口宽度
HEIGHT      = 5
WINTOP      = 8                                ;窗口左上角行号
WINLEFT     = 20                                ;窗口左上角列号
WINBOTTOM   = WINTOP + HEIGHT - 1              ;窗口右下角行号
WINRIGHT    = WINLEFT + WINWIDTH - 1           ;窗口右下角列号
COLOR       = 47H                              ;47H 红屏白字
DATA        SEGMENT
ASC         DB ' hello world! '
DATA        ENDS
CODE        SEGMENT
                ASSUME CS:CODE,DS:DATA          ;ASSUME 伪指令指定段寄存器与对应段名

START:  MOV AX,DATA
        MOV DS,AX                              ;加载数据段基址
        MOV AX,0B800H                          ;显示缓冲区首址
        MOV ES,AX
        MOV BX,WINTOP * 160                    ;BX 用作行指针
LP0:    MOV SI,WINLEFT * 2                      ;SI 用作列指针
LP:     MOV AH,COLOR
        MOV ES:[BX + SI],AX                    ;颜色属性送到显示缓冲区
        CMP SI,WINRIGHT * 2                    ;是否到窗口最后一列
        JE  NEXTLINE
        INC SI
        INC SI
        JMP LP
NEXTLINE: CMP BX,WINBOTTOM * 160                ;是否到窗口最后一行
        JE  ASCI
        ADD BX,160
        JMP LP0
ASCI:   MOV BX,(WINTOP + 2) * 160
        MOV SI,(WINLEFT + 5) * 2
        MOV AH,COLOR
        MOV CX,12
        MOV DI,0
LPASC:  MOV AL,ASC[DI]
        MOV ES:[BX + SI],AX                    ;颜色属性送到显示缓冲区
        INC SI

```

```

        INC SI
        INC DI
        LOOP LPASC
        MOV AH,4CH
        INT 21H
CODE ENDS
        END START

```

例 2-2 以红色前景在屏幕的第 4 行 20 列开始显示如图 2-13 所示背景字体。

1) 参考程序。

```

HEIGHT      = 16
WINTOP      = 4           ;窗口左上角行号
WINLEFT     = 20          ;窗口左上角列号
WINBOTTOM   = WINTOP + HEIGHT - 1 ;窗口右下角行号
COLOR       = 47H         ;47H 红屏白字

```



图 2-13 背景字体

```

DATA      SEGMENT
;取模工具 PCTOOL 阴码,顺向,逐行式
ASC DB 008H, 000H, 008H, 07CH, 008H, 044H, 0FFH, 044H
    DB 008H, 07CH, 008H, 044H, 008H, 044H, 07EH, 044H;
    DB 042H, 07CH, 042H, 044H, 042H, 044H, 042H, 044H;
    DB 07EH, 044H, 042H, 044H, 002H, 094H, 001H, 008H;"胡",0
DATA      ENDS
STACK SEGMENT  STACK
        DW 100 DUP(?)
STACK ENDS
CODE      SEGMENT
        ASSUME CS:CODE,DS:DATA
START:    MOV AX,DATA
          MOV DS,AX           ;加载数据段基址
          MOV AX,0B800H       ;显示缓冲区首址
          MOV ES,AX
          MOV BX,WINTOP * 160 ;BX 用作行指针
          MOV DI,0
LP0:      MOV SI,WINLEFT * 2   ;SI 用作列指针
          MOV CX,16           ;16 列
LP:       MOV DX,WORD PTR ASC[DI]
          XCHG DH,DL          ;交换汉字点阵高低字节

```

```

LP1:    SHL DX,1
        JC  LP2
        MOV AH,0
        JMP LP3
LP2:    MOV AH,COLOR
LP3:    MOV BYTE PTR ES:1[BX+SI],AH ;颜色属性送到显示缓冲区
        ADD SI,2
        MOV BYTE PTR ES:1[BX+SI],AH ;颜色属性送到显示缓冲区
        ADD SI,2
        LOOP LP1
NEXTLINE: CMP BX,WINBOTTOM * 160 ;是否到窗口最后一行
        JE  EXI
        ADD DI,2
        ADD BX,150
        JMP LP0
EXI:    MOV AH,4CH
        INT 21H
CODE ENDS
        END START

```

2) 汉字点阵。程序中的汉字点阵可用 PCTOOL 字模提取工具生成,取模方式为阴码、顺向、逐行式。

3) 练习。在屏幕上将姓名逐个字显示输出,要求每个字显示 1s 再显示下一个汉字,并不断循环。用 masm5.0 编译连接生成 my_name.exe 可执行文件。

参考程序:

```

HEIGHT      = 16
WINTOP       = 4 ;窗口左上角行号
WINLEFT      = 20 ;窗口左上角列号
WINBOTTOM    = WINTOP + HEIGHT - 1 ;窗口右下角行号
COLOR        = 47H ;47H 红屏白字
DATA SEGMENT
ADDR DW 0
;PCTOOL 阴码、顺向、逐行式
ASC DB 008H, 000H, 008H, 07CH, 008H, 044H, 0FFH, 044H;
    DB 008H, 07CH, 008H, 044H, 008H, 044H, 07EH, 044H;
    DB 042H, 07CH, 042H, 044H, 042H, 044H, 042H, 044H;
    DB 07EH, 044H, 042H, 044H, 002H, 094H, 001H, 008H;"胡",0
    DB 000H, 040H, 078H, 040H, 00BH,0F8H, 010H, 048H;
    DB 017H, 0FEH, 020H, 048H, 07BH,0F8H, 008H, 040H;
    DB 04BH, 0FCH, 048H, 040H, 028H, 040H, 017H, 0FCH;

```

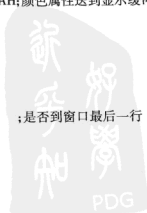


```

    DB 028H, 040H, 046H, 040H, 081H, 0FEH, 000H, 000H;"建",1
    DB 020H, 040H, 010H, 040H, 010H, 040H, 007H, 0FEH;
    DB 084H, 044H, 054H, 040H, 054H, 040H, 017H,0F8H;
    DB 025H, 008H, 024H, 090H, 0E4H, 090H, 024H, 060H;
    DB 028H, 060H, 028H, 098H, 031H, 00EH, 026H, 004H;"波",2
    ASC_END    = $
DATA        ENDS
STACK SEGMENT    STACK
    DW 100 DUP(?)
STACK ENDS
CODE        SEGMENT
    ASSUME CS:CODE,DS:DATA
START:  MOV AX,DATA
        MOV DS,AX                ;加载数据段基址
        MOV AX,0B800H           ;显示缓冲区首址
        MOV ES,AX

AGAIN:  MOV DI,0
NEXT:   MOV BX,WINTOP*160        ;BX 用作行指针
LP0:    MOV SI,WINLEFT*2         ;SI 用作列指针
        MOV CX,16               ;16 列
LP:     MOV DX,WORD PTR ASC[DI]
        XCHG DH,DL              ;交换汉字点阵高低字节
LP1:    SHL DX,1
        JC  LP2
        MOV AH,07H
        JMP LP3
LP2:    MOV AH,COLOR
LP3:    MOV BYTE PTR ES:1[BX+SI],AH;颜色属性送到显示缓冲区
        ADD SI,2
        MOV BYTE PTR ES:1[BX+SI],AH;颜色属性送到显示缓冲区
        ADD SI,2
        LOOP LP1
NEXTLINE: ADD DI,2
        CMP DI,OFFSET ASC_END
        JE  AGAIN
        CMP BX,WINBOTTOM*160   ;是否到窗口最后一行
        JE  CLEA
        ADD BX,160
        JMP LP0

```



```

CLEA:  CALL DELAY
        JMP  NEXT
DELAY  PROC  NEAR                                ;延时
        PUSH  BX
        PUSH  CX
        MOV  BX,600H
DEL1:  MOV  CX,4690H
DEL2:  PUSHF
        POPF
        LOOP DEL2
        MOV  AH,0BH                            ;返回到 DOS
        INT  21H
        DEC  BX
        JNZ  DEL1
        POP  CX
        POP  BX
        RET
DELAY  ENDP
CODE  ENDS
      END START

```

例 2-3 用简化的程序段格式从键盘输入两个一位十进制数，并进行加法运算，如图 2-14 所示，相加后结果以蓝底黄字显示在屏幕上。



图 2-14 加法运算

从 MASM5.0 开始，提供了简化段定义结构。

使用简化段程序结构便于与高级语言接口。在本例中，.MODEL SMALL 是小型模式，在这种小型模式的简化段程序结构中，可以有一个代码段、一个数据段，每段不大于 64KB 且数据段和堆栈段、附加段是共用的。

(1) 设计思路

- 1) 键盘键入用 DOS 中断调用 1 号功能；显示采用写显存方法。
- 2) 经非压缩 BCD 码加法调整指令 AAA 调整后会将 AL 的高 4 位清 0，因此键盘键入的数字不必去掉 30H，可直接运算。

- 3) 用 BIOS 中断调用 INT 10H 的 3 号功能获得光标位置，让结果显示在光标处。

(2) 参考程序

```

.MODEL SMALL
.STACK 100
.CODE
START:

```

```

    MOV AH,1
    INT 21H

```

;键盘输入

```

MOV BL,AL                      ;保存第一个数
INT 21H                        ;输入第二个数
ADD AL,BL                      ;直接相加
MOV AH,0
AAA                            ;非压缩 BCD 码加法调整
ADD AX,3030H                   ;变为 ASCII 码
MOV SI,AX
MOV AX,0B800H                  ;显存首址
MOV ES,AX
MOV AH,3
MOV BH,0
INT 10H                        ;INT 10H 获得光标位置,行号→DH,
                                ;列号→DL

MOV AL,160
MUL DH
ADD AL,DL                      ;计算当前光标所对应的显存地址
MOV BX,AX
MOV AX,SI
MOV BYTE PTR ES:[BX+2],3DH     ;显示“=”号
MOV ES:[BX+4],AH
MOV BYTE PTR ES:[BX+5],1EH     ;蓝底黄字
MOV ES:[BX+6],AL
MOV BYTE PTR ES:[BX+7],1EH
MOV AH,4CH                     ;返回 DOS
INT 21H
END START

```

例 2-4 CMOS 实时钟。

CMOS RAM 状态寄存器的位 7 是计时更新标志位,若为 1,表示实时钟正在计时,若为 0 表示实时钟信息可以读出,通过读取 RT/CMOS 状态寄存器的位 7 判断系统时钟。

参考程序:

```

CMOS_PORT EQU 70H              ;CMOS 端口地址
CMOS_REGA EQU 0AH              ;CMOS 状态端口地址
UPDATE_F EQU 80H               ;时钟更新标志
CMOS_SEC EQU 00H               ;读秒命令字
CMOS_MIN EQU 02H               ;读分命令字
CMOS_HOUR EQU 04H
DATA SEGMENT
SECOND DB ?
MINUTE DB ?

```

```

    HOUR    DB ?
    TIME    DB '00:00:00'           ;待显示字符
    DATA   ENDS
    STACK   SEGMENT STACK
            DW 100 DUP(?)
    STACK   ENDS
    CODE    SEGMENT
            ASSUME CS:CODE,DS:DATA
    START:  MOV AX,DATA               ;数据段基址
            MOV DS,AX
    UIP:    MOV AL,CMOS_REGA
            OUT CMOS_PORT,AL         ;读状态命令字
            JMP $+2
            IN AL,CMOS_PORT+1
            TEST AL,UPDATE_F         ;时钟更新标志
            JNZ UIP
            MOV AL,CMOS_SEC
            OUT CMOS_PORT,AL         ;读秒命令字
            JMP $+2
            IN AL,CMOS_PORT+1
            MOV SECOND,AL
            MOV AL,CMOS_MIN
            OUT CMOS_PORT,AL         ;读分命令字
            JMP $+2
            IN AL,CMOS_PORT+1
            MOV MINUTE,AL
            MOV AL,CMOS_HOUR         ;读时命令字
            OUT CMOS_PORT,AL
            JMP $+2
            IN AL,CMOS_PORT+1
            MOV HOUR,AL
            MOV AL,SECOND             ;拆分秒
            AND AL,0FH
            OR AL,30H
            MOV TIME+7,AL
            MOV AL,SECOND
            MOV CL,4
            SHR AL,CL

```

```

OR AL,30H
MOV TIME+6,AL
MOV AL,MINUTE           ;拆分
AND AL,0FH
OR AL,30H
MOV TIME+4,AL
MOV AL,MINUTE
MOV CL,4
SHR AL,CL
OR AL,30H
MOV TIME+3,AL
MOV AL,HOURL           ;拆分时
AND AL,0FH
OR AL,30H
MOV TIME+1,AL
MOV AL,HOURL
MOV CL,4
SHR AL,CL
OR AL,30H
MOV TIME+0,AL
TODISP_INIT:MOV AX,0B800H           ;彩色显示缓冲区首址
MOV ES,AX
MOV CX,8           ;LOOP 循环次数
MOV BX,0           ;数组变量 TIME 下标
MOV DI,654H           ;待显示字符显存有效地址
TODISP:MOV AL,TIME[BX]           ;取数组变量 TIME 的一个字节
MOV ES:[DI],AL           ;送显存
INC BX           ;TIME 下标加 1
ADD DI,2           ;指向下一个显存位置
LOOP TODISP
MOV AH,0BH           ;返回到 DOS
INT 21H
JMP UIP
CODE ENDS
END START

```



第3章 Proteus 操作基础

Proteus 是英国 Labcenter 公司开发的电路分析与实物仿真及印制电路板设计软件。它主要由 ISIS 和 ARES 两部分组成, ISIS 的主要功能是原理图设计及电路原理图的交互仿真, ARES 主要用于印制电路板的设计。ISIS 提供的 Proteus VSM (Virtual System Modeling) 实现了混合式的 SPICE 电路仿真, 它将虚拟仪器、高级图表应用、CPU 仿真, 以及第三程序开发与调试环境有机地结合起来, 在搭建硬件模型之前即可在个人计算机上完成原理图设计、电路分析及程序代码实时仿真、测试及验证。

从 Proteus 7.5 版开始增加了对 8086 CPU 的仿真, 本书的所有实例是基于 Proteus 7.5 版进行测试的。Proteus VSM 8086 交互仿真器必将成为微机原理与接口技术课程较为理想的教学实验平台。

3.1 Proteus ISIS 操作界面

Proteus ISIS 的操作界面如图 3-1 所示。它主要包括标题栏, 菜单栏, 命令工具条, 模

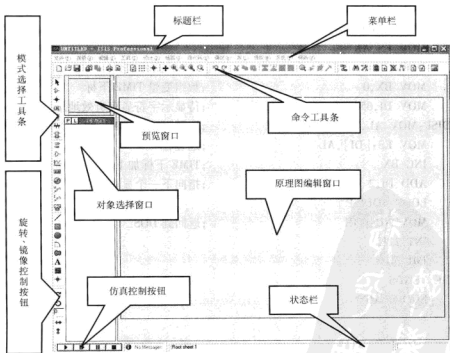


图 3-1 Proteus ISIS 操作界面

式选择工具条, 预览窗口, 对象选择窗口, 原理图编辑窗口, 旋转、镜像控制按钮, 仿真控制按钮和状态栏。

其中, 标题栏用于指示当前设计的文件名; 状态栏用于指示当前鼠标的坐标值; 原理图编辑窗口用于放置元器件, 进行连线, 绘制原理图; 预览窗口用于显示原理图编辑窗口的缩略图或用于预览选中对象。

3.1.1 ISIS 菜单栏

ISIS 菜单栏主要有: 文件、查看、编辑、工具、设计、绘图、源代码、调试、库、模板、系统和帮助菜单栏。

3.1.2 ISIS 命令工具条

ISIS 命令工具条包含四部分: 分别为文件工具条、编辑工具条、查看工具条和设计工具条, 工具条的显示与隐藏可通过“查看”菜单→“工具条”菜单实现。勾选或去掉相应工具条前面的“√”, 即可实现工具条的显示或隐藏。工具条的每个按钮, 都对应一个具体的菜单命令。

图 3-2 所示的文件工具条从左至右分别为新建、打开、保存、导入区域、导出区域、打印和标记输出区域命令按钮。

图 3-3 所示的编辑工具条从左至右分别为撤消、重做、剪切、复制、粘贴、块复制、块移动、块旋转、块删除、从库中选取元器件、创建元器件、封装工具和分解命令按钮。



图 3-2 文件工具条



图 3-3 编辑工具条

图 3-4 所示的查看工具条从左至右分别为刷新显示、切换网格、切换原点、光标居中、放大、缩小、缩放到整图和缩放到区域命令按钮。

图 3-5 所示的设计工具条从左至右分别为切换到自动连线器、搜索选中元器件、属性分配工具、设计浏览器、新(根)页面、移除/删除页面、退到父页面、查看 BOM 报告、查看电气报告、生成网表并传输到 ARES 命令按钮。



图 3-4 查看工具条



图 3-5 设计工具条

3.1.3 ISIS 模式选择工具条

窗口左边是含有三个组成部分的模式选择工具条, 主要包括主模式图标、部件模式图标和二维图形模式图标。上述图标用来确定原理图编辑窗口的编辑模式, 也就是选择不同的图标, 在编辑窗口鼠标将执行不同的操作。模式选择工具条没有对应的菜单项, 不能隐藏, 始终出现在窗口的最左端。各模式图标功能见表 3-1。

表 3-1 各模式图标功能

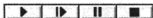
类别	图标	功 能
主模式图标		即时编辑任意选中的元器件
		选择元器件
		在原理图中放置连接点
		在原理图中放置或编辑连线标签
		在原理图中输入新的文本或编辑已有文本
		在原理图中绘制总线
		在原理图中放置子电路图或放置子电路元器件
部件模式图标		使对象选择器列出可供选择的各终端（如：输入、输出、电源等）
		使对象选择器列出 6 种常用的元器件引脚，用户也可从引脚库中选择其他引脚
		使对象选择器列出可供选择的各仿真分析图表（如：模拟图表、数字图表、A/C 图表等）
		对原理图进行分割仿真时采用此模式，用来记录前一步仿真的输出，并作为下一步仿真的输入
		使对象选择器列出可供选择的模拟和数字激励源（如：直流电源、正弦激励源、稳定状态的逻辑电平、数字时钟信号源和任意逻辑电平序列等）
		在原理图中添加电压探针，用来记录原理图中该探针处的电压值，可记录模拟电压值或者数字电压的逻辑值和时长
		在原理图中添加电流探针，用来记录原理图中该探针处的电流值，只能用于记录模拟电路的电流值
二维图形模式图标		使对象选择器列出可供选择的虚拟仪器（如：示波器、逻辑分析仪、定时计数器等）
		使对象选择器列出可供选择的连线的各种样式，用于在创建元器件时画线或直接在原理图中画线
		使对象选择器列出可供选择的方框的各种样式，用于在创建元器件时画方框或直接在原理图中画方框
		使对象选择器列出可供选择的圆的各种样式，用于在创建元器件时画圆或直接在原理图中画圆
		使对象选择器列出可供选择的弧线的各种样式，用于在创建元器件时画弧线或直接在原理图中画弧线
		使对象选择器列出可供选择的任意多边形的各种样式，用于在创建元器件时画任意多边形或直接在原理图中画多边形
		使对象选择器列出可供选择的字符文本的各种样式，用于在原理图中添加注释或说明
		用于从符号库中选择符号元器件
		使对象选择器列出可供选择的各标记类型，用于创建或编辑元器件、符号、各终端引脚时产生各种标记图标

3.1.4 ISIS 旋转、镜像控制按钮

对于具有方向性的对象，ISIS 提供了旋转、镜像控制按钮，来改变对象的方向（见表 3-2），ISIS 原理图编辑窗口中各对象只能以 90° 间隔来改变方向。

表 3-2 旋转、镜像控制按钮功能

类别	图标	功 能
旋转		对原理图编辑窗口选中的对象以 90° 间隔顺时针旋转（或在对象放入原理图之前）
		对原理图编辑窗口选中的对象以 90° 间隔逆时针旋转（或在对象放入原理图之前）
编辑		直接输入 90°、180°、270° 逆时针旋转
镜像		对原理图编辑窗口选中的对象或放入原理图之前的对象以 Y 轴为对称轴进行水平镜像操作
		对原理图编辑窗口选中的对象或放入原理图之前的对象以 X 轴为对称轴进行垂直镜像操作



3.1.5 ISIS 仿真控制按钮

仿真控制按钮主要有开始仿真、单步仿真、暂停和结束仿真四个命令按钮，如图 3-6 所示。

图 3-6 仿真控制按钮

3.2 Proteus ISIS 电路原理图设计

3.2.1 ISIS 鼠标使用规则

在 ISIS 中，鼠标操作与传统的方式不同，右键选取、左键编辑或移动。

- ① 右键单击——选中对象，此时对象呈红色；再次右键单击已选中对象，即可删除该对象。
- ② 右键拖曳——框选一个块的对象。
- ③ 左键单击——放置对象或对选中的对象编辑属性。
- ④ 左键拖曳——移动对象。
- ⑤ 滚轮单击——拖动窗口。
- ⑥ 滚轮转动——向前转动放大窗口，向后转动缩小窗口。

3.2.2 原理图设计

以图 3-7 所示的发光二极管（LED）电路为例，简要地介绍 Proteus ISIS 仿真电路原理图的基本设计过程。

1. 创建新文档

启动 ISIS，系统自动建立一个空的文档，默认文件名为“UNTITLED.DSN”，选择文件存储路径和文件名并保存。

2. 添加元器件到对象选择器

添加元器件到对象选择器，本例用到的元器件见表 3-3。

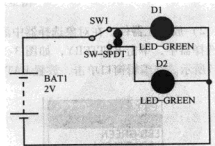


图 3-7 发光二极管（LED）电路

表 3-3 电路中所用元器件

元器件名称	数值
BATTERY	2V
LED - GREEN	2V
LED - YELLOW	2V
SW - SPDT	

1) 选择主模式工具条中的  图标, 并单击如图 3-8 所示对象选择器中的 P 按钮, 弹出如图 3-9 所示的选择元器件对话框。在关键字编辑框中输入待查寻的元器件类型 BATTERY 或元器件值, 通过勾选完全匹配复选框实现精确查找, 否则模糊查找。在元器件列表中选择 BATTERY 并单击“确定”按钮或双击, 将 BATTERY 元器件添加到对象选择器。用同样的方法将其余元器件添加到对象选择器。



图 3-8 对象选择器中的 P 按钮

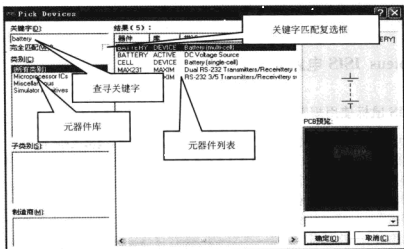


图 3-9 选择元器件对话框

2) 放置元器件。在对象选择器中添加元器件之后, 就要在原理图中放置元器件。在对象选择器中, 单击 BATTERY, 如图 3-10 所示, 同时预览窗口将会显示所选元器件, 如图 3-11 所示。在编辑窗口单击, 放置 BATTERY。

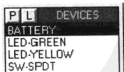


图 3-10 从对象选择器中选择 BATTERY



图 3-11 预览窗口显示的 BATTERY

依上述方法将单刀双掷开关和 LED 放置到原理图编辑窗口, 如图 3-12 所示。

3) 移动元器件。在编辑窗口中右键单击选中对象，并按住左键拖动到合适位置，然后在编辑窗口的空白处单击，撤消对象的选中状态。

4) 删除元器件。对于误放置的元器件，右键双击该对象即可删除，如果不小心误删除操作，可通过编辑工具栏中的“撤消”按钮进行恢复。

5) 调整元器件方向。在编辑窗口中右键单击选中 LED - GREEN，使其高亮度显示，单击“旋转”按钮，调整 LED - GREEN 为水平方向，并依同样的方法调整另一只 LED 的方向，最后调整后的元器件效果如图 3-12 所示。

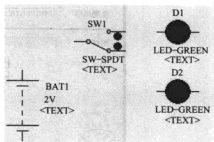


图 3-12 调整后的元器件效果

6) 编辑元件属性。在编辑窗口中右键单击选中的对象，弹出快捷菜单，选择“编辑属性”，弹出“编辑元件”对话框。如图 3-13 所示为 BATTERY 属性编辑对话框。

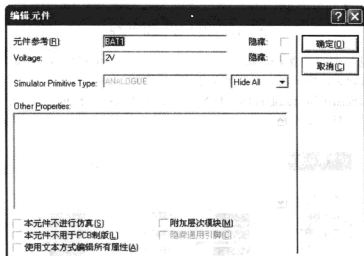


图 3-13 BATTERY 属性编辑对话框

“元件参考”表示元器件在原理图中的参考号，原理图中每个元器件应有唯一的参考名称。“Voltage”表示该元器件的电压标称值，本例中将电压改为 2V，单击“确定”按钮，结束对元器件的编辑。

图 3-13 所示的“编辑元件”对话框中的两个隐藏复选框决定着它前面的各项是否在原理图中显示，选中复选框，前面编辑窗口中的内容则不会显示在原理图中，这样可使原理图紧凑简洁。

7) TEXT 标号。对比图 3-7 和图 3-12，在图 3-12 中每个元器件下面都有一个 <TEXT> 框，为了保持原理图的美观，可把每个 <TEXT> 去掉，需要对元器件的 TEXT 属性进行编辑。左键单击选中元器件，单击 <TEXT> 框，进入到元器件属性编辑对话框，并且单击“Style”选项卡，如图 3-14 所示。

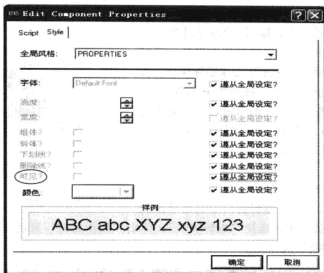


图 3-14 元器件属性编辑对话框

取消“可见”项的“遵从全局设定”属性，“可见”将由灰色不可编辑状态变为黑色，同样取消其勾选状态，将<TEXT>框图从原理图中隐藏。

若想去掉原理图中所有元器件的<TEXT>框，可以选择“模板”菜单中的“设置设计默认值”菜单项，打开如图 3-15 所示的“设置默认规则”对话框，去掉“显示隐藏文本”后的复选框。



图 3-15 “设置默认规则”对话框

3. 连线

在 ISIS 环境下，只需要直接单击两个元器件的连接点，ISIS 即可自动定出走线路径并完成两连接点的连线操作。如果想自己决定走线路径，只需单击第一个元器件的连接点，然后在希望放置拐点的地方单击，最后单击另一个元器件的连接点即可；放置拐点的地方鼠标

会呈“×”样式，如图3-16所示。左键单击选中连线，当鼠标移动到连线变为十字箭头时可拖动连线。按照上述方法，连接各个元器件，连接后的原理图如图3-17所示。

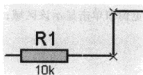


图3-16 放置直线拐点

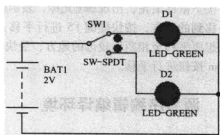


图3-17 连接后的原理图

4. 生成网络表文件

ISIS 默认的网络表输出格式是 SDF (Schematic Description Format)。同时，ISIS 支持另外 10 种网络表输出格式以便在第三方编辑软件下运行。选择“工具”菜单下的“编译网络表”，弹出网络表编译对话框，按照默认设置单击“确定”按钮生成网络表。

5. 电气规则检查

电气规则检查是利用电路设计软件对用户设计好的电路进行测试，以便能检查出人为的错误或疏忽，执行测试后，ISIS 会自动生成电路中各种可能存在错误的报表。例如，空的引脚、没有连接的电源、没有连接的网络标号等。选择“工具”菜单下的“电气规则检查”，对原理图进行电气规则检查，生成报告单，如图3-18所示，在该报告单中，系统报告原理图中没有错误。

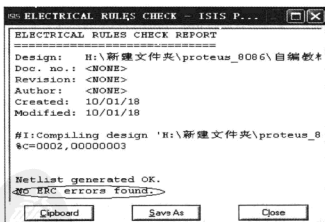


图3-18 电气规则检查报告单

6. 窗口缩放方式

鼠标移动到需要缩放的地方，滚动滚轮缩放；鼠标移动到需要缩放的地方，按 F6 键放大，F7 键缩小；按下 Shift 键，鼠标左键拖拽出需要放大的区域；使用工具条中的 Zoom in (放大)、Zoom Out (缩小)、Zoom All (全图)、Zoom Area (放大区域) 进行操作；按 F8 键可以在任何时候显示整张图纸；使用 Shift、Zoom 及滚轮均可应用于预览窗口，在预览窗

口进行操作, 编辑窗口将有相应变化。

7. 窗口平移方式

按下鼠标滚轮, 出现光标, 表示图纸已经处于提起状态, 可以进行平移; 鼠标置于要平移到的地方, 按快捷键 F5 进行平移; 按下 Shift 键, 在编辑窗口移动鼠标, 进行平移; 如果想要平移至相距比较远的地方, 最快捷的方式是在预览窗口单击显示该区域; 使用工具栏 Pan 按钮进行平移。

3.3 源程序编辑编译环境

3.3.1 编译器

支持 Proteus 8086 交互仿真的汇编程序和编译器是非常广泛的, 在表 3-4 中所列出来的工具都是已经通过测试的。不同版本的调试信息格式存在细微的差异, 推荐使用与表 3-4 中所列出的工具一样的版本。

表 3-4 Proteus VSM 8086 模型可用编译器

编译器	许可证	调试格式	网络站点
MASM32	免费	Codeview	www.masm32.com
Borland Turbo Assembler (TASM)	收费	Borland	
Digital Mars C++ Compiler	免费	Codeview	www.digitalmars.com
Microsoft C/C++ Compiler 7.00	收费	Codeview	www.microsoft.com
Borland C++ Compiler for Windows 5.02	收费	Borland	www.codegear.com

Proteus VSM 8086 模型能直接加载 BIN、COM 和 EXE 格式的文件到内部 RAM 中去, 而不需要 DOS, 并且允许对 Microsoft (Codeview) 和 Borland 格式中包含了调试信息的程序进行源和/或反汇编级别的调试。所有的调试格式允许源码级调试和全局变量的观察, 但是只有 Borland 格式支持对局部变量的观察。

Proteus VSM 8086 模型虽然加载的是 EXE 和 COM 格式的文件, 但是在没有 IBM PC BIOS 或者 MS-DOS 存在的情况下运行程序。它们不能在标准的 RTL (Run-Time Libraries) 下被编译, 因为标准的编译器的 RTL 利用的是 BIOS 和 MS-DOS 的函数调用。不同汇编程序的编译器对调试信息的生成有特定的选项。所有的例子将生成没有 RTL 但带有调试信息的 EXE 或 COM 文件, 所以它们能够加载到 Proteus VSM 8086 内部存储器中进行调试。

本教程中的所有汇编程序示例都是用 MASM32 编译器汇编、生成的 EXE 文件。每个示例下的源程序文件都统一命名为“sample.asm”。这样做的好处是不用再重写编译与连接的批处理文档。有关其他编译器的使用请查阅 Proteus VSM Model Help 的帮助文档。

MASM32 汇编器使用简化段定义格式, 源程序格式如下:

```

; SAMPLE.ASM          ; 文件名
.MODEL SMALL           ; 所有数据段都放在一个 64KB 的数据段内, 所有代码都放在
                        ; 另一个 64KB 的代码段内, 数据和代码都是近访问
.8086                  ; 8086 指令系统

```

```
. STACK ; 默认堆栈长度为 1KB
. CODE ; 代码段
. STARTUP ; 程序入口点
    MOV DX, 0020H
    MOV AL, 35H
    OUT DX, AL
END_ LOOP:
    JMP END_ LOOP
```

```
. DATA ; 数据段
END
```

编译和连接的批处理文档 BUILD. BAT 内容如下:

```
ML /c /Zd /Zi sample.asm
link16 /CODEVIEW sample.obj, sample.exe,, nul.def
```

第 1 行命令的作用是编译 sample.asm 源程序; 第 2 行命令的作用是连接 sample.obj 并生成 sample.exe 文件。

3.3.2 环境变量设置

MASM32 编译程序“ML.EXE”和连接程序“link16.exe”默认安装路径是 C:\masm32\bin, 一般情况下与当前工作路径不一致, 在编译和连接时可能会报错找不到路径, 这就要求要在 Windows 环境变量 Path 下添加“C:\masm32\bin”可执行程序的路径, 方法如下: 右键单击“我的电脑”, 选择“属性”, 弹出“系统属性”对话框, 选择“高级”选项卡, 单击“环境变量”按钮, 弹出“环境变量”对话框。选择“用户变量”, 单击“新建”或“编辑”命令, 弹出“编辑用户变量”对话框, 在“变量值”编辑框前面添加“C:\masm32\bin;”, 如图 3-19 所示。

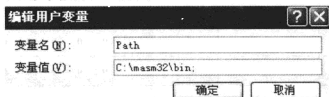


图 3-19 环境变量设置

第4章 接口技术实验

目前, Proteus 对 8086 CPU 的虚拟仿真仅能提供最小模式下的仿真。书中所有实例都是基于最小模式下开发完成的。8086 最小模式下的总线结构及 I/O 地址分配如图 4-1 所示。为减少重复, 每一个实例都略去了图 4-1 所示的总线结构和 I/O 地址译码部分。分析图 4-1 得到表 4-1 的 I/O 端口地址。

表 4-1 I/O 端口地址

[illegible]

4.1 74LS273 输出口控制 LED

4.1.1 74LS273 输出口控制循环彩灯

本例用两片 74LS273 输出接口控制 16 只 LED，高、低 4 位交替闪烁实现一路循环彩灯，接口电路如图 4-2 所示。由于仅限于仿真，电路中没加限流电阻，在实际应用中应考虑加限流电阻或增加驱动器。

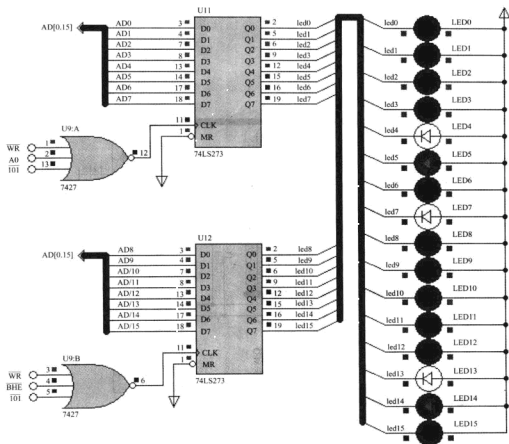


图 4-2 74LS273 输出口控制循环彩灯的接口电路

为了比较清楚地了解 Proteus 仿真实验中源程序到可执行程序的生成过程，先从 Proteus 的帮助文档开始本节的实验过程。

- 1) 首先建立工作目录，例如在 D 盘建立当前工作目录“273_caideng”。
- 2) 执行【程序】→【Proteus 7 Professional】→【Proteus VSM Model Help】→【8086 Model】(见图 4-3)，打开 Intel 8086 Processor Model 帮助文档。

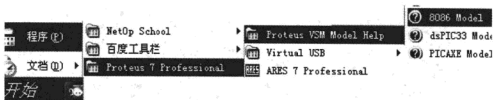


图 4-3 Proteus VSM 8086 帮助文档路径

3) 在帮助文档中单击打开 Supported Assemblers and Compilers (可支持的汇编和编译器), 找到 Creating a .EXE file with MASM32 (用 MASM32 汇编及编译器生成 EXE 文件)。

4) 打开 MASM32 Editor 应用程序编辑器, 复制 SAMPLE.ASM 下的文本 (下述黑体字) 至 MASM32 Editor 编辑器窗口, 并以 “sample.asm” 存盘至当前工作目录 “273_caideng”。在本章后续的实例中都以 “sample.asm” 作为源程序的文件名, 这样就不用重新编写或更改 “BUILD.BAT” 批处理文档, 可以直接从帮助文档中复制即可。

; SAMPLE.ASM (源程序文件名, 扩展名为 .ASM)

.MODEL SMALL

.8086

.STACK

.CODE

.STARTUP

MOV DX, 0020H

MOV AL, 35H

OUT DX, AL

END_ LOOP; JMP END_ LOOP

.DATA

END

5) 在 MASM32 Editor 应用程序窗口中新建另一文档, 复制 BUILD.BAT 下的文本 (见黑体字) 至 MASM32 Editor 编辑器窗口, 并以 “BUILD.BAT” 存盘至当前工作目录。

; BUILD.BAT (汇编和连接的批处理文件, 扩展名为 .BAT)

ml /c /Zd /Zi sample.asm

link16 /CODEVIEW sample.obj, sample.exe, , , nul, def

6) 执行 MASM32 Editor 应用程序 File 菜单下的 Cmd Prompt 命令, 转至 DOS 当前工作目录。执行 build 批处理文档, 完成编译和连接。如果操作正确应没有编译和连接错误, 如图 4-4 所示, 否则说明操作不正确。

7) 查看当前工作目录 “D: \ 273_caideng”, 会发现目录下多了几个文档, sample.asm 是汇编语言源程序文档, build.bat 是对源程序执行编译和连接的批处理文档。Sample.obj 是 ml 编译器生成的目标文件, sample.exe 是连接器 link16 生成的可执行文件。通过以上的介绍, 目的是掌握 MASM32 编辑器和汇编器的使用。

上述所复制的 “sample.asm” 程序不能实现交替循环彩灯, 按如下参考程序修改前面复

```

D:\273_caideng>build

D:\273_caideng>nl /c /Zd /Zi sample.asm
Microsoft (R) Macro Assembler Version 6.14.8444
Copyright (C) Microsoft Corp 1981-1997. All rights reserved.

Assembling: sample.asm

D:\273_caideng>link16 /CODEVIEW sample.obj, sample.exe, /, nul.def
Microsoft (R) Segmented Executable Linker Version 5.60.339 Dec 5 1994
Copyright (C) Microsoft Corp 1984-1993. All rights reserved.

D:\273_caideng>

```

图 4-4 编译和连接批处理

制建立的“sample.ase”源程序。

```

.MODEL SMALL
.8086
.STACK
.CODE
.STARTUP
AGAIN: MOV DX, 0200H           ; 273 地址
      MOV AX, 1111000011110000B ; 对应 0 电平的 LED 点亮
      OUT DX, AX
      CALL DELAY               ; 延时
      MOV AX, 0000111100001111B
      OUT DX, AX
      CALL DELAY               ; 延时
      JMP AGAIN
DELAY PROC NEAR               ; 延时子程序
      MOV BX, 500
      LP1: MOV CX, 469
      LP2: LOOP LP2
      DEC BX
      JNZ LP1
      RET
DELAY ENDP
.DATA
END

```

8) 在 DOS 下执行 build 批处理文档重新编译连接并保证没有编译和连接错误。

9) 复制目录“4.1.1_273_交替彩灯”中原理图“273_caideng.dsn”至当前工作目录。

10) 打开当前工作目录下的“273_caideng.dsn”原理图, 右键单击 8086 CPU 打开其属性窗口, 单击 program file 后的文件夹, 添加可执行程序 sample.exe, 单击“确定”按钮退出。

11) 全速执行观察仿真效果或单步执行调试程序。8086 CPU 提供了 4 个调试窗口: 源代码窗口、存储器窗口、寄存器窗口和变量窗口。暂停程序执行, 右键单击 8086 CPU, 选择最下面的选项: 8086, 勾选对应的选项, 单步执行观察 8086 CPU 寄存器和存储器数据的变化。另外, 在代码窗口还可设置程序断点、执行到光标行等调试方法。

4.1.2 单个移动点亮 LED

电路原理同图 4-2 所示, 通过编程实现 LED 由上到下移动点亮 (每次只亮一只)。

1) 参考程序。

```
.MODE LSMALL
.8086
.STACK
.CODE
.STARTUP
START: MOV DX, 0200H           ; 273 地址
      MOV AX, 111111111111110B ; 对应 0 电平的 LED 点亮
NEXT:  OUT DX, AX
      CALL DELAY               ; 延时
      ROL AX, 1                ; 左移 1 位
      JMP NEXT
DELAY PROC NEAR                ; 延时子程序
      MOV BX, 500
LP1:   MOV CX, 469
LP2:   LOOP LP2
      DEC BX
      JNZ LP1
      RET
DELAY ENDP
.DATA
END
```

2) 实验步骤。

① 新建工作目录, 复制目录“4.1.2_273_单亮彩灯”下的原理图“273_caideng.dsn”至当前工作目录。

② 打开 MASM32 Editor 编辑器, 新建“sample.asm”文档, 输入上述参考程序并存盘至当前工作目录; 新建“build.bat”编译连接批处理文档并存盘至当前工作目录。执行 File 菜单下的 CMD Prompt 命令转 DOS 当前工作目录, 执行 build 批处理直至没有编译和连接错误。

③ 打开“273_caideng.dsn”原理图, 添加可执行程序“sample.exe”, 全速执行观察仿真效果或单步执行调试程序(后续章节实验步骤略)。

4.1.3 逐个递增点亮 LED

电路原理同图 4-2 所示, 编程实现 LED 由上到下逐只递增点亮, 每隔一段时间点亮下一只, 但前面的不灭。参考程序:

```
. MODEL SMALL
. 8086
. STACK
. CODE
. STARTUP
START:  MOV DX, 0200H           ; 273 地址
AGAIN:  MOV CX, 16
        MOV AX, 111111111111110B ; 对应 0 电平的 LED 点亮
NEXT:   OUT DX, AX
        CALL DELAY             ; 延时
        SHL AX, 1              ; 左移 1 位
        LOOP NEXT
        JMP AGAIN
DELAY   PROC NEAR              ; 延时子程序
        PUSH CX                ; CX 入栈
        MOV BX, 500
        LP1: MOV CX, 469
        LP2: LOOP LP2
        DEC BX
        JNZ LP1
        POP CX                 ; CX 出栈
        RET
DELAY   ENDP
. DATA
END
```

4.1.4 LED 霓虹灯

前面 3 个实例中 LED 的变化只有两种状态, 本节利用查表的方法实现多状态 LED 闪烁变化, 这在现实生活中是比较常见的, 例如装饰城市夜景、色彩艳丽且变化多样的霓虹灯。本节仿真电路原理同图 4-2 所示, 参考程序:

```
. MODEL SMALL
. 8086
. STACK
```

```

.CODE
.STARTUP
AGAIN:  MOV  SI, OFFSET SITUATION
        MOV  DX, 0200H                ; 273 地址
NEXT:   MOV  AX, [SI]                 ; 查表取 LED 状态
        OUT  DX, AX
        CALL DELAY
        ADD  SI, 2                    ; 下一只 LED 状态地址
        CMP  SI, OFFSET SIT_ END
        JB  NEXT
        JMP  AGAIN
DELAY   PROC  NEAR                    ; 延时
        MOV  BX, 100
        LP1: MOV  CX, 469
        LP2: LOOP LP2
        DEC  BX
        JNZ  LP1
        RET
DELAY   ENDP
.DATA
SITUATION  DW      1111111111111111B
           DW      011111111111110B
           DW      001111111111100B
           DW      000111111111000B
           DW      000011111110000B
           DW      000001111100000B
           DW      000000111100000B
           DW      000000011000000B
           DW      000000001000000B
           DW      000000000000000B
           DW      000000011000000B
           DW      000000111100000B
           DW      000001111100000B
           DW      000111111110000B
           DW      001111111111000B
           DW      01111111111110B
           DW      11111111111111B

SIT_ END = $
END

```



4.1.5 LED 模拟交通灯

本例中的 12 只 LED 分成东西南北 4 组，每组用红黄绿 3 只 LED 模拟交通信号灯，外部输出接口用两片 74LS273，用其低 12 位控制 12 只 LED，如图 4-5 所示。

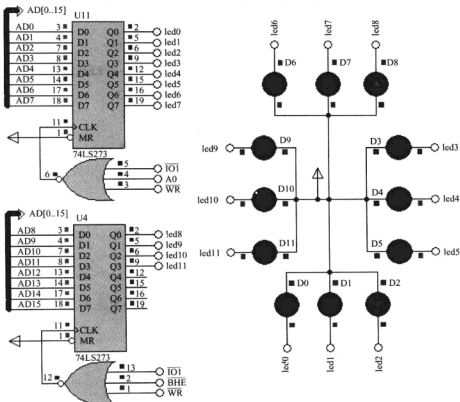


图 4-5 交通灯的电路原理

1) 编程实现。开始：南北红灯、东西绿灯亮，然后南北红灯、东西黄灯亮，然后南北绿灯、东西红灯亮，然后南北黄灯、东西红灯亮，返回开始。

2) 参考程序。

```

.MODEL SMALL
.8086
.STACK
.CODE
.STARTUP
START:  MOV AX, ALL_LIGHT
        MOV DX, 0200H
        MOV DX, AX
AGAIN:  MOV SI, OFFSET SITUATION
        MOV DX, 0200H

```

; 273 地址

PDG


```

NEXT:  MOV AX, [SI]
        OUT DX, AX
        CALL DELAY1
        ADD SI, 2                                ; 下一状态
        MOV AX, [SI]
        OUT DX, AX
        CALL DELAY2
        ADD SI, 2                                ; 下一状态
        CMP SI, OFFSET SIT_END
        JB NEXT
        JMP AGAIN
DELAY1  PROC NEAR                                ; 延时子程序 1
        MOV BX, 10000
LP1:    MOV CX, 469
LP2:    LOOP LP2
        DEC BX
        JNZ LP1
        RET
DELAY1  ENDP
DELAY2  PROC NEAR                                ; 延时子程序 2
        MOV BX, 500
LP1:    MOV CX, 469
LP2:    LOOP LP2
        DEC BX
        JNZ LP1
        RET
DELAY2  ENDP
. DATA
SITUATION DW 00000111110011110B                ; 南北红, 东西绿
S1         DW 0000101110101110B                ; 南北红, 东西黄
S2         DW 0000110011110011B                ; 南北绿, 东西红
S3         DW 0000110101110101B                ; 南北黄, 东西红
SIT_END = $
ALL_LIGHT EQU 0000001001001001B
END

```

4.2 74LS273 输出口控制 LED 数码管

4.2.1 74LS273 输出口控制 1 位共阴极 LED 数码管

用一片 74LS273 输出口控制 1 位共阴极 LED 数码管实现每隔一段时间从 0~9 显示, 然

后回 0，再递增到 9 反复。其电路原理如图 4-6 所示。

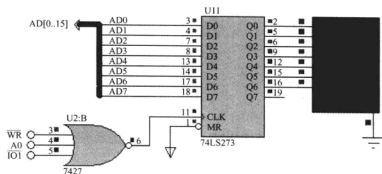


图 4-6 74LS273 输出口控制 1 位共阴极 LED 数码管的电路原理

由于仅限于仿真，电路中没加限流电阻，在实际应用中应考虑加限流电阻或增加驱动器。图 4-6 所示电路中使用的是共阴极数码管，当某段输出为 1 即高电平时对应的 LED 亮。LED 数码管显示原理如图 4-7 所示。根据数码管的显示原理可以写出十六进制 0~F 的显示段码以及数码管熄灭时的显示段码，例如在共阴极显示模式下，要想使数码管显示 0，hgfedcba 应输出 00111111B，即十六进制数 3FH；要想使数码管显示 1，hgfedcba 应输出 00000110B，即十六进制数 06H。共阳极显示段码和共阴极显示段码互为反码。十

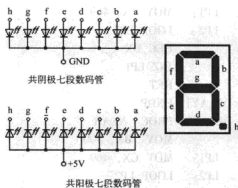


图 4-7 LED 数码管显示原理

六进制数共阴极和共阳极七段 LED 显示段码见表 4-2。

表 4-2 数码管显示段码

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	灭
共阴极	3FH	06H	5BH	4FH	66H	6DH	7DH	07H	7FH	6FH	77H	7CH	58H	5EH	79H	71H	00H
共阳极	C0H	F9H	A4H	B0H	99H	92H	82H	F8H	80H	90H	88H	83H	C6H	A1H	86H	8EH	FFH

根据以上分析，要想使数码管循环显示 0~9，只要每隔一段时间由 74LS273 输出对应数字的显示段码就可以，把显示段码存放在 TAB 表中，通过读取 TAB 表数据获取显示段码。

参考程序：

```
.MODEL SMALL
.8086
.STACK
.CODE
.STARTUP
```

```

AGAIN: MOV SI, OFFSET TAB          ; SI 指向显示段码表首址
        MOV DX, 0200H              ; 273 地址
NEXT:   MOV AL, [SI]                ; 取显示段码
        OUT DX, AL                  ; 输出显示段码
        CALL DELAY                  ; 延时
        ADD SI, 1                    ; 指向下一个显示段码
        CMP SI, OFFSET TAB_END      ; 是否显示完最后一个数
        JB NEXT                     ; 没有, 继续显示下一个数
        JMP AGAIN                    ; 显示完 9, 重新从 0 开始显示
DELAY PROC NEAR                      ; 延时子程序
        MOV BX, 500
LP1:    MOV CX, 469
LP2:    LOOP LP2
        DEC BX
        JNZ LP1
        RET
DELAY ENDP
. DATA
TAB     DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 6FH ; 共阴极
TAB_END = $
END

```

4.2.2 74LS273 输出口控制 2 位共阴极 LED 数码管

用两片 74LS273 输出口控制 2 位共阴极 LED 数码管实现秒表计数器, 每隔 1s 增 1, 递增到 59 然后回 0。其电路原理如图 4-8 所示。

1) 软件延时。

由于每条指令执行所需的 T 状态是固定的, 可以用多重循环的方法实现延时。设 $f_{osc} = 5\text{MHz}$, 则时钟周期 $T_c = 0.2\mu\text{s}$, 即 1 个 T 状态为 $0.2\mu\text{s}$ 。

```

DELAY PROC          ; 指令 T 状态
        MOV BX, 50   ; 4
DEL1:   MOV CX, 5882 ; 4
DEL2:   LOOP DEL2    ; 17/5
        DEC BX        ; 2
        JNZ DEL1      ; 16/4
        RET           ; 8
DELAY ENDP

```

延时 $T_d = |50 \times [17 \times (5882 - 1) + 5 + 6 + 16] - 12 + 4 + 8| \times 0.2\mu\text{s} = 1000.0004\text{ms} \approx 1\text{s}$ 。

2) 参考程序。

```
. MODEL SMALL
```

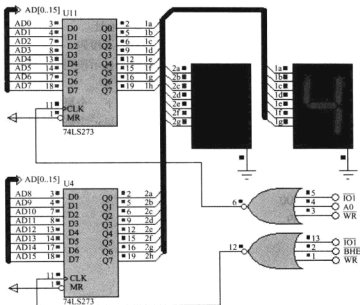


图 4-8 74LS273 输出口控制 2 位共阴极 LED 数码管的电路原理

. 8086

. STACK

. CODE

. STARTUP

MOV DX, 0200H

LOOP0: MOV BL, SEC

AND BX, 000FH ; 秒个位

MOV SI, BX

MOV AL, SITUATION [SI] ; 秒个位段显码

MOV BL, SEC

AND BX, 00F0H

MOV CL, 4

SHR BX, CL ; 秒十位

MOV SI, BX

MOV AH, SITUATION [SI] ; 秒十位段显码

OUT DX, AX

CALL DELAY

MOV AL, SEC

ADD AL, 1 ; 秒加 1

DAA ; 十进制调整

MOV SEC, AL

数字
显示
控制

PDG

```

CMP SEC, 60H                ; 是否增到 60
JB  LOOP0
MOV SEC, 0                  ; 到 60 回 0
JMP LOOP0
; fosc = 5MHz, T = 0.2μs
; Td = [50 × [17 × (5882 - 1) + 5 + 6 + 16] - 12 + 4 + 8] × 0.2μs = 1000.0004ms ≈ 1s
DELAY PROC NEAR
    PUSH BX
    PUSH CX
    MOV BX, 50                ; 4
DEL1: MOV CX, 5882            ; 4
DEL2: LOOP DEL2              ; 17/5
    DEC BX                    ; 2
    JNZ DEL1                  ; 16/4
    POP CX
    POP BX
    RET
DELAY ENDP
.DATA
SEC DB 00H
SITUATION DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 6FH; 共阴极
SIT_ END = $
END

```

4.2.3 74LS273 输出口控制 8 位 LED 数码管滚动显示单个数字

数字 0~7 逐个显示在 8 只集成数码管的相应位置上, 其电路原理如图 4-9 所示。由于仅限于仿真, 电路中没有加入限流电阻, 在实际应用中应考虑加限流电阻或增加驱动器。

本例使用了 8 只集成共阴极数码管, 所有数码管的引脚 a 连接在一起, 引脚 b、c、d、e、f、g、dp 也分别是并联的。任何时候发送的段码会传送到所有数码管上, 每位数码管的阴极是独立的。程序运行时, 任一时刻仅允许一只数码管共阴极的电平为 0, 相应数字就会显示在该位数码管上, 依次循环选中 8 只数码管中的一只时, 即可形成滚动显示效果。

图 4-9 所示电路中使数码管第 4 位 (从右至左) 显示 4, 应使右面的 273 接口输出位控码 11101111B, 使第 4 位数码管公共端为低电平, 左面的 273 接口输出数字 4 的共阴极段显码为 66H。

1) 参考程序。

```

.MODEL SMALL
.8086
.STACK
.CODE

```



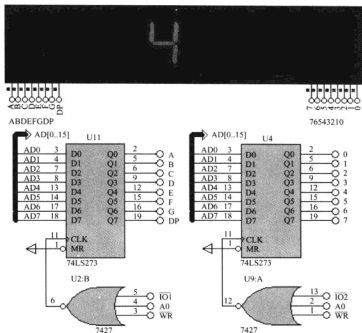


图 4-9 74LS273 输出口控制 8 位 LED 数码管滚动显示单个数字的电路原理

. STARTUP

AGAIN: MOV BP, 0FFFEH

MOV SI, OFFSET SITUATION

MOV CX, 8

NEXT: MOV DX, 0400H

; 位控口地址

MOV AL, 0FFH

OUT DX, AL

; 关显示

MOV DX, 0200H

; 段数码输出口地址

MOV AL, [SI]

OUT DX, AL

MOV DX, 0400H

; 位控口地址

MOV AX, BP

OUT DX, AL

CALL DELAY

STC

RCL BP, 1

ADD SI, 1

; 下一位

LOOP NEXT

JMP AGAIN

DELAY PROC NEAR

; 延时子程序

PUSH BX

```

        PUSH CX
        MOV  BX, 100
LP1:    MOV  CX, 3000
LP2:    LOOP LP2
        DEC  BX
        JNZ  LP1
        POP  CX
        POP  BX
        RET
DELAY  ENDP
. DATA
SITUATION  DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 6FH; 共阴极
END
```

2) 练习。

① 修改源程序使数字从左向右滚动。

② 缩短延时时间程序, 执行时会出现什么效果?

4.2.4 74LS273 输出口控制 8 位共阴极 LED 数码管动态扫描

本例同上一个例子的不同是在集成数码管上同时显示了多个数字，电路原理如图 4-10 所示。

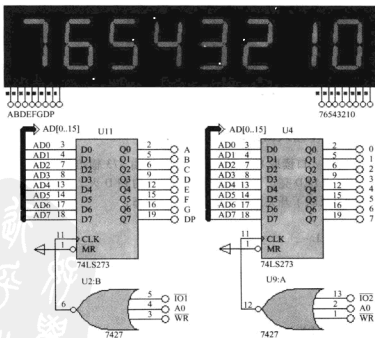


图 4-10 74LS273 输出口控制 8 位共阴极 LED 数码管动态扫描的电路原理

前面已经讨论过,对于集成数码管,任何时候发送的段码会被所有数码管接收到,如果本例中所有数码管的位码为 0,即 00000000B,则所有数码管都会显示同一数字符。

为了使不同的数码管显示不同的数字字符,本例使用的是集成式多位数码管常用的动态扫描显示技术,它利用人的视觉暂留特征,选通第 0 只数码管时,发送 0 的段码,选通第 1 只数码管时,发送 1 的段码,选通第 2 只数码管时,发送 2 的段码。每次仅选通一只数码管,发送对应的段码,每次切换选通下一只数码管并发送相应段码的时间间隔非常短,视觉惰性使人感觉不到字符是一个接一个显示在不同数码管上的,而会觉得所有字符很稳定地同时显示在不同数码管上。

可见,这种设计方法和上一案例类似的是仍然是在数码管上不同位置上逐个显示不同字符,只是切换的速度大大增加了。要注意的是,全屏扫描的频率要高于视觉暂留频率 16 ~ 20Hz。参考程序基本同上一节,仅使延时时间缩短些,改动如下述子程序黑体字部分。

```

DELAY  PROC  NEAR                                ; 延时子程序
          PUSH BX
          PUSH CX
          MOV  BX, 1                                ; 变动部分
LP1:      MOV  CX, 360                                ; 变动部分
LP2:      LOOP LP2
          DEC BX
          JNZ LP1
          POP CX
          POP BX
          RET
DELAY  ENDP

```

4.3 74LS244 简单输入接口

4.3.1 用 74LS244 作输入口读取开关状态

用 74LS244 作为输入口读取 8 个开关状态,然后由 273 输出到 LED 显示出开关状态。当开关闭合时,对应的 LED 亮;当开关断开时,对应的 LED 灭。其电路原理如图 4-11 所示,即 LED 的亮和灭反映了开关当前的断开和闭合状态。

参考程序:

```

.MODEL    SMALL
.8086
.STACK
.CODE
.STARTUP
AGAIN:    MOV  DX, 0400H      ; 244 地址
          IN   AL, DX         ; 读入开关状态
          MOV  DX, 0200H      ; 273 地址

```



```

OUT DX, AL          ; 输出开关状态
JMP AGAIN

.DATA
END

```

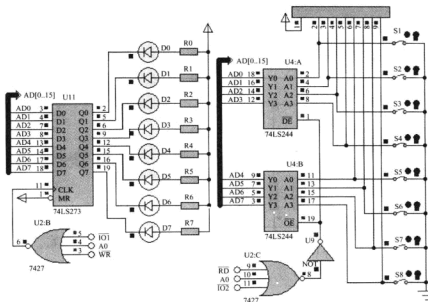


图 4-11 74LS244 输入口读取开关状态的电路原理

4.3.2 独立式按键控制 LED 左右移动

用两个独立式按键控制 LED 左右移动，当每按下 1 次 SW1 时，LED 向左移动 1 位，当每按下 1 次 SW2 时，LED 向右移动 1 位，其电路原理如图 4-12 所示。本例主要用于说明独立式按键的软件延时消抖方法。

参考程序：

```

.MODEL SMALL
.8086
.STACK
.CODE
.STARTUP
NEXT: MOV DX, 0200H          ; 273 地址
      MOV AL, LEDSTAT
      OUT DX, AL             ; 输出显示
SW1:  MOV DX, 0400H          ; 244 输入口地址
      IN AL, DX
      TEST AL, 01H
      JNZ SW2

```

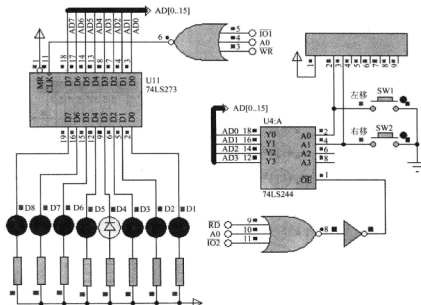


图 4-12 独立式按键控制 LED 左右移动的电路原理

```

SW1_2: CALL    DELAY                ; 消抖
        MOV     DX, 0400H            ; 244 输入/出口地址
        IN      AL, DX
        TEST    AL, 01H
        JNZ     NEXT

SW1_WAIT: CALL    DELAY              ; 消抖
        MOV     DX, 0400H            ; 244 输入/出口地址
        IN      AL, DX
        TEST    AL, 01H
        JZ      SW1_WAIT             ; 等待键释放
        CALL    DELAY
        ROL     LEDSTAT, 1           ; 左移

SW2: MOV     DX, 0400H              ; 244 输入/出口地址
        IN      AL, DX
        TEST    AL, 02H
        JNZ     NEXT

SW2_2: CALL    DELAY                ; 消抖
        MOV     DX, 0400H            ; 244 输入/出口地址
        IN      AL, DX
        TEST    AL, 02H
        JNZ     NEXT

SW2_WAIT: CALL    DELAY

```

```

MOV    DX, 0400H                ; 244 输入口地址
IN      AL, DX
TEST   AL, 02H
JZ      SW2_WAIT                ; 等待键释放
CALL    DELAY
ROR     LEDSTAT, 1              ; 右移
JMP     NEXT

;  $f_{osc} = 5\text{MHz}$ ,  $T = 0.2\mu\text{s}$ ,  $T_d \approx 20\text{ms}$ 
DELAY PROC NEAR                ; 延时子程序
    PUSH BX
    PUSH CX
    MOV  BX, 1                  ; 4
DEL1:  MOV CX, 5882              ; 4
DEL2:  LOOP DEL2                ; 17/5
    DEC  BX                    ; 2
    JNZ  DEL1                  ; 16/4
    POP  CX
    POP  BX
    RET
DELAY ENDP
. DATA
LEDSTAT DB 0FEH
END

```

4.3.3 独立式按键控制 LED 数码管增减 1

用两个按键分别控制 LED 数码管增减 1，当每按下 1 次 SW1 时，LED 数据管显示数字加 1 位，加到 10 回 0；当每按下 1 次 SW2 时，LED 数码管显示数字减 1 位，减到 0 回 9。其电路原理图如图 4-13 所示。

参考程序：

```

. MODEL SMALL
. 8086
. STACK
. CODE
. STARTUP
AGAIN: MOV  SI, OFFSET SITUATION ; SI 指向显示段码表首址
NEXT:  MOV  DX, 0200H            ; 273 地址
        MOV  AL, [SI]            ; 取显示段码
        OUT  DX, AL              ; 输出显示段码
SW1:    MOV  DX, 0400H           ; 244 输入口地址
        IN   AL, DX

```

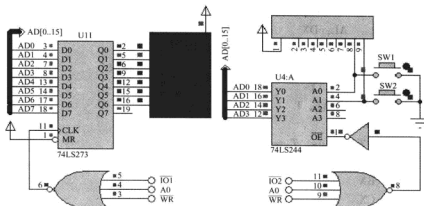


图 4-13 独立式按键控制 LED 数码管增减 1 的电路原理

```

TEST     AL, 01H
JNZ      SW2
SW1_2:   CALL     DELAY                ; 消抖
MOV      DX, 0400H                    ; 244 输入口地址
IN       AL, DX
TEST     AL, 01H
JNZ      NEXT
SW1_WAIT: CALL     DELAY
MOV      DX, 0400H                    ; 244 输入口地址
IN       AL, DX
TEST     AL, 01H
JZ       SW1_WAIT                     ; 等待键释放
CALL     DELAY
ADD      SI, 1                        ; 指向下一个显示段码
CMP      SI, OFFSET SIT_END           ; 是否显示完最后一个数
JB       SW2                           ; 没有, 继续显示下一个数
MOV      SI, OFFSET SITUATION
SW2:     MOV      DX, 0400H            ; 244 输入口地址
IN       AL, DX
TEST     AL, 02H
JNZ      NEXT
SW2_2:   CALL     DELAY                ; 消抖
MOV      DX, 0400H                    ; 244 输入口地址
IN       AL, DX
TEST     AL, 02H
JNZ      NEXT

```

```

SW2_WAIT:CALL    DELAY
            MOV    DX,0400H                ;244 输入口地址
            IN     AL,DX
            TEST   AL,02H
            JZ     SW2_WAIT                ;等待键释放
            CALL   DELAY
            SUB    SI, 1                    ;指向下一个显示段码
            CMP    SI, OFFSET SITUATION - 1 ;是否减到 0
            JA     NEXT                    ;没有,继续显示下一个数
            MOV    SI, SIT_END - 1
            JMP    NEXT
;fosc = 5MHz, T = 0.2μs, Td ≈ 20ms
DELAY      PROC    NEAR                    ;延时子程序
            PUSH   BX
            PUSH   CX
            MOV    BX,1                    ;4
DEL1:      MOV    CX,5882                  ;4
DEL2:      LOOP   DEL2                    ;17/5
            DEC    BX                      ;2
            JNZ    DEL1                    ;16/4
            POP    CX
            POP    BX
            RET
DELAY      ENDP
. DATA
SITUATION DB 0C0H,0F9H,0A4H,0B0H,99H,92H,82H,0F8H,80H,90H;共阳极
SIT_END = $
END

```

4.3.4 LED 数码管显示行列式键盘键值

用一位 LED 数码管显示行列式键盘当前按下的键值,电路原理如图 4-14 所示。独立式按键每个键要占用 1 个 I/O 端口,当按键较多时会占用更多的 I/O 端口,为减少对端口的占用,本例使用 4×4 行列式键盘,这样会减少端口占用,但识别按键的代码比独立式按键要复杂一些。

识别行列式键盘键值的常用方法有行扫描法和线反转法等,本例使用了行扫描法判断所按下的键值,该方法的基本思想是由程序对键盘进行逐行扫描,通过检测到的列输出状态来确定闭合键。图 4-14 所示键盘矩阵行扫描码输出用一片 74LS273 实现,列扫描码的读入用一片 74LS244 实现。首先判断 16 个键中是否有键按下,在 4 个行线上输出都为 0,如果有任一按键按下,则 4 条列线上必有一位为 0。

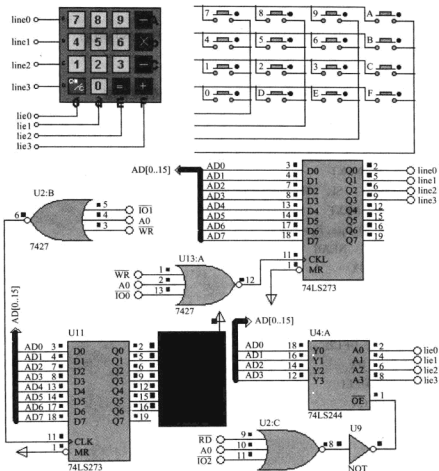


图 4-14 LED 数码管显示行列式键盘键值的电路原理

如果已有键按下，则判断按键所在的行、列位置，并返回按键序号。代码中行扫描码初值为 0FEH，通过将该值循环右移，对 4 条行线逐行发送 0，每发送一次行线值，读入一次列线值，并判断列线值是否为 0。例如，若输出行线值为 11111101B，读入列线值为 00000111B，说明在 line1 行和 lie3 列交点处按键 (b) 按下，行扫描码 11111101B 和列扫描码 00000111B 十六进制组合表示为 0FD07H，对应 TABLE 表中第 7 个数，对应的键值为 0BH。

参考程序：

```

.MODEL SMALL
.8086
.STACK
.CODE
.STARTUP
AGAIN: CALL KEYPROC
      MOV AL,KEY

```

；键值送 AL

```

MOV BX,OFFSET SITUATION
XLAT                                ;查表求键号值的段码
MOV DX,0200H
OUT DX,AL                          ;输出显示键号
JMP AGAIN
KEYPROC PROC                        ;键扫描子程序,扫描到的键值存入 KEY
MOV AL,00H
MOV DX,0000H
OUT DX,AL                          ;行线输出都为 0
MOV DX,0400H
IN AL,DX                           ;读列值
AND AL,0FH
CMP AL,0FH                         ;都为 1 无键按下
JNZ SCAN                           ;不都为 1 则说明有键按下,转 SCAN
RET                                 ;查键值
                                   ;没有闭合键返回
SCAN: CALL DELAY                  ;键盘消抖
PROG: MOV CL,0FEH                 ;行列扫描,送扫描初值,从第 0 行开
                                   ;始扫描
                                   ;共 4 行
MOV HANGNUM,4
FROW: MOV AL,CL
MOV DX,0000H
OUT DX,AL                          ;行线输出
MOV DX,0400H
IN AL,DX                           ;读列值
AND AL,0FH
CMP AL,0FH                         ;判断列值是否都为 1
JNZ FCOL                           ;查到行值(CL)和列值(AL),转 FCOL
                                   ;处理
                                   ;下一行
ROL CL,1
DEC HANGNUM
JNZ FROW                            ;4 行末扫描完继续扫描下一行
RET                                 ;没查到闭合键返回
FCOL: MOV AH,CL                  ;行值存入 AH
MOV SI,OFFSET TABLE + 15 * 2     ;取键号表的末地址 16 - 1
MOV CX,16                          ;4 * 4 共 16 个键
LOPO: CMP AX,[SI]                 ;在 TABLE 表中查找匹配值
JZ KEYPRO                          ;行码和列码对应唯一键值,转
                                   ;KEYPRO 求键值

```

```

        DEC SI
        DEC SI
        LOOP LOP0
        RET                                ;没找到返回
KEYPRO:  MOV BX,OFFSET TABLEX           ;查表求键值
        DEC CL
        MOV AL,CL
        XLAT
        MOV KEY,AL
        RET
KEYPROC ENDP
;FOSC = 5MHz,T = 0.2US,TD ≈ 20MS
DELAY PROC NEAR
        PUSH BX
        PUSH CX
        MOV BX,1                          ;4
DEL1:    MOV CX,5882                       ;4
DEL2:    LOOP DEL2                        ;17/5
        DEC BX                            ;2
        JNZ DEL1                          ;16/4
        POP CX
        POP BX
        RET
DELAY    ENDP
. DATA
KEY      DB 0                             ;键值
HANGNUM  DB 4                             ;行数
SITUATION DB 0C0H,0F9H,0A4H,0B0H,99H,92H,82H,0F8H,80H,90H
DB 88H,83H,0C6H,0A1H,86H,8EH             ;共阳极
TABLE    DW 0FE0EH                        ;行列扫描码表
        DW 0FE0DH
        DW 0FE0BH
        DW 0FE07H
        DW 0FD0EH
        DW 0FD0DH
        DW 0FD0BH
        DW 0FD07H
        DW 0FB0EH
        DW 0FB0DH

```




```

        DW 0FB0BH
        DW 0FB07H
        DW 0F70EH
        DW 0F70DH
        DW 0F70BH
        DW 0F707H

TABLEX  DB 7,8,9,0AH,4,5,6,0BH,1,2,3,0CH,0,0DH,0EH,0FH ; 键值表
SIT_END = $
END

```

4.4 点阵 LED 显示汉字

4.4.1 16×16 点阵 LED 显示汉字

用 16×16 点阵 LED 显示汉字“济”，其电路原理如图 4-15 所示。

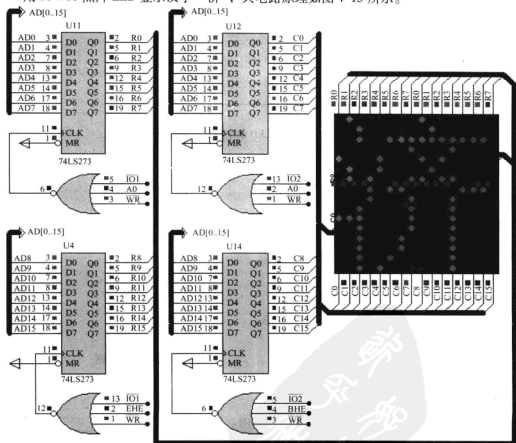
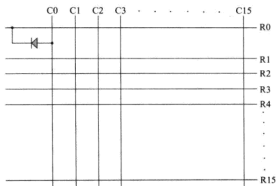


图 4-15 16×16 点阵 LED 显示汉字的电路原理

1) 点阵 LED 汉字显示原理。

点阵 LED 汉字显示是利用人的视觉暂留特征, 对点阵 LED 屏逐列 (或逐行) 扫描输出汉字点阵编码。如图 4-16 所示, 对汉字逐列取模 (共阳极), 当某一列为高电平时, 对应于该行行线为 0 的 LED 点亮。例如开始时 C0 列为高电平, 在行线上输出 C0 列的字模为 1111111111110000B, 则对应 C0 列的上面 4 行的 4 个 LED 点亮; 延时一段时间后, 再使 C1 列为高电平, 在行线上输出 C1 列的字模; 延时一段时间后, 再使 C2 列为高电平, 在行线上输出 C2 列的字模; 再延时一段时间后, 依次使下一列为高电平并输出该列对应的 16 位字模。对于 16×16 点阵屏, 扫描输出了 16 次, 每列用 16 位即 2B 的编码, 也就是说 16×16 点阵屏要显示一个汉字要有用 16×2 共 32B 的编码。同时, 应注意的是全屏扫描的频率要高于视觉暂留频率 $16 \sim 20\text{Hz}$, 最好不低于 50Hz 。

图 4-16 16×16 点阵 LED 汉字显示的电路原理

2) 参考程序。

```

.MODEL SMALL
ROW_ADRESS EQU 0200H
COL_ADRESS EQU 0400H
.8086
.STACK
.CODE
.STARTUP
AGAIN:  MOV SI, OFFSET TAB
        MOV COUNT,5
        MOV DI, 0001H
        MOV CX,16
NEXT:   MOV DX,COL_ADRESS
        MOV     AX,0000H
        OUT     DX,AX
        MOV DX,ROW_ADRESS

```

;第 1 列
;共 16 列
;列地址
;禁止显示
;行地址

```

MOV AX,[SI]
OUT DX,AX
MOV DX,COL_ADDRESS           ;列地址
MOV     AX,DI
OUT     DX,AX
CALL DELAY2
DEC COUNT
JNZ NEXT
MOV COUNT,5
ROL DI,1
ADD SI, 2                     ;字模指针加2
LOOP NEXT
JMP AGAIN

DELAY2 PROC NEAR              ;延时子程序
    PUSH BX
    PUSH CX
    MOV BX,1
LP1:  MOV CX,200
LP2:  LOOP LP2
    DEC BX
    JNZ LP1
    POP CX
    POP BX
    RET
DELAY2 ENDP

. DATA                      ;PCTOOL 取模工具:阳码、逆向、逐列式
TAB  DB 0EFH, 0FBH, 09EH, 0FBH, 0F9H, 001H, 03FH, 0FEH
    DB 0CFH, 07FH, 07BH, 0BFH, 07BH, 0CFH, 0B3H,0F0H;
    DB 0AAH, 0FFH, 0D9H, 0FFH, 0ABH, 0FFH, 0B3H, 000H;
    DB 07BH, 0FFH, 07BH, 0FFH, 07BH, 0FFH, 0FFH, 0FFH    ;"济",0
SIT_END = $
COUNT DB 5
END

```

4.4.2 16×16 点阵 LED 移动显示多个汉字

在 16×16 点阵 LED 移动显示“济宁职业技术学院”，电路原理同第 4.4.1 节如图 4-15 所示。

要想实现汉字的移动显示，只要每屏显示之后，使读取汉字字模的指针移动两个字节（16×16 点阵），将最左边一列两个字节的字模移出，就可实现汉字的移动显示。

1) 参考程序:

```

.MODEL SMALL
ROW_ADDRESS EQU 0200H
COL_ADDRESS EQU 0400H
.8086
.STACK
.CODE
.STARTUP
START:    MOV COUNT,20                ;每屏显示次数
          MOV  M,  OFFSET TAB         ;M 变量暂存字模地址
NEXT0:    MOV  DI, 0001H              ;第 1 列
NEXT1:    MOV  CX, 16                 ;共 16 列
          MOV  SI,M                   ;SI 字模指针
NEXT2:    MOV  DX,COL_ADDRESS         ;列地址
          MOV  AX,0000H
          OUT  DX,AX                  ;禁止显示
          MOV  DX,ROW_ADDRESS         ;行地址
          MOV  AX,[SI]
          OUT  DX,AX
          MOV  DX,COL_ADDRESS         ;列地址
          MOV  AX,DI
          OUT  DX,AX
          CALL DELAY2
          ROL  DI,1
          ADD  SI, 2                   ;字模指针加 2
          LOOP NEXT2
          DEC  COUNT                  ;每屏显示次数
          JNZ  NEXT1
          MOV  COUNT,20
          ADD  M,2                     ;移动 1 列
          CMP  M,OFFSET SIT_END-32    ;判断是否显示到最后一个汉字
          JAE  START
          JMP  NEXT0
DELAY2    PROC NEAR                  ;延时子程序
          PUSH BX
          PUSH CX
          MOV  BX,1
LP1:      MOV  CX,469
LP2:      LOOP LP2

```

```

DEC BX
JNZ LP1
POP CX
POP BX
RET
DELAY2      ENDP
. DATA
;PCTOOL 取模工具:阳码、逆向、逐列式
TAB DB 0EFH, 0FBH, 09EH, 0FBH, 0F9H, 001H, 03FH, 0FEH;
DB 0CFH, 07FH, 07BH, 0BFH, 07BH, 0CFH, 0B3H, 0F0H;
DB 0AAH, 0FFH, 0D9H, 0FFH, 0ABH, 0FFH, 0B3H, 000H;
DB 07BH, 0FFH, 07BH, 0FFH, 07BH, 0FFH, 0FFH, 0FFH ;"济",0
DB 0FFH, 0FFH, 06FH, 0FFH, 073H, 0FFH, 07BH, 0FFH;
DB 07BH, 0FFH, 07BH, 0BFH, 07AH, 07FH, 079H, 080H;
DB 07BH, 0FFH, 07BH, 0FFH, 07BH, 0FFH, 07BH, 0FFH;
DB 06BH, 0FFH, 071H, 0FFH, 0FBH, 0FFH, 0FFH, 0FFH ;"宁",1
DB 0FDH, 0EFH, 0FDH, 0EFH, 001H, 0F0H, 06DH, 0F7H;
DB 06DH, 0F7H, 001H, 000H, 0FDH, 0FBH, 0FFH, 0BBH;
DB 001H, 0DEH, 07DH, 0E3H, 07DH, 0F7H, 07DH, 0FFH;
DB 07DH, 0FBH, 001H, 0F6H, 0FFH, 0CFH, 0FFH, 0FFH ;"职",2
DB 0FFH, 0DFH, 0EFH, 0DFH, 09FH, 0DFH, 07FH, 0DCH;
DB 0FFH, 0DEH, 000H, 0C0H, 0FFH, 0DFH, 0FFH, 0DFH;
DB 0FFH, 0DFH, 000H, 0C0H, 0FFH, 0DDH, 07FH, 0DEH;
DB 09FH, 0DFH, 0C7H, 0CFH, 0EFH, 0DFH, 0FFH, 0FFH ;"业",3
DB 0F7H, 0FEH, 0F7H, 0BEH, 077H, 07FH, 000H, 080H;
DB 0B7H, 0FFH, 0D7H, 0BFH, 0FFH, 0BFH, 037H, 0DFH;
DB 0B7H, 0ECH, 0B7H, 0F3H, 080H, 0F3H, 0B7H, 0EDH;
DB 037H, 0DEH, 0B7H, 09FH, 0F7H, 0DFH, 0FFH, 0FFH ;"技",4
DB 0EFH, 0EFH, 0EFH, 0EFH, 0EFH, 0F7H, 0EFH, 0FBH;
DB 0EFH, 0FDH, 06FH, 0FEH, 0AFH, 0FFH, 000H, 080H;
DB 0AFH, 0FFH, 06FH, 0FFH, 0EDH, 0FEH, 0EBH, 0F9H;
DB 0EFH, 0F3H, 0EFH, 0E7H, 0EFH, 0F7H, 0FFH, 0FFH ;"术",5
DB 0BFH, 0FFH, 0CFH, 0FDH, 0EFH, 0FDH, 0EDH, 0FDH;
DB 0A3H, 0FDH, 0ABH, 0FDH, 0AFH, 0BDH, 0AEH, 07DH;
DB 0A1H, 080H, 02BH, 0FDH, 0AFH, 0FDH, 0E7H, 0FDH;
DB 0A8H, 0FDH, 0CDH, 0FDH, 0EFH, 0FDH, 0FFH, 0FFH ;"学",6
DB 001H, 000H, 0FDH, 0FFH, 0CDH, 0FDH, 0B5H, 0FBH;
DB 079H, 07CH, 0F3H, 0BEH, 0DBH, 0CEH, 0DBH, 0F0H;
DB 0DAH, 0FEH, 0D9H, 0FEH, 0DBH, 080H, 0DBH, 07EH;

```

```

DB 0DBH, 07EH, 0F3H, 07EH, 0FBH, 00EH, 0FFH, 0FFH    ; "院", 7
SIT_END = $
COUNT DB ?
MOV     DW ?
END

```

2) 练习。将上述汉字的移动显示改成每隔一段时间逐个汉字显示。

提示：要想实现汉字逐个轮流显示，只要使取模指针每隔一段时间指向下一个汉字的字模就可以，其源程序和第 4.4.2 节中的汉字移动变化不大，仅需修改上述源程序中如下三条指令中的第二个操作数（黑体字）部分和延时子程序（使单个字显示时间稍长一些）。

```

MOV     COUNT, 100                ; 每屏刷新次数
ADD     M, 32                     ; 指向下一个汉字字模
CMP     M, OFFSET SIT_END        ; 判断是否显示到最后一个汉字

```

4.4.3 16×32 点阵 LED 移动显示多个汉字

在 16×32 点阵 LED 上移动显示“济宁职业技术学院”，其电路原理如图 4-17 所示。

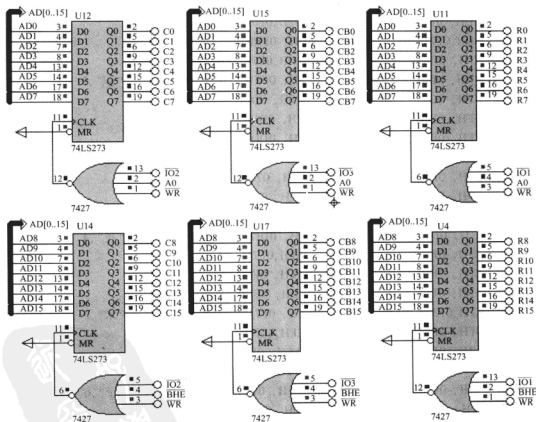


图 4-17 16×32 点阵 LED 移动显示多个汉字的电路原理

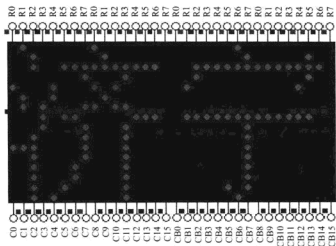


图4-17 16×32点阵LED移动显示多个汉字的电路原理(续)

```

.MODEL SMALL
ROW_ADDRESS EQU 0200H
COL_ADDRESS EQU 0400H
COL_ADDRESS_B EQU 0600H
.8086
.STACK
.CODE
.STARTUP
START: MOV     M, OFFSET TAB      ;变量 M 暂存字模地址
        MOV     COUNT,10         ;每屏显示次数
NEXT0:  MOV     DI, 0001H         ;第 1 列
NEXT1:  MOV     CX, 16           ;共 16 列
        MOV     SI,M             ;SI 字模指针
NEXT2:  MOV     DX,COL_ADDRESS    ;列地址
        MOV     AX,0000H
        OUT     DX,AX            ;禁止显示
        MOV     DX,ROW_ADDRESS   ;行地址
        MOV     AX,[SI]
        OUT     DX,AX
        MOV     DX,COL_ADDRESS   ;列地址
        MOV     AX,DI
        OUT     DX,AX
        CALL    DELAY2
        ROL     DI,1

```

```

ADD     SI, 2                      ;字模指针加 2
LOOP    NEXT2
MOV     DX, COL_ADDRESS           ;列地址
MOV     AX, 0000H
OUT     DX, AX                    ;禁止显示
MOV     DI, 0001H
MOV     CX, 16
BNEXT2: MOV    DX, COL_ADDRESS_B   ;列地址
        MOV    AX, 0000H
        OUT    DX, AX              ;禁止显示
        MOV    DX, ROW_ADDRESS     ;行地址
        MOV    AX, [SI]
        OUT    DX, AX
        MOV    DX, COL_ADDRESS_B   ;列地址
        MOV    AX, DI
        OUT    DX, AX
        CALL   DELAY2
        ROL    DI, 1
        ADD    SI, 2              ;字模指针加 2
        LOOP   BNEXT2
        MOV    DX, COL_ADDRESS_B   ;列地址
        MOV    AX, 0000H
        OUT    DX, AX              ;禁止显示
        DEC    COUNT              ;每屏显示次数
        JNZ    NEXT0
        MOV    COUNT, 10
        ADD    M, 2
        CMP    M, OFFSET SIT_END - 64 ;判断是否显示到最后一个汉字
        JAE    START
        JMP    NEXT0
DELAY2  PROC    NEAR              ;延时子程序
        PUSH   BX
        PUSH   CX
        MOV    BX, 1
LP1:    MOV    CX, 200
LP2:    LOOP   LP2
        DEC    BX
        JNZ    LP1
        POP    CX

```



```

    POP BX
    RET
DELAY2     ENDP
. DATA
    ;PCTOOL 取模工具:阳码,逆向,逐列式
    TAB DB 0EFH, 0FBH, 09EH, 0FBH, 0F9H, 001H, 03FH, 0FEH;
    DB 0CFH, 07FH, 07BH, 0BFH, 07BH, 0CFH, 0B3H, 0F0H;
    DB 0AAH, 0FFH, 0D9H, 0FFH, 0ABH, 0FFH, 0B3H, 000H;
    DB 07BH, 0FFH, 07BH, 0FFH, 07BH, 0FFH, 0FFH, 0FFH;"济",0
    .....;略
    DB 001H, 000H, 0FDH, 0FFH, 0CDH, 0FDH, 0B5H, 0FBH;
    DB 079H, 07CH, 0F3H, 0BEH, 0DBH, 0CEH, 0DBH, 0F0H;
    DB 0DAH, 0FEH, 0D9H, 0FEH, 0DBH, 080H, 0DBH, 07EH;
    DB 0DBH, 07EH, 0F3H, 07EH, 0FBH, 00EH, 0FFH, 0FFH;"院",7
    SIT_END = $
    COUNT DB ?
    M      DW ?
    END

```

4.4.4 32×32 点阵 LED 移动显示多个汉字

在 32×32 点阵 LED 移动显示“济宁职业技术学院”，其电路原理如图 4-18 所示。图 4-18 中用 4 片 74LS273 作点阵 LED 的行扫描输出接口，另用 4 片 74LS273 作点阵 LED 的列扫描输出接口。为了方便使用 PCTOOL 等字模提取工具提取字模，源程序中给出了部分汉字的 32 点阵结构，以便比较参考。

参考程序：

```

. MODEL SMALL
ROW_ADDRESS EQU 0000H
ROW_ADDRESS_2 EQU 0200H
COL_ADDRESS EQU 0400H
COL_ADDRESS_B EQU 0600H
. 8086
. STACK
. CODE
. STARTUP
START:MOV     M,    OFFSET TAB
            MOV     COUNT,10                ;每屏刷新次数
NEXT0:MOV     DI,    0001H
NEXT1:MOV     CX,    16
            MOV     SI,    M

```

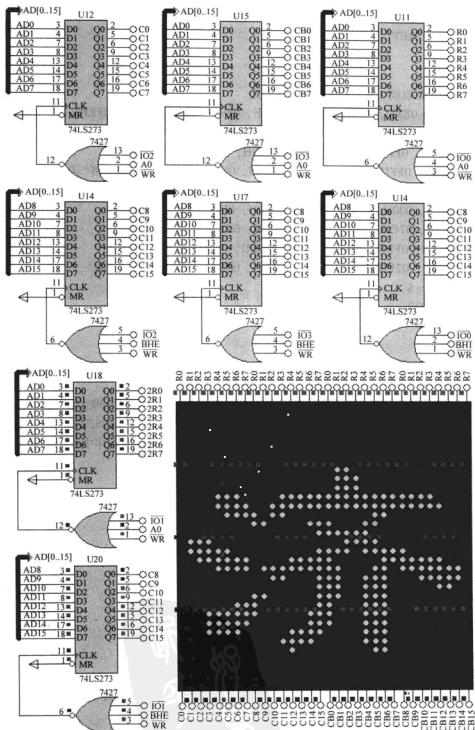


图 4-18 32×32 点阵 LED 移动显示多个汉字的电路原理

```

NEXT2:  MOV DX, COL_ADRESS           ;列地址
        MOV AX, 0000H
        OUT DX, AX                   ;禁止显示
        MOV DX, ROW_ADRESS          ;行地址
        MOV AX, [SI]
        OUT DX, AX
        ADD SI, 2
        MOV DX, ROW_ADRESS_2        ;行地址 2
        MOV AX, [SI]
        OUT DX, AX
        ADD SI, 2                   ;下一列
        MOV DX, COL_ADRESS          ;列地址
        MOV AX, DI
        OUT DX, AX
        CALL DELAY2
        ROL DI, 1
        LOOP NEXT2
        MOV DX, COL_ADRESS          ;列地址
        MOV AX, 0000H
        OUT DX, AX                   ;禁止显示
        MOV DI, 0001H
        MOV CX, 16
BNEXT2: MOV DX, COL_ADRESS_B         ;列地址
        MOV AX, 0000H
        OUT DX, AX                   ;禁止显示
        MOV DX, ROW_ADRESS          ;行地址
        MOV AX, [SI]
        OUT DX, AX
        ADD SI, 2
        MOV DX, ROW_ADRESS_2        ;行地址 2
        MOV AX, [SI]
        OUT DX, AX
        ADD SI, 2                   ;NEXT SITUATION
        MOV DX, COL_ADRESS_B        ;列地址
        MOV AX, DI
        OUT DX, AX
        CALL DELAY2
        ROL DI, 1
        LOOP BNEXT2

```

```

MOV DX, COL_ADDRESS_B           ;列地址
MOV AX, 0000H
OUT DX, AX                       ;禁止显示
DEC COUNT
JNZ NEXT0
MOV COUNT, 10
ADD M,4
CMP M, OFFSET SIT_END - 128
JAE START
JMP NEXT0
DELAY2 PROC NEAR                 ;延时子程序
PUSH BX
PUSH CX
MOV BX, 1
LP1: MOV CX, 200
LP2: LOOP LP2
DEC BX
JNZ LP1
POP CX
POP BX
RET
DELAY2 ENDP
. DATA
; PCTOOL 阳码、逆向、逐列式
TAB DB 0FFH, 0FFH, 0FFH, 0FFH, 0FFH, 07FH, 0FEH, 0FFH;
DB 0FFH, 07FH, 0FCH, 0FFH, 0FFH, 07FH, 07CH, 0FCH;
DB 0FFH, 0F3H, 078H, 0FCH, 0FFH, 0E3H, 078H, 0FCH;
DB 0FFH, 0E3H, 039H, 0FCH, 0FFH, 0E7H, 039H, 0FEH;
DB 0FFH, 0CFH, 01FH, 0FFH, 0FFH, 0DFH, 0DBH, 0FFH;
DB 0FFH, 0E7H, 0F3H, 0FFH, 0FFH, 0E3H, 0F3H, 0FBH;
DB 0FFH, 0F3H, 0F9H, 0F1H, 0FFH, 0F3H, 0F9H, 0F9H;
DB 0FFH, 0D3H, 079H, 0F8H, 0FFH, 093H, 000H, 0FCH;
DB 0FFH, 093H, 000H, 0FEH, 07FH, 0B0H, 0C2H, 0FFH;
DB 07FH, 030H, 0FEH, 0FFH, 0FFH, 030H, 0FFH, 0FFH;
DB 0FFH, 013H, 006H, 0F8H, 0FFH, 043H, 002H, 0F0H;
DB 0FFH, 0E3H, 000H, 0F0H, 0FFH, 0F3H, 0FCH, 0FFH;
DB 0FFH, 0F3H, 0F8H, 0FFH, 0FFH, 0F3H, 0F9H, 0FFH;
DB 0FFH, 0F3H, 0F1H, 0FFH, 0FFH, 0F3H, 0F1H, 0FFH;
DB 0FFH, 0F7H, 0E1H, 0FFH, 0FFH, 0FFH, 0E3H, 0FFH;

```

```

DB 0FFH, 0FFH, 0F3H, 0FFH, 0FFH, 0FFH, 0FFH, 0FFH; "济", 0
.....( '宁职业技术学院' 略)
SIT_END = $
COUNT DB ?
M      DW ?
END

```

4.5 液晶显示器

4.5.1 LM032L 字符液晶显示器

使用字符液晶显示器循环显示一段字符, 其电路原理如图 4-19 所示。图 4-19 中 LCD 与 CPU 接口采用了总线方式, 用 RD 和 WR 信号控制读写, 地址线 A1 和 A2 用作 LCD 内部寄存器的选择。

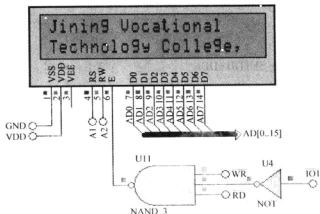


图 4-19 LM032L 字符液晶显示器的电路原理

参考程序:

```

.MODEL SMALL
;LCD REGISTERS ADDRESSES
LCD_CMD_WR EQU 0200H
LCD_DATA_WR EQU 0202H
LCD_BUSY_RD EQU 0204H
LCD_DATA_RD EQU 0206H
;LCD COMMANDS
LCD_CLS EQU 1

```

```

LCD_HOME      EQU    2
LCD_SETMODE    EQU    4
LCD_SETVISIBLE EQU    8
LCD_SHIFT      EQU   16
LCD_SETFUNCTION EQU   32
LCD_SETTCGADDR EQU   64
LCD_SETDDADDR  EQU  128

.8086
. STACK
. CODE
. STARTUP
    MOV AL,LCD_SETFUNCTION + 16 + 8 ;0 0 1 DL N F - -8 位两行
    CALL WRCMD                       ;8 位/4 位数据接口 2 行/1 行 5×10/5×8
    MOV AL,LCD_SETVISIBLE + 4        ;0 0 0 0 1 D C B 显示
    CALL WRCMD                       ;(D)显示 (C)光标 (B)闪烁
AGAIN: MOV AL,LCD_SETVISIBLE + 4     ;光标闪烁
    CALL WRCMD
    MOV AL,LCD_CLS                   ;清屏
    CALL WRCMD
    MOV AL,LCD_SETDDADDR              ;第 1 行 0 列
    CALL WRCMD
    MOV SI,OFFSET STRING1A
    CALL WRSTR
    MOV AL,LCD_SETDDADDR OR 64        ;第 2 行 0 列
    CALL WRCMD
    MOV SI,OFFSET STRING1B
    CALL WRSTR
    MOV CX,4
DELO: CALL DELAY                     ;延时
    LOOP DELO
    MOV AL,LCD_CLS                   ;清屏
    CALL WRCMD
    MOV AL,LCD_SETDDADDR              ;第 1 行 0 列
    CALL WRCMD
    MOV SI,OFFSET STRING1B
    CALL WRSTR
    MOV AL,LCD_SETDDADDR OR 64        ;第 2 行 0 列
    CALL WRCMD
    MOV SI,OFFSET STRING2A

```

```

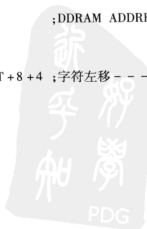
CALL WRSTR
MOV CX,4
DEL1: CALL DELAY ;延时
      LOOP DEL1
      MOV AL,LCD_CLS ;清屏
      CALL WRCMD
      MOV AL,LCD_SETDDADDR ;第1行0列
      CALL WRCMD
      MOV SI,OFFSET STRING2A
      CALL WRSTR
      MOV AL,LCD_SETDDADDR OR 64 ;第2行0列
      CALL WRCMD
      MOV SI,OFFSET STRING2B
      CALL WRSTR
      MOV CX,4
DEL2: CALL DELAY ;延时
      LOOP DEL2

      MOV AL,LCD_CLS ;清屏
      CALL WRCMD
      MOV AL,LCD_SETDDADDR ;第1行0列
      CALL WRCMD
      MOV SI,OFFSET SHIFT_LINE1
      CALL WRSTR
      MOV AL,LCD_SETDDADDR + 64 ;第2行0列
      CALL WRCMD
      MOV SI,OFFSET SHIFT_LINE2
      CALL WRSTR

      MOV AL,LCD_HOME ;DDRAM ADDRESS 0
      CALL WRCMD
      MOV CX,20
SCREEN_LEFT_SHIFT: MOV AL,LCD_SHIFT + 8 + 4 ;字符左移 -- -1 S/C R/L --
      CALL WRCMD
      CALL DELAY
      LOOP SCREEN_LEFT_SHIFT
      CALL DELAY

      MOV CX,20

```



```

SCREEN_RIGHT_SHIFT: MOV AL,LCD_SHIFT+8 ;字符右移
                     CALL WRCMD
                     CALL DELAY
                     LOOP SCREEN_RIGHT_SHIFT
                     CALL DELAY

                     MOV AL,LCD_CLS ;清屏
                     CALL WRCMD
                     MOV AL,LCD_SETDDADDR + 64 + 8 ;第2行8列
                     CALL WRCMD

                     MOV AL,LCD_SETVISIBLE+7 ;闪烁光标
                     CALL WRCMD
                     MOV CX,10
CURSE_RIGHT_SHIFT: MOV AL,LCD_SHIFT+4 ;光标右移 -- -1 S/C R/L --
                     CALL WRCMD
                     CALL DELAY
                     CALL DELAY
                     LOOP CURSE_RIGHT_SHIFT
                     JMP AGAIN

WTBUSY PROC NEAR ;LCD 忙等待子程序
    MOV DX,LCD_BUSY_RD
WAIT0: IN AL,DX
    TEST AL,80H
    JNZ WAIT0
    RET
WTBUSY ENDP

WRCMD PROC NEAR ;写命令子程序
    PUSH AX
    CALL WTBUSY
    POP AX
    MOV DX,LCD_CMD_WR
    OUT DX,AL
    RET
WRCMD ENDP

WRSTR PROC NEAR ;发送字符串程序

```



```

WRSLW1:CALL WTBUSY
        MOV DX,LCD_DATA_WR
        MOV AL,[SI]
        CMP AL,0
        JE EXIT_STR ;是否到字符串尾
        OUT DX,AL
        INC SI
        JMP WRSLW1
EXIT_STR:RET
WRSTR   ENDP

DELAY   PROC NEAR ;延时子程序
        PUSH BX
        PUSH CX
        MOV BX,500
LP1:MOV CX,469
LP2:LOOP LP2
        DEC BX
        JNZ LP1
        POP CX
        POP BX
        RET
DELAY   ENDP

. DATA
STRING1A DB 'Welcome To ',0
STRING1B DB 'Jining Vocational ',0
STRING2A DB 'Technology College ',0
STRING2B DB 'Shandong,China ',0
SHIFT_LINE1 DB '0123456789ABCDEFGHIJ0123456789ABCDEFGHIJ ',0
SHIFT_LINE2 DB 'E_MAIL:HJB372@SOHU.COM TEL:13405377035 ',0
END

```

4.5.2 12864 图形液晶显示器

利用 12864 图形液晶显示器显示一幅图形，其电路原理如图 4-20 所示。

获取待显示图像的点阵数据时需要使用辅助工具软件，本例使用 Zimo21 图形工具软件。可先选取一幅图形，预先用 Windows 附件中的画图工具对其尺寸等进行调整和修饰，取像素为宽度 128，高度 64，再使用 Zimo21 工具软件获取图形的点阵数据。

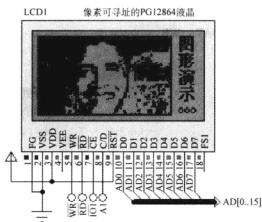


图 4-20 12864 图形液晶显示器的电路原理

参考程序:

```
.MODEL SMALL
```

```
;12864 端口定义
```

```
LCMDW EQU 0200H
```

```
;数据口
```

```
LCMCW EQU 0202H
```

```
;命令口
```

```
;DISRAM_SIZE 7FFFH
```

```
;设置显示 RAM 大小
```

```
TXTSTART EQU 0000H
```

```
;设置文本区的起始地址
```

```
GRSTART EQU 6800H
```

```
;设置图像区的地址
```

```
CGRAMSTART EQU 7800H
```

```
;设置 CGRAM 的起始地址
```

```
;12864 命令定义
```

```
LC_CUR_POS EQU 21H
```

```
;光标位置设置
```

```
LC_CGR_POS EQU 22H
```

```
;CGRAM 偏置地址设置
```

```
LC_ADD_POS EQU 24H
```

```
;地址指针位置
```

```
LC_TXT_STP EQU 40H
```

```
;文本区首址
```

```
LC_TXT_WID EQU 41H
```

```
;文本区宽度
```

```
LC_GRH_STP EQU 42H
```

```
;图形区首址
```

```
LC_GRH_WID EQU 43H
```

```
;图形区宽度
```

```
LC_MOD_OR EQU 80H
```

```
;显示方式
```

```
LC_MOD_XOR EQU 81H
```

```
LC_MOD_AND EQU 82H
```

```
LC_MOD_TCH EQU 83H
```

```
LC_DIS_SW EQU 90H
```

```
;显示开关:
```

```
;D0 = 1/0: 光标闪烁启用/禁用
```

```
;D1 = 1/0: 光标显示启用/禁用
```

```
;D2 = 1/0: 文本显示启用/禁用
```

```
;D3 = 1/0: 图形显示启用/禁用
```

```

LC_CUR_SHP EQU 0A0H      ;光标形状选择:A0~A7 表示光标占的行数
LC_AUT_WR EQU 0B0H       ;自动写设置
LC_AUT_RD EQU 0B1H       ;自动读设置
LC_AUT_OVR EQU 0B2H      ;自动读/写结束
LC_INC_WR EQU 0C0H       ;数据写地址加1
LC_INC_RD EQU 0C1H       ;数据读地址加1
LC_DEC_WR EQU 0C2H       ;数据写地址减1
LC_DEC_RD EQU 0C3H       ;数据读地址减1
LC_NOC_WR EQU 0C4H       ;数据写地址不变
LC_NOC_RD EQU 0C5H       ;数据读地址不变
LC_SCN_RD EQU 0E0H       ;屏读
LC_SCN_CP EQU 0E8H       ;屏复制
LC_BIT_OP EQU 0F0H       ;位操作:D0~D2 定义 D0~D7 位,D3:1 置位/0

```

;清除

```

LCD_WIDTH EQU 16;        ;宽 128 像素,128/8 = 16B

```

```

LCD_HEIGHT EQU 64;

```

```

.8086

```

```

.STACK

```

```

.CODE

```

```

.STARTUP

```

```

    CALL LCD_INITIALISE ;液晶显示器初始化

```

```

    MOV ROW,0

```

```

    MOV COL,0

```

```

    CALL SET_LCD_POS

```

```

    CALL CLS

```

```

    MOV PARA1,0

```

```

    MOV PARA2,0

```

```

    MOV CMD,LC_GRH_STP

```

```

    CALL LCD_WRITE_COMMAND_P2

```

```

    MOV I,0

```

```

ROWLOOP:MOV AL,I

```

```

    MOV ROW,AL

```

```

    MOV COL,0

```

```

    CALL SET_LCD_POS

```

```

    MOV CMD,LC_AUT_WR

```

```

    CALL LCD_WRITE_COMMAND

```

```

    MOV J,0

```

```

COLLOOP:MOV AL,I

```

```

MOV MUL2,LCD_WIDTH
MUL MUL2
ADD AL,J
ADC AH,0
MOV SI,AX
MOV BX,OFFSET IMAGE
MOV AL,[BX][SI]
MOV DAT,AL
CALL LCD_WRITE_DATA
INC J
CMP J,LCD_WIDTH
JNE COLLOOP
MOV CMD,LC_AUT_OVR
CALL LCD_WRITE_COMMAND
INC I
CMP I,LCD_HEIGHT
JNE ROWLOOP
JMP $
STATUS_BIT_01 PROC NEAR                                ;读写指令和读写数据状态位
    PUSH CX
    MOV CX,5
NEXT01:MOV DX,LCMCW
    IN AL,DX
    AND AL,03H
    CMP AL,03H
    JZ EXIT01
    LOOP NEXT01
EXIT01:MOV STATUS,CL
    POP CX
    RET
STATUS_BIT_01 ENDP
STATUS_BIT_3 PROC NEAR                                ;数据自动写状态位
    PUSH CX
    MOV CX,5
NEXT03:MOV DX,LCMCW
    IN AL,DX
    AND AL,08H
    CMP AL,08H
    JZ EXIT03

```



```
        LOOP NEXT03
EXIT03: MOV STATUS, CL
        POP CX
        RET
STATUS_BIT_3 ENDP
LCD_WRITE_COMMAND_P2 PROC NEAR           ;写双参数指令
        CALL STATUS_BIT_01
        CMP STATUS, 0
        JNE P2WR1
        RET
P2WR1:  MOV DX, LCMDW
        MOV AL, PARA1
        OUT DX, AL
        CALL STATUS_BIT_01
        CMP STATUS, 0
        JNE P2WR2
        RET
P2WR2:  MOV DX, LCMDW
        MOV AL, PARA2
        OUT DX, AL
        CALL STATUS_BIT_01
        CMP STATUS, 0
        JNE P2WR3
        RET
P2WR3:  MOV DX, LCMCW
        MOV AL, CMD
        OUT DX, AL
        RET
LCD_WRITE_COMMAND_P2 ENDP
LCD_WRITE_COMMAND_P1 PROC NEAR           ;写单参数指令
        CALL STATUS_BIT_01
        CMP STATUS, 0
        JNE P1WR1
        RET
P1WR1:  MOV DX, LCMDW
        MOV AL, PARA1
        OUT DX, AL
        CALL STATUS_BIT_01
        CMP STATUS, 0
```

```
JNE P1WR2
RET
P1WR2:MOV DX,LCMCW
MOV AL,CMD
OUT DX,AL
RET
LCD_WRITE_COMMAND_P1 ENDP
LCD_WRITE_COMMAND PROC NEAR ;写无参数指令
CALL STATUS_BIT_01
CMP STATUS,0
JNE WRCMD
RET
WRCMD:MOV DX,LCMCW
MOV AL,CMD
OUT DX,AL
RET
LCD_WRITE_COMMAND ENDP
LCD_WRITE_DATA PROC NEAR ;写无参数指令
CALL STATUS_BIT_3
CMP STATUS,0
JNE WRDATA
RET
WRDATA:MOV DX,LCMDW
MOV AL,DAT
OUT DX,AL
RET
LCD_WRITE_DATA ENDP
SET_LCD_POS PROC NEAR ;设置当前地址
MOV AL,ROW
MOV MUL2,LCD_WIDTH
MUL MUL2
ADD AL,COL
ADC AH,0
MOV DX,0
MOV DIV2,256
DIV DIV2
MOV PARA1,DL
MOV PARA2,AL
MOV CMD,LC_ADD_POS
```



```

CALL LCD_WRITE_COMMAND_P2
RET
SET_LCD_POS ENDP
CLS PROC NEAR ;清屏
    MOV PARA1,0
    MOV PARA2,0
    MOV CMD,LC_ADD_POS
    CALL LCD_WRITE_COMMAND_P2
    MOV CMD,LC_AUT_WR
    CALL LCD_WRITE_COMMAND
    MOV CX, 2000H
CLSAGAIN:CALL STATUS_BIT_3
    MOV DAT,0
    CALL LCD_WRITE_DATA
    LOOP CLSAGAIN
    MOV CMD,LC_AUT_OVR
    CALL LCD_WRITE_COMMAND
    MOV PARA1,0
    MOV PARA2,0
    MOV CMD,LC_ADD_POS
    CALL LCD_WRITE_COMMAND_P2
    RET
CLS ENDP
LCD_INITIALISE PROC NEAR ;初始化 LCD
    MOV PARA1,0
    MOV PARA2,0
    MOV CMD,LC_TXT_STP
    CALL LCD_WRITE_COMMAND_P2
    MOV PARA1,LCD_WIDTH
    MOV PARA2,0
    MOV CMD,LC_TXT_WID
    CALL LCD_WRITE_COMMAND_P2
    MOV PARA1,0
    MOV PARA2,0
    MOV CMD,LC_GRH_STP
    CALL LCD_WRITE_COMMAND_P2
    MOV PARA1,LCD_WIDTH
    MOV PARA2,0
    MOV CMD,LC_GRH_WID

```



```
;.....(略)
```

```
END
```

4.5.3 PG160128 图形液晶显示器

利用 PG160128 图形液晶显示器显示汉字，其电路原理如图 4-21 所示。获取待显示汉字的点阵数据时需要使用辅助工具软件 PCtoLCD 字模软件。

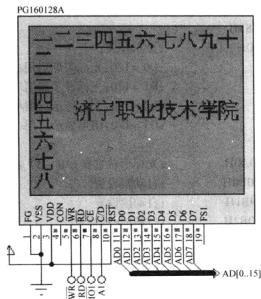


图 4-21 PG160128 图形液晶显示器显示汉字的电路原理

参考程序：

```
.MODEL SMALL
```

```
;T6963 端口定义
```

```
LCMDW EQU 0200H
```

```
LCMCW EQU 0202H
```

```
;DISRAM_SIZE 7FFFH
```

```
TXTSTART EQU 0000H
```

```
GRSTART EQU 6800H
```

```
CGRAMSTART EQU 7800H
```

```
HZ_CHR_HEIGHT EQU 16
```

```
;数据口
```

```
;命令口
```

```
;设置显示 RAM 大小
```

```
;设置文本区的起始地址
```

```
;设置图像区的地址
```

```
;设置 CGRAM 的起始地址
```

```
;T6963C 命令定义
```

```
LC_CUR_POS EQU 21H
```

```
LC_CGR_POS EQU 22H
```

```
LC_ADD_POS EQU 24H
```

```
;光标位置设置
```

```
;CGRAM 偏置地址设置
```

```
;地址指针位置
```

```

LC_TXT_STP EQU 40H      ;文本区首址
LC_TXT_WID EQU 41H      ;文本区宽度
LC_GRH_STP EQU 42H      ;图形区首址
LC_GRH_WID EQU 43H      ;图形区宽度
LC_MOD_OR EQU 80H       ;显示方式
LC_MOD_XOR EQU 81H
LC_MOD_AND EQU 82H
LC_MOD_TCH EQU 83H
LC_DIS_SW EQU 90H       ;显示开关:
                        ;D0 = 1/0:光标闪烁启用/禁用
                        ;D1 = 1/0:光标显示启用/禁用
                        ;D2 = 1/0:文本显示启用/禁用
                        ;D3 = 1/0:图形显示启用/禁用

LC_CUR_SHP EQU 0A0H     ;光标形状选择:A0 ~ A7 表示光标占的行数
LC_AUT_WR EQU 0B0H      ;自动写设置
LC_AUT_RD EQU 0B1H      ;自动读设置
LC_AUT_OVR EQU 0B2H     ;自动读/写结束
LC_INC_WR EQU 0C0H      ;数据写地址加 1
LC_INC_RD EQU 0C1H      ;数据读地址加 1
LC_DEC_WR EQU 0C2H      ;数据写地址减 1
LC_DEC_RD EQU 0C3H      ;数据读地址减 1
LC_NOC_WR EQU 0C4H      ;数据写地址不变
LC_NOC_RD EQU 0C5H      ;数据读地址不变
LC_SCN_RD EQU 0E0H      ;屏读
LC_SCN_CP EQU 0E8H      ;屏复制
LC_BIT_OP EQU 0F0H      ;位操作:D0 ~ D2 定义 D0 ~ D7 位,D3:1 置位/0
                        ;清除

LCD_WIDTH EQU 20;       ;宽 160 像素,160/8 = 20B
LCD_HEIGHT EQU 128;
.8086
.STACK
.CODE
.STARTUP
    CALL LCD_INITIALISE
    MOV COUNT,10        ;横向显示一二三四五六七八九十
    MOV X,0
    MOV Y,0
    MOV BX,OFFSET HZK

```

```

HANGLOOP:CALL DISP_HZ
          ADD BX,32
          INC X
          INC X
          DEC COUNT
          JNZ HANGLOOP
          MOV COUNT,7           ;纵向显示一二三四五六七八九十
          MOV X,0
          MOV Y,16
          MOV BX,OFFSET HZK +32
LIELOOP:CALL DISP_HZ
          ADD BX,32
          MOV AL,16
          ADD Y,AL
          DEC COUNT
          JNZ LIELOOP
          MOV COUNT,8           ;显示作者单位
          MOV X,4
          MOV Y,56
          MOV BX,OFFSET ADDRES
ADDRESSLOOP:CALL DISP_HZ
          ADD BX,32
          INC X
          INC X
          DEC COUNT
          JNZ ADDRESSLOOP
          JMP $
STATUS_BIT_01 PROC NEAR       ;读写指令和读写数据状态位
          PUSH CX
          MOV CX,5
NEXT01:MOV DX,LCMCW
          IN AL,DX
          AND AL,03H
          CMP AL,03H
          JZ EXIT01
          LOOP NEXT01
EXIT01:MOV STATUS,CL
          POP CX
          RET

```

```
STATUS_BIT_01 ENDP
STATUS_BIT_3 PROC NEAR      ;数据自动写状态位
    PUSH CX
    MOV CX,5
NEXT03 :MOV DX,LCMCW
    IN AL,DX
    AND AL,08H
    CMP AL,08H
    JZ EXIT03
    LOOP NEXT03
EXIT03:MOV STATUS,CL
    POP CX
    RET
STATUS_BIT_3 ENDP
LCD_WRITE_COMMAND_P2 PROC NEAR      ;写双参数指令
    CALL STATUS_BIT_01
    CMP STATUS,0
    JNE P2WR1
    RET
P2WR1 :MOV DX,LCMDW
    MOV AL,PARA1
    OUT DX,AL
    CALL STATUS_BIT_01
    CMP STATUS,0
    JNE P2WR2
    RET
P2WR2 :MOV DX,LCMDW
    MOV AL,PARA2
    OUT DX,AL
    CALL STATUS_BIT_01
    CMP STATUS,0
    JNE P2WR3
    RET
P2WR3:MOV DX,LCMCW
    MOV AL,CMD
    OUT DX,AL
    RET
LCD_WRITE_COMMAND_P2 ENDP
LCD_WRITE_COMMAND_P1 PROC NEAR      ;写单参数指令
```

```

CALL STATUS_BIT_01
CMP STATUS,0
JNE P1WR1
RET
P1WR1:MOV DX,LCMDW
MOV AL,PARA1
OUT DX,AL
CALL STATUS_BIT_01
CMP STATUS,0
JNE P1WR2
RET
P1WR2:MOV DX,LCMCW
MOV AL,CMD
OUT DX,AL
RET
LCD_WRITE_COMMAND_P1 ENDP
LCD_WRITE_COMMAND PROC NEAR ;写无参数指令
CALL STATUS_BIT_01
CMP STATUS,0
JNE WRCMD
RET
WRCMD:MOV DX,LCMCW
MOV AL,CMD
OUT DX,AL
RET
LCD_WRITE_COMMAND ENDP
LCD_WRITE_DATA PROC NEAR ;写无参数指令
CALL STATUS_BIT_3
CMP STATUS,0
JNE WRDATA
RET
WRDATA:MOV DX,LCMDW
MOV AL,DAT
OUT DX,AL
RET
LCD_WRITE_DATA ENDP
SET_LCD_POS PROC NEAR ;设置当前地址
MOV AL,ROW
MOV MUL2,LCD_WIDTH

```

```

MUL MUL2
ADD AL,COL
ADC AH,0
MOV DX,0
MOV DIV2,256
DIV DIV2
MOV PARA1,DL
MOV PARA2,AL
MOV CMD,LC_ADD_POS
CALL LCD_WRITE_COMMAND_P2
RET
SET_LCD_POS ENDP
CLS PROC NEAR ;清屏
MOV PARA1,0
MOV PARA2,0
MOV CMD,LC_ADD_POS
CALL LCD_WRITE_COMMAND_P2
MOV CMD,LC_AUT_WR
CALL LCD_WRITE_COMMAND
MOV CX,2000H
CLSAGAIN:CALL STATUS_BIT_3
MOV DAT,0
CALL LCD_WRITE_DATA
LOOP CLSAGAIN
MOV CMD,LC_AUT_OVR
CALL LCD_WRITE_COMMAND
MOV PARA1,0
MOV PARA2,0
MOV CMD,LC_ADD_POS
CALL LCD_WRITE_COMMAND_P2
RET
CLS ENDP
LCD_INITIALISE PROC NEAR ;初始化 LCD
MOV PARA1,0
MOV PARA2,0
MOV CMD,LC_TXT_STP
CALL LCD_WRITE_COMMAND_P2
MOV PARA1,LCD_WIDTH
MOV PARA2,0

```

```

MOV CMD,LC _ TXT _ WID
CALL LCD _ WRITE _ COMMAND _ P2
MOV PARA1,0
MOV PARA2,0
MOV CMD,LC _ GRH _ STP
CALL LCD _ WRITE _ COMMAND _ P2
MOV PARA1,LCD _ WIDTH
MOV PARA2,0
MOV CMD,LC _ GRH _ WID
CALL LCD _ WRITE _ COMMAND _ P2
MOV PARA1,CGRAMSTART SHR 11
MOV CMD,LC _ CGR _ POS
CALL LCD _ WRITE _ COMMAND _ P1
MOV CMD,LC _ CUR _ SHP OR 01H
CALL LCD _ WRITE _ COMMAND
MOV CMD,LC _ MOD _ OR
CALL LCD _ WRITE _ COMMAND
MOV CMD,LC _ DIS _ SW OR 08H
CALL LCD _ WRITE _ COMMAND
RET
LCD _ INITIALISE  ENDP
DISP _ HZ PROC NEAR
    MOV K,0

HZLOOP: MOV SI,K                                ;汉字显示子程序
        SHL SI,1
        MOV AL,[BX][SI]
        MOV DAT1,AL
        INC SI
        MOV AL,[BX][SI]
        MOV DAT2,AL
        MOV AL,Y
        ADD AL, BYTE PTR K
        MOV ROW,AL
        MOV AL,X
        MOV COL,AL
        CALL SET _ LCD _ POS
        MOV CMD,LC _ AUT _ WR
        CALL LCD _ WRITE _ COMMAND

```

```

MOV AL,DAT1
MOV DAT,AL
CALL LCD_WRITE_DATA
MOV AL,DAT2
MOV DAT,AL
CALL LCD_WRITE_DATA
MOV CMD,LC_AUT_OVR
CALL LCD_WRITE_COMMAND
INC K
CMP K,HZ_CHR_HEIGHT
JNE HZLOOP
DISP_HZ ENDP
DELAY PROC NEAR ;延时子程序
    PUSH BX
    PUSH CX
    MOV BX,500
LP1: MOV CX,469
LP2: LOOP LP2
    DEC BX
    JNZ LP1
    POP CX
    POP BX
    RET
DELAY ENDP
.DATA
    RET
DELAY ENDP
.DATA
STATUS DB 5 ;最多5次读液晶状态,非0有应答,可读写
PARA1 DB ? ;液晶指令参数变量1
PARA2 DB ? ;液晶指令参数变量2
CMD DB ? ;液晶指令变量
DAT DB ? ;液晶数据变量
DAT1 DB ? ;液晶数据变量
DAT2 DB ? ;液晶数据变量
ROW DB ? ;行
COL DB ? ;列
MUL2 DB ? ;乘数
DIV2 DW ? ;除数

```



```

X    DB 0      ;行临时变量
Y    DB 0      ;列临时变量
K    DW 0      ;字高临时变量
COUNT DB ?    ;计数器
;PCtoLCD 取模工具:阴码,顺向,逐行式
HZK DB 000H, 000H, 000H, 000H, 000H, 000H, 000H, 000H;
DB 000H, 000H, 000H, 000H, 000H, 004H, 07FH, 0FEH;
DB 000H, 000H, 000H, 000H, 000H, 000H, 000H, 000H;
DB 000H, 000H, 000H, 000H, 000H, 000H, 000H, 000H;"—" ,0
.....(略)
END

```

4.6 8255A 可编程接口芯片

4.6.1 8255A 可编程接口芯片 PC 口应用

要求通过用 8255A 芯片的 PC2 位控制 LED 每隔一段时间闪烁, 其电路原理如图4-22所示。

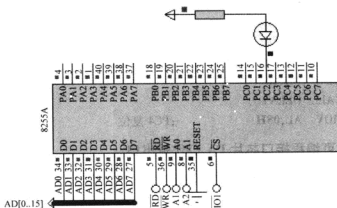


图 4-22 8255A 可编程接口芯片 PC 口应用的电路原理

1) 参考程序。

```

.MODEL SMALL
.8086
.STACK
.CODE
.STARTUP

```

```

        MOV DX,0206H      ;控制寄存器地址
AA:     MOV AL,05H         ;PC2 置位

```



```

OUT DX,AL
CALL DELAY
MOV AL,04H           ;PC2 复位
OUT DX,AL
CALL DELAY
JMP AA
DELAY PROC NEAR      ;延时子程序
    MOV BX,500
LP1:  MOV CX,469
LP2:  LOOP LP2
      DEC BX
      JNZ LP1
      RET
DELAY ENDP
END

```

2) 8255A 调试窗口。暂停程序执行，右键单击 8255A 仿真芯片，在弹出的快捷菜单中，勾选最下面的选项“8255 Internal Status Window”，单步执行并观察 8255A 输入/输出端口、控制寄存器和状态寄存器的数据变化。

3) 练习。通过 8255A 芯片的 PC4 位控制 LED 每隔一段时间闪烁。

4) 提示。先修改电路（见图 4-22），LED 阴极改接 PC4。

修改源程序，部分改动如下述**黑体**字部分。

```

AA:  MOV AL,09H           ;PC4 置位
      OUT DX,AL
      CALL DELAY
      MOV AL,08H           ;PC4 复位

```

4.6.2 8255A 可编程接口芯片 PA 口作输出口

用 8255A 可编程接口芯片的 PA 口作输出口，通过查状态表的方法控制一路循环彩灯，其电路原理如图 4-23 所示。

1) 参考程序。

```

.MODEL SMALL
.8086
.STACK
.CODE
.STARTUP
    MOV DX,0206H      ;控制寄存器地址
    MOV AL,80H        ;A 口输出
    OUT DX,AL
AGAIN: MOV SI, OFFSET SITUATION

```



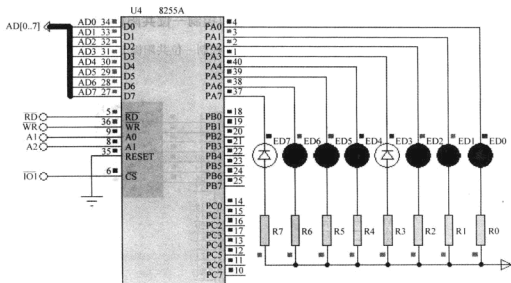


图 4-23 8255A 可编程接口芯片 PA 口做输出电路的原理

```

MOV    DX,0200H                ; A 口地址
NEXT:  MOV    AL, [SI]
        OUT    DX,AL
        CALL   DELAY
        ADD    SI, 1             ; 下一个待显示状态
        CMP    SI, OFFSET SIT_END
        JB     NEXT
        JMP     AGAIN
DELAY  PROC    NEAR              ; 延时子程序
        MOV    BX,500
LP1:   MOV    CX,469
LP2:   LOOP   LP2
        DEC    BX
        JNZ    LP1
        RET
DELAY  ENDP
. DATA
SITUATION  DB  3FH,06H,5BH,4FH,66H,6DH,7DH,07H,7FH,6FH ;共阴极
SIT_END = $
END

```

2) 练习。先修改电路 (见图 4-23), 将 8 只 LED 数码管删除用一位 7 段共阴极数码管 (7SEG-COM-CAT-BLUE) 替换, 使数码管显示从 0~9, 然后回 0 循环。

4.6.3 8255A 可编程接口芯片 PA 口作输出口控制一位共阳极 LED 数码管

使用 8255A 可编程接口芯片的 PA 口作输出口，控制一位共阳极 LED 数码管显示从 0 ~ F，熄灭后回 0 重新显示，其电路原理如图 4-24 所示。

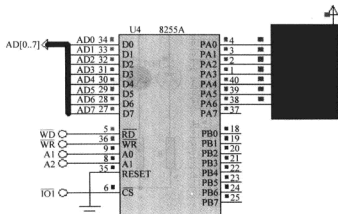


图 4-24 8255A 可编程接口芯片 PA 口作输出口控制一位共阳极 LED 数码管的电路原理

1) 参考程序。

```
.MODEL SMALL
.8086
.STACK
.CODE
.STARTUP
```

```
    MOV DX,0206H           ;控制寄存器地址
    MOV AL,80H             ;A 口输出
    OUT DX,AL
AGAIN: MOV SI, OFFSET SITUATION
    MOV DX,0200H           ;A 口地址
NEXT:  MOV AL, [SI]
    OUT DX,AL
    CALL DELAY
    ADD SI, 1               ;NEXT SITUATION
    CMP SI, OFFSET SIT_END
    JB NEXT
    JMP AGAIN
DELAY PROC NEAR
    MOV BX,500
LP1:  MOV CX,469
LP2:  LOOP LP2
```

新华书店
PDG

```

DEC    BX
JNZ    LP1
RET
DELAY  ENDP
.DATA
SITUATION  DB 0C0H,0F9H,0A4H,0B0H,099H,092H,082H,0F8H,080H,090H
            DB 088H,083H,0C6H,0A1H,086H,08EH,0BFH,0FFH    ;共阳极
SIT_END = $
END

```

2) 练习。修改源程序使 LED 数码管的数字到序显示。

3) 提示。改动部分如**黑体字**所示。

```

AGAIN:  MOV  SI, SIT_END-1
        MOV  DX,0200H          ;A 口地址
NEXT:   MOV  AL, [SI]
        OUT  DX,AL
        CALL DELAY
        DEC  SI                ;NEXT SITUATION
        CMP  SI,OFFSET SITUATION
        JAE  NEXT

```

4.6.4 8255A 可编程接口芯片 PA 口方式 0 作输入口、PB 口方式 0 作输出口

使用 8255A 可编程接口芯片 PA 口作输入口，检测 8 个按键状态，PB 口作输出口，显示按键状态。其电路原理如图 4-25 所示。

1) 参考程序。

```

.MODEL  SMALL
.8086
.STACK
.CODE
.STARTUP
        MOV  DX,0206H          ;控制寄存器地址
        MOV  AL,90H            ;A 口输入、方式 0,B 口输出、方式 0
        OUT  DX,AL
AGAIN:  MOV  DX,0200H          ;A 口地址
        IN   AL,DX
        MOV  DX,0202H          ;B 口地址
NEXT:   OUT  DX,AL
        JMP  AGAIN
.DATA
END

```

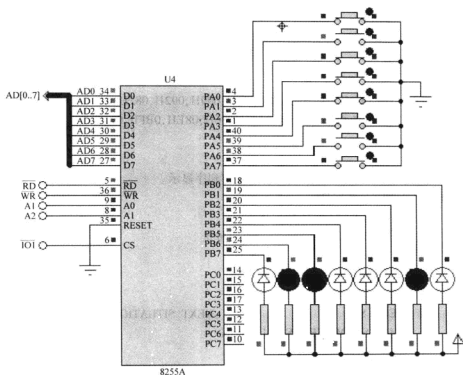


图 4-25 8255A 可编程接口芯片 PA 口方式 0 输入 PB 口方式 0 作输出的电路原理

2) 练习。将上述例子改成用 PB 口作输入检测按键状态, PA 口作输出口显示按键状态。

3) 提示。

① 先修改电路中 8 只 LED 阴极改接 PA 口, 开关改接 PB 口。

② 修改源程序, 部分改动如黑体字部分所示。

```

MOV DX,0206H      ;控制寄存器地址
MOV AL,82H        ;A 口输出、方式 0,B 口输入、方式 0
OUT DX,AL

AGAIN: MOV DX,0202H ;B 口地址
       IN  AL,DX
       MOV DX,0200H ;A 口地址
NEXT:  OUT DX,AL
       JMP AGAIN
  
```

4.6.5 8255A 可编程接口芯片作按键检测及多位 LED 数码管显示接口

如图 4-26 所示, 使用 8255A 可编程接口芯片的 PA 口用作 8 位共阴极 LED 数码管的段码输出端口, PB 口用作 8 位 LED 数码管的位控码输出端口, PC1 用作独立式按键的输入检

测端口。8 位 LED 初始显示 76543210，每按下一次按键，LED 数码管的数字向左循环移动一位。

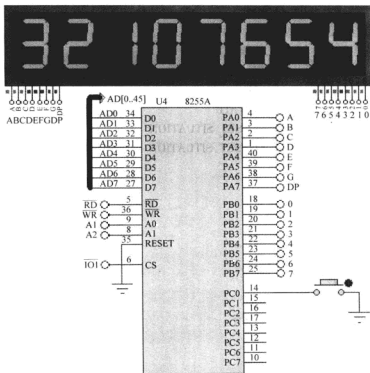


图 4-26 8255A 可编程接口芯片作按键检测和多位 LED 数码管显示接口的电路原理

参考程序：

.MODEL SMALL

.8086

.STACK

.CODE

.STARTUP

MOV DX,0206H

;控制寄存器地址

MOV AL,81H

;A 口输出,B 口输出,PC0 输入

OUT DX,AL

AGAIN: CALL DISP

KEY: MOV DX,0204H

;C 口地址

IN AL,DX

TEST AL,01H

JNZ AGAIN

CALL DEALY20MS

MOV DX,0204H

;C 口地址

```

IN      AL,DX
TEST    AL,01H
JNZ     AGAIN
KEY _ WAIT: CALL DISP
MOV     DX,0204H           ;C 口地址
IN      AL,DX
TEST    AL,01H
JZ      KEY _ WAIT        ;等待键释放
MOV     BX,OFFSET SITUATION
MOV     SI, OFFSET SITUATION
MOV     AL,[SI]
MOV     AH,AL
MOV     CX,7
TRANS:  INC     SI         ;移动显示缓冲区
MOV     AL,[SI]
MOV     [BX],AL
INC     BX
LOOP    TRANS
MOV     [BX],AH
JMP     AGAIN
DISP    PROC           ;显示子程序
MOV     BP, 0FFFEH
MOV     SI, OFFSET SITUATION
NEXT:   MOV     DX,0202H   ;B 口地址
MOV     AX,0FFH
OUT     DX,AL             ;关显示
MOV     DX,0200H         ;A 口地址
MOV     AL, [SI]
OUT     DX,AL
MOV     DX,0202H         ;B 口地址
MOV     AX,BP
OUT     DX,AL
CALL    DELAY
STC
RCL     BP,1
ADD     SI, 1             ; NEXT SITUATION
CMP     SI, OFFSET SIT _ END
JB      NEXT
RET

```



```

DISP      ENDP
DEALY20MS PROC                                ;KEY 消抖延时子程序
            MOV  BX,5
LP1:       CALL DISP
            DEC  BX
            JNZ  LP1
            RET
DEALY20MS  ENDP
DELAY     PROC  NEAR                            ;扫描延时
            MOV  CX,300
LP2:       LOOP LP2
            RET
DELAY     ENDP
. DATA
SITUATION  DB 3FH,06H,5BH,4FH,66H,6DH,7DH,07H;共阴极,7FH,6FH
SIT_END = $
END

```

4.6.6 8255A 可编程接口芯片 PB 口方式 1 作输出口

设置 8255A 可编程接口芯片 PB 口为选通输出方式，即工作于方式 1，控制 8 只 LED 轮流点亮，如图 4-27 所示。

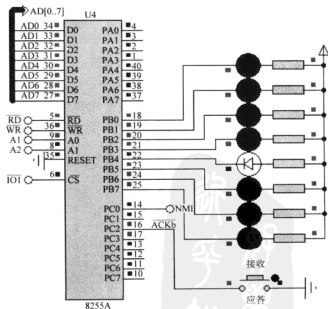


图 4-27 8255A 可编程接口芯片 PB 口方式 1 作输出口的电路原理

参考程序:

; 查询方式

.MODEL SMALL

.8086

.STACK

.CODE

.STARTUP

MOV DX,0206H ;控制寄存器地址

MOV AL,084H ;port A mode 0 output,port b mode 1 out

OUT DX,AL

NEXT:

WAITPB:MOV DX,0204H ;状态口地址

IN AL,DX

TEST AL,02H ;OBF_B

JZ WAITPB

MOV DX,0202H ;B 口地址

MOV AL,LED_STATUS

OUT DX,AL

ROL LED_STATUS,1

CALL DELAY

JMP NEXT

DELAY PROC NEAR

PUSH BX

PUSH CX

MOV BX,500

LP1: MOV CX,469

LP2: LOOP LP2

DEC BX

JNZ LP1

POP CX

POP BX

RET

DELAY ENDP

.DATA

LED_STATUS DB 0FEH

END

;中断方式

.MODEL SMALL

.8086

```

.STACK
.CODE
.STARTUP
    PUSH ES                      ;ES 入栈
    XOR AX, AX
    MOV ES, AX
    MOV AL, 02H
    XOR AH, AH
    SHL AX, 1                    ;计算 2 号中断矢量在中断矢量表中的地址
    SHL AX, 1
    MOV SI, AX                   ;SI 指向 2 号中断矢量地址
    MOV AX, OFFSET NMI_SERVICE
    MOV ES:[SI], AX             ;保存中断服务程序的 IP 地址
    INC SI
    INC SI
    MOV BX, CS
    MOV ES:[SI], BX             ;保存中断服务程序的 CS 段基址
    POP ES
    STI
    MOV DX, 0206H               ;控制寄存器地址
    MOV AL, 05H                 ;允许 B 口输出中断
    OUT DX, AL
    MOV AL, 084H                ;port A mode 0 output, port b mode 1 out
    OUT DX, AL
    MOV DX, 0202H               ;B 口地址
    MOV AL, 0FEH
    OUT DX, AL
    JMP $
NMI_SERVICE:                    ;中断服务程序
    MOV DX, 0202H               ;B 口地址
    MOV AL, LED_STATUS
    OUT DX, AL
    ROL LED_STATUS, 1
EXIT: IRET
.DATA
LED_STATUS DB 0FEH
END

```

4.6.7 8255A 可编程接口芯片 PB 口方式 1 作输入口

设置 8255A 可编程接口芯片 PB 口为选通输入方式，即工作于方式 1，检测 8 个开关状

态,并用 PA 口作输出口显示开关状态,如图 4-28 所示。

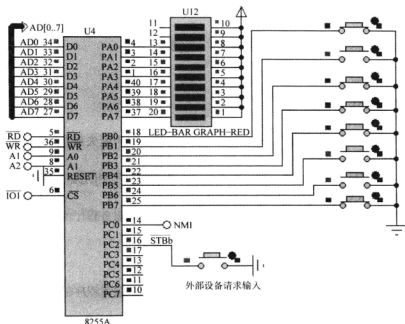


图 4-28 8255A 可编程接口芯片 PB 口方式 1 作输入口的电路原理

参考程序:

;查询方式

.MODEL SMALL

.8086

.STACK

.CODE

.STARTUP

MOV DX,0206H

;控制寄存器地址

MOV AL,086H

;port A mode 0 output,port b mode 1 input

OUT DX,AL

NEXT:

WAITPB:MOV DX,0204H

;状态口地址

IN AL,DX

TEST AL,02H

;IBF_B

JZ WAITPB

MOV DX,0202H

;B 口地址

IN AL,DX

MOV DX,0200H

;A 口地址

OUT DX,AL

```

;CALL DELAY
JMP NEXT
DELAY PROC NEAR
    PUSH BX
    PUSH CX
    MOV BX,500
    LP1: MOV CX,469
    LP2: LOOP LP2
    DEC BX
    JNZ LP1
    POP CX
    POP BX
    RET
DELAY ENDP
. DATA
END
;中断方式
. MODEL SMALL
. 8086
. STACK
. CODE
. STARTUP
    PUSH ES ;ES 入栈
    XOR AX, AX
    MOV ES, AX
    MOV AL, 02H
    XOR AH, AH
    SHL AX, 1 ;计算 2 号中断矢量在中断矢量表中的地址
    SHL AX, 1
    MOV SI, AX ;SI 指向 2 号中断矢量地址
    MOV AX, OFFSET NMI_SERVICE
    MOV ES:[SI], AX ;保存中断服务程序的 IP 地址
    INC SI
    INC SI
    MOV BX, CS
    MOV ES:[SI], BX ;保存中断服务程序的 CS 段基址
    POP ES
    STI

```

```

MOV DX,0206H      ;控制寄存器地址
MOV AL,05H        ;允许 B 口输入中断
OUT DX,AL
MOV AL,086H       ;port A mode 0 output,port b mode 1 input
OUT DX,AL
MOV DX,0202H      ;B 口地址
IN AL,DX          ;检测 B 口状态
MOV DX,0200H      ;A 口地址
OUT DX,AL         ;A 口输出
JMP $

NMI_SERVICE:      ;中断服务程序
MOV DX,0202H      ;B 口地址
IN AL,DX
MOV DX,0200H      ;A 口地址
OUT DX,AL         ;A 口输出

EXIT: IRET

. DATA
END

```

4.6.8 8255A 可编程接口芯片 PA 口方式 1 作输出口

设置 8255A 可编程接口芯片 PA 口为选通输出方式，即工作于方式 1，控制条形 LED 二极管轮流点亮。其电路原理如图 4-29 所示。

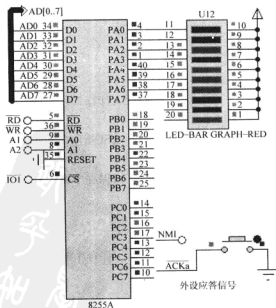


图 4-29 8255A 可编程接口芯片 PA 口方式 1 作输出口的电路原理

参考程序:

;查询方式

.MODEL SMALL

.8086

.STACK

.CODE

.STARTUP

```

        MOV    DX,0206H           ;控制寄存器地址
        MOV    AL,0A0H           ;port A mode 1 output
        OUT    DX,AL

```

LOP:

WAITPA: MOV DX,0204H ;状态口地址

IN AL,DX

TEST AL,80H ;OBF_a

JZ WAITPA

MOV AL,LED_STATUS

ROL LED_STATUS,1

MOV DX,0200H ;A口地址

OUT DX,AL

CALL DELAY

JMP LOP

DELAY PROC NEAR ;延时子程序

PUSH BX

PUSH CX

MOV BX,500

LP1: MOV CX,469

LP2: LOOP LP2

DEC BX

JNZ LP1

POP CX

POP BX

RET

DELAY ENDP

.DATA

LED_STATUS DB 0FEH

END

;中断方式

.MODEL SMALL

.8086



```

.STACK
.CODE
.STARTUP
    PUSH ES                ;ES 入栈
    XOR AX, AX
    MOV ES, AX
    MOV AL, 02H
    XOR AH, AH
    SHL AX, 1              ;计算 2 号中断矢量在中断矢量表中的地址
    SHL AX, 1
    MOV SI, AX             ;SI 指向 2 号中断矢量地址
    MOV AX, OFFSET NMI_SERVICE
    MOV ES:[SI], AX       ;保存中断服务程序的 IP 地址
    INC SI
    INC SI
    MOV BX, CS
    MOV ES:[SI], BX       ;保存中断服务程序的 CS 段基址
    POP ES
    STI

    MOV DX, 0206H         ;控制寄存器地址
    MOV AL, 0DH           ;允许 A 口输出中断
    OUT DX, AL
    MOV AL, 0A0H          ;port A mode 1 output, port b mode 0 out
    OUT DX, AL
    MOV DX, 0200H         ;A 口地址
    MOV AL, 0FEH
    OUT DX, AL
    JMP $

NMI_SERVICE:             ;中断服务程序
    MOV DX, 0200H         ;A 口地址
    MOV AL, LED_STATUS
    OUT DX, AL
    ROL LED_STATUS, 1
EXIT: IRET
.DATA
LED_STATUS DB 0FEH
END

```



```

        OUT DX,AL
LOP:
WAITPA:MOV DX,0204H    ;状态口地址
        IN  AL,DX
        TEST AL,20H     ;IBFn
        JZ  WAITPA
        MOV DX,0200H    ;A 口地址
        IN  AL,DX
        MOV DX,0600H    ;Port 273
        OUT DX,AL
        JMP LOP
DELAY  PROC  NEAR
        PUSH BX
        PUSH CX
        MOV  BX,500
LP1:   MOV  CX,469
LP2:   LOOP LP2
        DEC  BX
        JNZ  LP1
        POP  CX
        POP  BX
        RET
DELAY  ENDP
. DATA
LED _STATUS DB 0FEH
END
;中断方式
. MODEL SMALL
. 8086
. STACK
. CODE
. STARTUP
START:
        CLI                                ;CPU 关中断
        MOV  DX,0400H                      ;8259A 初始化
        MOV  AX,13H
        OUT  DX,AX
        MOV  DX,0402H
        MOV  AX,00H
        ;ICW1,边沿触发,ICW4 NEEDED

```

```
OUT    DX,AX                      ;ICW2 中断类型 00H
MOV     AX,01H
OUT     DX,AX                      ;ICW4,正常 EOI
MOV     AX,00H
OUT     DX,AX                      ;OCW1, 开放所有中断

PUSH    DS
MOV     AX,0                      ;初始化中断向量表
MOV     DS,AX
MOV     DI,00H                    ;0 号中断
MOV     AX,OFFSET INT_SERVER
MOV     [DI],AX
ADD     DI,2
MOV     AX,CS
MOV     [DI],AX
POP     DS
STI                                           ;CPU 开中断

MOV     DX,0206H                  ;控制寄存器地址
MOV     AL,09H                    ;允许 A 口输入中断
OUT     DX,AL
MOV     AL,0D6H                    ;port A mode 2 input,port b mode 1 input
OUT     DX,AL
LOP: MOV DX,0601H                  ;主任务实现 8 只 LED 闪烁
MOV     AL,55H
OUT     DX,AL
CALL    DELAY
MOV     AL,0AAH
OUT     DX,AL
CALL    DELAY
JMP     LOP

INT_SERVER:CLI                      ;中断服务程序
PUSH    DX
PUSH    AX
MOV     DX,0400H
MOV     AL,0BH                    ;OCW3 读 ISR 命令字
OUT     DX,AL
```

```

IN      AL,DX
MOV     AH,AL                      ;暂存 ISR 中断服务寄存器的状态
IRQ0:   MOV     AL,AH
        AND     AL,01H
        JZ      EXIT_INT
        MOV     DX,0200H           ;A 口地址
        IN      AL,DX
        MOV     DX,0600H
        OUT     DX,AL
EXIT_INT:MOV DX,0400H
        MOV     AL,20H
        OUT     DX,AL              ;OCW2,发 EOI 清 ISR
        POP     AX
        POP     DX
        IRET                       ;中断返回

DELAY   PROC    NEAR              ;延时子程序
        PUSH    BX
        PUSH    CX
        MOV     BX,200
LP1:     MOV     CX,469
LP2:     LOOP    LP2
        DEC     BX
        JNZ     LP1
        POP     CX
        POP     BX
        RET
DELAY   ENDP
        .DATA
VALUE_273 DB 00H
        END

```

4.7 不可屏蔽中断

4.7.1 不可屏蔽中断控制 8 位 LED 交替闪烁

利用不可屏蔽中断检测按键状态，当有按键按下时，LED 状态取反一次。其电路原理如图 4-31 所示。

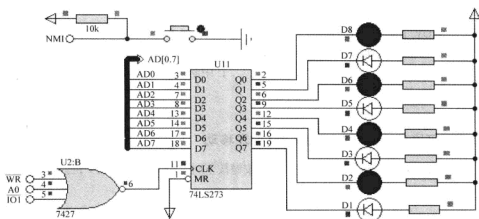


图 4-31 不可屏蔽中断控制 8 位 LED 交替闪烁的电路原理

参考程序：

```
.MODEL SMALL
```

```
.8086
```

```
.STACK
```

```
.CODE
```

```
.STARTUP
```

```
NMI_INIT: PUSH ES
```

```
; ES 入栈
```

```
XOR AX, AX
```

```
MOV ES, AX
```

```
MOV AL, 02H
```

```
XOR AH, AH
```

```
SHL AX, 1
```

```
; 计算 2 号中断矢量在中断矢量表中的地址
```

```
SHL AX, 1
```

```
MOV SI, AX
```

```
; SI 指向 2 号中断矢量地址
```

```
MOV AX, OFFSET NMI_SERVICE
```

```
MOV ES: [SI], AX
```

```
; 保存中断服务程序的 IP 地址
```

```
INC SI
```

```
INC SI
```

```
MOV BX, CS
```

```
MOV ES: [SI], BX
```

```
; 保存中断服务程序的 CS 段基址
```

```
POP ES
```

```
MOV AL, 55H
```

```
MOV DX, 0200H
```

```
; 273 输出口地址
```

```
OUT DX, AL
```

```
JMP $
```

```

NMI_SERVICE;                ; 中断服务程序
    XOR AL, 0FFH              ; 取反
    MOV DX, 0200H
    OUT DX, AL
EXIT: IRET                    ; 中断返回
. DATA
END

```

4.7.2 不可屏蔽中断控制 8 位 LED 向左移动

利用不可屏蔽中断检测按键状态，当有按键按下时，LED 向左移动 1 位。其电路原理如图 4-32 所示。

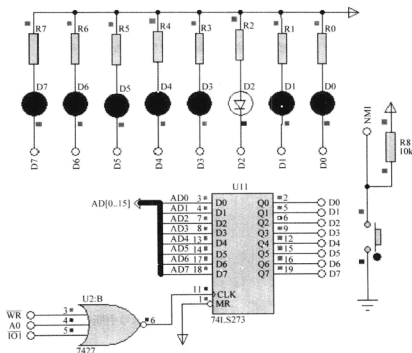


图 4-32 不可屏蔽中断控制 8 位 LED 向左移动的电路原理

1) 参考程序。

```

. MODEL SMALL
. 8086
. STACK
. CODE
. STARTUP
NMI_INIT: PUSH ES            ; ES 入栈

```

```

XOR AX, AX
MOV ES, AX
MOV AL, 02H
XOR AH, AH
SHL AX, 1                ; 计算 2 号中断矢量在中断矢量表中的地址
SHL AX, 1
MOV SI, AX                ; SI 指向 2 号中断矢量地址
MOV AX, OFFSET NMI_SERVICE
MOV ES: [SI], AX          ; 保存中断服务程序的 IP 地址
INC SI
INC SI
MOV BX, CS
MOV ES: [SI], BX          ; 保存中断服务程序的 CS 段基址
POP ES
MOV AL, 0FEH              ; LED 初值 (最上面的点亮)
MOV DX, 0200H             ; 273 输出口地址
OUT DX, AL
JMP $
NMI_SERVICE:              ; 中断服务程序
    ROL AL, 1              ; 左移
    MOV DX, 0200H
    OUT DX, AL
EXIT: IRET                 ; 中断返回
. DATA
END

```

2) 练习。利用不可屏蔽中断检测按键状态, 当有按键按下时, LED 向右移动 1 位。其电路原理如图 4-32 所示。

3) 提示。主程序中先使最低位点亮, 进入中断后向右移 1 位。

4.7.3 不可屏蔽中断控制 1 位 LED 数码管递减

利用不可屏蔽中断检测按键状态, 当有按键按下时, 七段 LED 数码管减 1, 减到 0 后回 9。其电路原理如图 4-33 所示。

1) 参考程序。

```

. MODEL SMALL
. 8086
. STACK
. CODE
. STARTUP
NMI_INIT: PUSH ES          ; ES 入栈

```

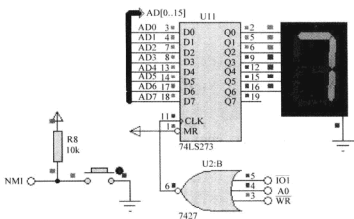


图 4-33 不可屏蔽中断控制 1 位 LED 数码管递减的电路原理

```

XOR AX, AX
MOV ES, AX
MOV AL, 02H
XOR AH, AH
SHL AX, 1                                ;计算 2 号中断矢量在中断矢量表中的地址
SHL AX, 1
MOV SI, AX                               ;SI 指向 2 号中断矢量地址
MOV AX, OFFSET NMI_SERVICE
MOV ES:[SI], AX                          ;保存中断服务程序的 IP 地址
INC SI
INC SI
MOV BX, CS
MOV ES:[SI], BX                          ;保存中断服务程序的 CS 段基址
POP ES
MOV SI, SIT_END - 1                      ;数码管初值指针
MOV DX, 0200H
LOOP0: MOV AL, [SI]
      OUT DX, AL
      JMP LOOP0
NMI_SERVICE:                             ;中断服务程序
      DEC SI
      CMP SI, OFFSET SITUATION           ;是否减到 0
      JAE EXIT
      MOV SI, SIT_END - 1                ;减到 0 回 9
EXIT: IRET
      .DATA

```



```

SITUATION DB 3FH,06H,5BH,4FH,66H,6DH,7DH,07H,7FH,6FH ;共阴极
SIT_END = $
END

```

2) 练习。利用不可屏蔽中断检测按键状态，当有按键按下时，LED 数码管增 1，到 9 后回 0，其电路原理如图 4-33 所示。

3) 提示。主程序中使 SI 指向 SITUATION 表首，进入中断后使 SI 加 1，并判断是否到表尾。

```

INC SI
CMP SI,OFFSET SIT_END
JB EXIT
MOV SI,OFFSET SITUATION

```

4.7.4 不可屏蔽中断控制 2 位 LED 数码管

利用不可屏蔽中断检测按键状态，当有按键按下时，7 段 LED 数码管加 1，加到 60 后回 0。其电路原理如图 4-34 所示。

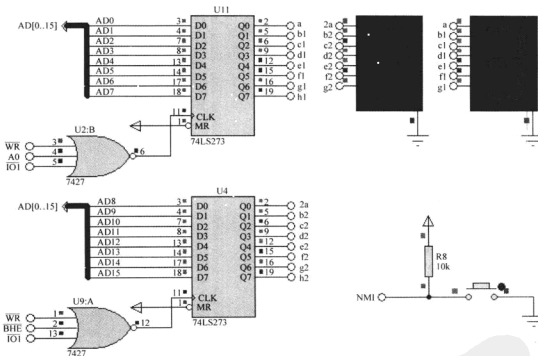


图 4-34 不可屏蔽中断控制 2 位 LED 数码管的电路原理

1) 参考程序。

```

.MODEL SMALL
.8086
.STACK

```



```
.CODE
.STARTUP
NMI_INIT: PUSH ES
          XOR AX, AX
          MOV ES, AX
          MOV AL, 02H
          XOR AH, AH
          SHL AX, 1
          SHL AX, 1
          MOV SI, AX
          MOV AX, OFFSET NMI_SERVICE
          MOV ES:[SI], AX
          INC SI
          INC SI
          MOV BX, CS
          MOV ES:[SI], BX
          POP ES
          MOV DX, 0200H
LOOP0: MOV BL, SEC
        AND BX, 000FH
        MOV SI, BX
        MOV AL, SITUATION[SI]
        MOV BL, SEC
        AND BX, 00F0H
        MOV CL, 4
        SHR BX, CL
        MOV SI, BX
        MOV AH, SITUATION[SI]
        OUT DX, AX
        JMP LOOP0

NMI_SERVICE:
        MOV AL, SEC
        ADD AL, 1
        DAA
        MOV SEC, AL
        CMP SEC, 60H
        JB EXIT
        MOV SEC, 0
```

```

EXIT: IRET
    .DATA
SEC      DB 23H
SITUATION DB 3FH,06H,5BH,4FH,66H,6DH,7DH,07H,7FH,6FH ;共阴级
SIT_END = $
END

```

2) 练习。利用不可屏蔽中断检测按键状态, 当有按键按下时, 数码管减 1, 减到 0 后返回 60, 其电路原理如图 4-34 所示。

3) 提示。主程序中使 SI 指向 SITUATION 表尾, 进入中断后使 SI 减 1, 并判断是否到表头。

```

MOV AL, SEC
SUB AL, 1
DAS
MOV SEC, AL
CMP SEC, 0
JGE EXIT
MOV SEC, 60H

```

4.8 8251A 可编程串行接口芯片

4.8.1 8251A 可编程串行接口芯片发送接口及编程

8251A 可编程串行接口芯片每隔一段时间发送一串字符, 虚拟终端取默认设置: 波特率为 9600B/s、无校验位、8 位数据位和 1 位停止位。其电路原理如图 4-35 所示。

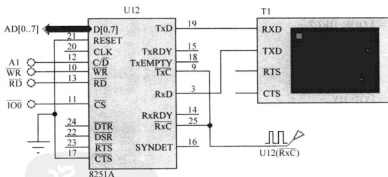


图 4-35 8251A 可编程串行接口芯片发送接口的电路原理

参考程序:

```

.MODEL SMALL
.8086

```

```

.STACK
.CODE
.STARTUP
    MOV DX,0002H           ;8251A 控制口地址送 DX
    MOV AL,4EH             ;1 位停止位、无校验、8 位数据位、16 分频
    OUT DX,AL
    MOV AL,17H             ;允许发送和接收
    OUT DX,AL
AGAIN:  LEA SI,FA           ;取数据区偏移地址
NEXT:   MOV DX,0002H       ;设置 8251A 控制口地址
WAIT1:  IN AL,DX           ;读取状态字
        AND AL,01H        ;检测 TxRDY 位
        JZ WAIT1          ;发送器缓冲器不空,等待
        MOV DX,0000H      ;设置 8251A 数据口地址
        MOV AL,[SI]       ;取数据
        OUT DX,AL         ;将数据传送给另一台计算机
        INC SI            ;修改数据区地址指针
        CMP SI,FA_END     ;是否到字符串尾
        JB  NEXT
        CALL DELAY        ;延时
AA1:    JMP AGAIN
DELAY   PROC NEAR
        PUSH BX
        PUSH CX
        MOV BX,50         ;4clk
DEL1:   MOV CX,5882       ;4
DEL2:   LOOP DEL2        ;17/5
        DEC BX           ;2
        JNZ DEL1         ;16/4
        POP CX
        POP BX
        RET
DELAY   ENDP
.DATA
FA DB'272000',0DH,0AH    ;将要传送的数据
   DB'ADDRESS;SHANDONGSHENG JININGSHI JINYULU',0DH,0AH
   DB'JINING ZHIYE JISHU XUEYUAN',0DH,0AH
   DB 0DH,0AH
FA_END = $
END

```

4.8.2 基于虚拟串行接口的 8251A 收发程序测试

编程实现将 8251A 收到的字符回送到发送端, 虚拟串口 P1 实现仿真串行接口到实际物理串行接口的转换。实验时可通过串行接口交叉线在两台个人计算机上通信, 一台个人计算机上运行仿真程序, 另一台个人计算机上运行串行接口调试助手; 如果个人计算机上有两个串行接口, 也可通过串行接口交叉线连接同一台个人计算机的两个串行接口, 仿真程序使用 COM1, 串行接口调试助手使用 COM2, 其他取默认设置: 波特率为 9600B/s、无校验位、8 位数据位和 1 位停止位。其电路原理如图 4-36 所示。

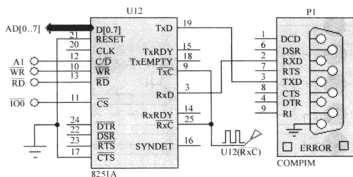


图 4-36 基于虚拟串行接口的 8251A 收发程序测试的电路原理

参考程序:

```
.MODEL SMALL
.8086
.STACK
.CODE
.STARTUP

MOV DX,0002H ;8251A 控制口地址送 DX
MOV AL,4EH   ;设置工作方式控制字,分频系数为 16
OUT DX,AL
MOV AL,07H   ;设置操作命令控制字
OUT DX,AL

NEXT: MOV DX,0002H ;设置 8251A 控制口地址
WAIT1: IN AL,DX    ;读取状态字
AND AL,02H      ;检测 RxRDY 位
JZ WAIT1        ;接收缓冲器空,等待
MOV DX,0000H    ;设置 8251A 数据口地址
IN AL,DX        ;读数据
OUT DX,AL
MOV DX,0002H    ;设置 8251A 控制口地址
WAIT2: IN AL,DX   ;读取状态字
AND AL,01H      ;检测 TxRDY 位
```

IMP NEXT

END

8251A 作为接口芯片输出数据送给一个串入/并出接口 74LS164 芯片, 控制 8 只 LED, 使 LED 管从下向上依次显示, 并不断循环。其电路原理如图 4-37 所示。

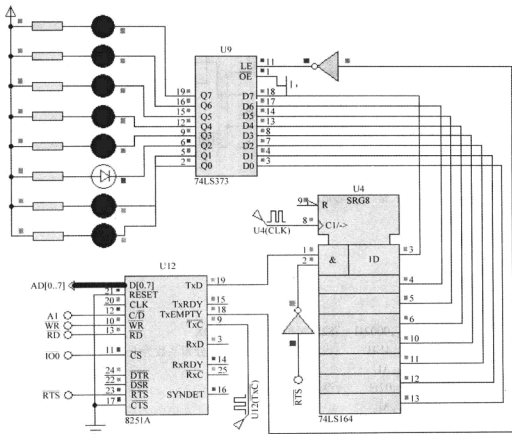


图 4-37 串并转换接口的电路原理

定义 8251A 发送方式为 1 位起始位、8 位数据位、1 位停止位、无奇偶校验位。用 74LS373 输出端口锁存 74LS164 并行输出数据, 并驱动 LED 点亮, 利用 TxEMPTY 发送移位寄存器信号作为 74LS373 输出口的锁存信号。

参考程序：

MODEL SMALL

8086

STACK

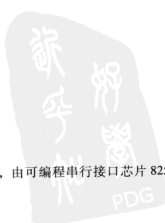
```

.CODE
.STARTUP
    MOV     DX,0002H      ;控制口地址送 DX
    MOV     AL,01001110B  ;异步 1 位停止位、无校验位、数据长度 8 位、16 分频
    OUT     DX,AL
AA1:  MOV     AH,08H       ;设置计数器初值
    MOV     AL,0FEH       ;低电平点亮指示灯
    MOV     BL,AL
AA:   MOV     AL,31H       ;操作命令控制字送 AL
    OUT     DX,AL
BUSY: MOV     DX,0002H
    IN      AL,DX         ;检测 8251A 工作状态
    AND     AL,05H        ;检查 TxRDY 是否为 1, TxEMPTY 是否为 1
    JZ      BUSY
    MOV     DX,0000H      ;8251A 数据口地址送 DX
    MOV     AL,BL
    OUT     DX,AL         ;输出数据
    CALL    DELAY
    ROL     BL,1          ;循环点亮指示灯
    DEC     AH
    JNZ     AA
    JMP     AA1
DELAY PROC NEAR          ;延时子程序
    PUSH    BX
    PUSH    CX
    MOV     BX,500
LP1:  MOV     CX,469
LP2:  LOOP   LP2
    DEC     BX
    JNZ     LP1
    POP     CX
    POP     BX
    RET
DELAY ENDP
.DATA
END

```

4.8.4 并串转换接口

利用 74LS165 芯片检测按键状态,由可编程串行接口芯片 8251A 接收后送给外部并行



接口 74LS273 显示输出，即由 LED 管反映对应的按键状态。其电路原理如图 4-38 所示。

8251A 接收方式为 1 位起始位、8 位数据位、1 位终止位、无奇偶校验位。74LS165 先发送高位 D7，在时钟的上升沿发送数据位，8251A 接收则是低位在前，在时钟的下降沿接收数据位。图 4-38 中 74LS165 芯片 U4 的 D7 引脚一直为高电平，模拟空闲位，D6 引脚一直为低电平，模拟起始位；芯片 U9 的 D5 引脚模拟停止位，D4 ~ D0 引脚模拟空闲位，均为高电平。273 (2) 输出接口控制 74LS165 芯片并行数据的锁存（低电平）及串行数据的输出（高电平）。

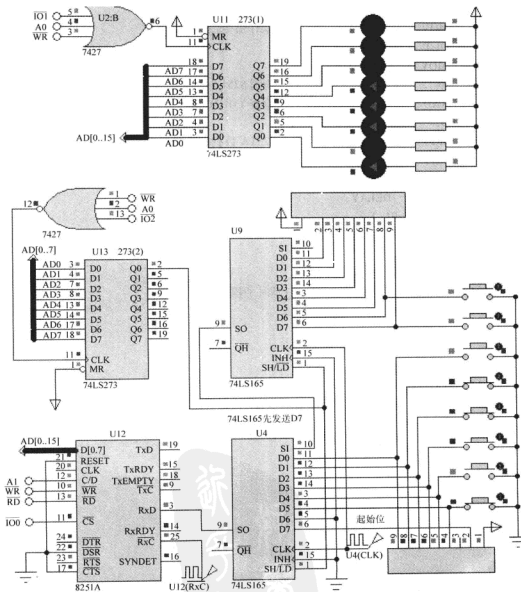


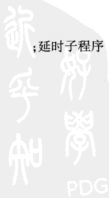
图 4-38 并串转换接口的电路原理

参考程序:

```

.MODEL SMALL
.8086
.STACK
.CODE
.STARTUP
AGAIN: MOV DX,0002H      ;8251A 控制口地址送到 DX
      MOV AL,01001110B   ;异步 1 位停止位、无校验位、数据位长度 8 位、16 分频
      OUT DX,AL
      MOV AL,17H         ;EH IR RTS ER SBRK Rx E DTR TxEN 作为命令字
      OUT DX,AL
      MOV DX,0400H       ;273(2) 地址
      MOV AL,01H         ;使能 165 移位
      OUT DX,AL
NEXT:  MOV DX,0002H       ;设置 8251A 状态口地址
WAIT1: IN AL,DX           ;读取状态字 144
      AND AL,02H         ;检测 RxRDY 位
      JZ WAIT1           ;接收缓冲器空,等待
      MOV DX,0000H       ;设置 8251A 数据口地址
      IN AL,DX           ;读数据
      PUSH AX            ;暂存接收到的数据
      MOV DX,0400H       ;273(2) 地址
      MOV AL,00H         ;禁止 165 移位并入串出
      OUT DX,AL
      POP AX             ;恢复接收到的数据
      MOV DX,0200H       ;273(1) 地址
      OUT DX,AL          ;将串行口接收到的数据由 273(1) 显示
      CALL DELAY2        ;延时
      MOV DX,0400H       ;273(2) 地址
      MOV AL,01H         ;使能 165 移位
      OUT DX,AL
      JMP NEXT
DELAY2 PROC NEAR        ;延时子程序
      PUSH BX
      PUSH CX
      MOV BX,1
LP1:   MOV CX,200
LP2:   LOOP LP2
      DEC BX

```



```

JNZ LP1
POP CX
POP BX
RET
DELAY2   ENDP
        .DATA
END

```

4.9 8253A 可编程定时/计数器

4.9.1 用虚拟示波器观察 8253A 的输出波形

用虚拟示波器观察 8253A 的输出波形,如图 4-39 所示,设置 8253A 的计数器 0 为 16 位计数、方式 3 (方波发生器)、十进制计数,生成 100Hz 的方波;8253A 的计数器 1 为 16 位计数、方式 2 (分频器)、十进制计数,分频系数取 100。

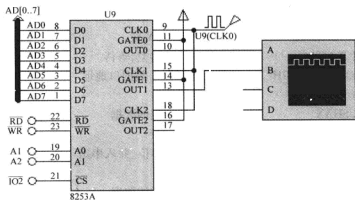


图 4-39 8253A 可编程定时/计数器

设输入 8253A 的时钟频率为 100kHz,要产生 100Hz (周期为 10ms) 的方波,则应分频 1000 倍,所以定时/计数器 0 的初值为 1000;定时/计数器 1 的分频系数为 100,所以定时/计数器 1 的初值为 100;其输出频率为 1kHz,周期为 1ms。图 4-40 所示是虚拟示波器显示的仿真波形,图 4-40 中每格周期为 1ms,该示波器的仿真图形也验证了上述分析计算的正确性。

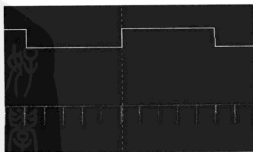


图 4-40 虚拟示波器显示的仿真波形

1) 参考程序。

```
.MODEL      SMALL
.8086
.STACK
.CODE
.STARTUP
MOV AL,00110111B ;T0 16 位 MODE3 BCD
MOV DX,0406H
OUT DX,AL
MOV DX,0400H
MOV AL,00H      ;低字节
OUT DX,AL
MOV AL,10H      ;高字节
OUT DX,AL
MOV AL,01110101B ;T1 16 位 MODE2 BCD
MOV DX,0406H
OUT DX,AL
MOV DX,0402H
MOV AL,00H      ;低字节
OUT DX,AL
MOV AL,01H      ;高字节
OUT DX,AL
JMP $
.DATA
END
```

2) 8253A 调试窗口。暂停程序执行，右键单击 8253A 仿真芯片，在弹出的快捷菜单中，勾选最下面的选项“8253A”，单步执行观察 8253A 内部寄存器数据变化。

4.9.2 用 8253A 定时/计数器控制 8 位 LED 循环移动

用 8253A 可编程定时/计数器作定时器，每隔 1s LED 向左循环移动 1 位。其电路原理如图 4-41 所示。设 8253A 可编程定时/计数器的输入时钟为 100kHz，设定时/计数器 0 的计数初值为 100，工作在方式 3，即方波发生器，其输出的 1kHz 方波做定时/计数器 2 的时钟。定时/计数器 2 的初值设为 1000，工作在方式 0，即每隔 1s 计数结束产生中断，用此信号作为不可屏蔽中断的申请信号，在中断服务程序中设置每中断一次 LED 向左循环移动 1 位。

参考程序：

```
.MODEL      SMALL
.8086
.STACK
.CODE
```



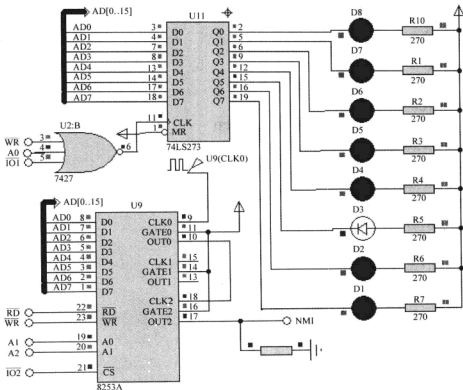


图 4-41 用 8253A 定时/计数器控制 8 位 LED 循环移动的电路原理

. STARTUP

NMI_INIT: PUSH ES

;NMI 中断向量表初始化

XOR AX,AX

MOV ES,AX

MOV AL,02H

XOR AH,AH

SHL AX,1

SHL AX,1

MOV SI,AX

MOV AX,OFFSET NMI_SERVICE

MOV ES:[SI],AX

INC SI

INC SI

MOV BX,CS

MOV ES:[SI],BX

POP ES

;可编程定时/计数器 8253A 初始化

```

MOV AL,00110111B           ;T0 16 位 MODE3 BCD
MOV DX,0406H                ;控制口地址
OUT DX,AL
MOV DX,0400H                ;T0 地址
MOV AX,0100H                ;100kHz 100 分频 1kHz
OUT DX,AL
MOV AL,AH                   ;高字节
OUT DX,AL
MOV AL,10110001B           ;T2 16 位 MODE0 BCD
MOV DX,0406H
OUT DX,AL
MOV DX,0404H
MOV AX,1000H                ;1kHz 1000 分频 1Hz
OUT DX,AL
MOV AL,AH                   ;高字节
OUT DX,AL
MOV BL,0FEH                 ;LED 初始状态
MOV DX,0200H                ;273 地址
MOV AL,BL
OUT DX,AL
JMP $

NMI_SERVICE: ROL BL,1        ;不可屏蔽中断服务程序
MOV AL,BL
MOV DX,0200H                ;273 地址
OUT DX,AL
MOV DX,0404H
MOV AX,1000H                ;1kHz 1000 分频 1Hz
OUT DX,AL                   ;重输入初值并启动定时器
MOV AL,AH                   ;高字节
OUT DX,AL

EXIT: IRET

. DATA
END

```

4.9.3 用 8253A 定时/计数器控制 1 位 LED 数码管数字递增

利用 8253A 可编程定时/计数器定时 1s, 使 LED 数码管每隔 1s 加 1, 到 9 后回 0 重新开始, 其电路原理如图 4-42 所示。设输入 8253A 的时钟频率为 100kHz, 使用 8253A 的定时/计数器 2 为 16 位计数、方式 2 (分频器)、十进制计数。则最长定时时间为 $(1 / (100 \times$

1000)) $\times 10000\text{s} = 0.1\text{s}$ 。不能实现 1s 定时。若取分频系数为 5000, 则可定时 0.05s。在中断服务程序中设置一计数器, 统计 20 次则可定时 1s。

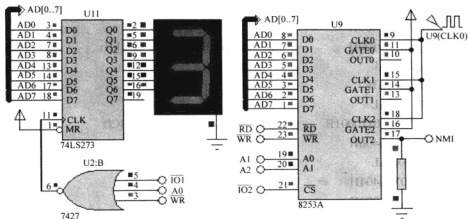


图 4-42 用 8253A 定时/计数器控制 1 位 LED 数码管数字递增的电路原理

参考程序:

```

.MODEL          SMALL
.8086
.STACK
.CODE
.STARTUP
NMI_INIT: PUSH ES          ;NMI 中断向量初始化
            XOR AX,AX
            MOV ES,AX
            MOV AL,02H
            XOR AH,AH
            SHL AX,1
            SHL AX,1
            MOV SI,AX
            MOV AX,OFFSET NMI_SERVICE
            MOV ES:[SI],AX
            INC SI
            INC SI
            MOV BX,CS
            MOV ES:[SI],BX
            POP ES
            MOV AL,10110101B ;T2 16 位 MODE2 BCD
            MOV DX,0406H
            OUT DX,AL

```

```

MOV DX,0404H
MOV AL,00H                                ;低字节 100kHz 5000 分频 20Hz
OUT DX,AL
MOV AL,50H                                ;高字节
OUT DX,AL
MOV SI,OFFSET SITUATION
JMP $
NMI_SERVICE:INC COUNT
MOV AL,COUNT
CMP AL,20                                  ;统计 20 次 1s
JNZ EXIT
MOV COUNT,0
MOV DX,0200H                              ;273 地址
MOV AL,[SI]
OUT DX,AL
ADD SI,1                                  ;NEXT SITUATION
CMP SI,OFFSET SIT_END
JB EXIT
MOV SI,OFFSET SITUATION
EXIT:IRET
. DATA
COUNT DB0                                ;统计 20 次 1s
SITUATION DB 3FH,06H,5BH,4FH,66H,6DH,7DH,07H,7FH,6FH;共阴极
SIT_END = $
END

```

4.9.4 用 8253A 定时/计数器设计跑表

用 8253A 可编程定时/计数器作定时器设计跑表。当第一次按下按键时，跑表开始计时；当第二次按下按键时，跑表停止计时；第三次按下按键时，跑表回 0。其电路原理如图 4-43 所示。设 8253A 可编程定时/计数器的输入时钟为 100kHz，定时/计数器 2 的初值设为 1000，工作在方式 2，分频输出为 100Hz 的时钟信号，时钟周期为 0.1s，用此信号作为不可屏蔽中断的定时信号。

参考程序：

```

. MODEL      SMALL
. 8086
. STACK
. CODE
. STARTUP

```



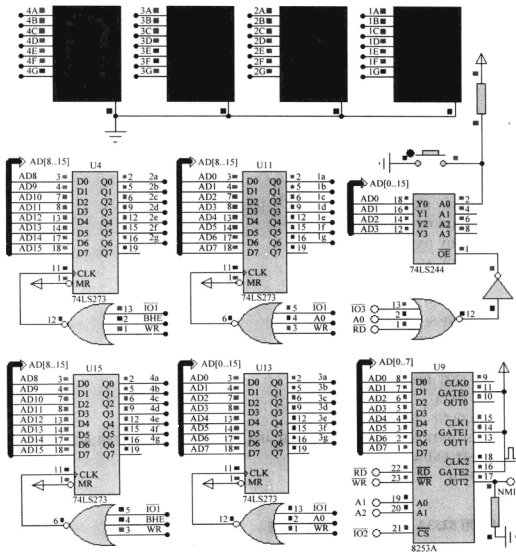


图 4-43 用 8253A 定时/计数器设计跑表的电路原理

```

NMI_INIT: PUSH ES          ;NMI 中断向量初始化
          XOR AX,AX
          MOV ES,AX
          MOV AL,02H
          XOR AH,AH
          SHL AX,1
          SHL AX,1
          MOV SI,AX
          MOV AX,OFFSET NMI_SERVICE

```



```

MOV ES:[SI],AX
INC SI
INC SI
MOV BX,CS
MOV ES:[SI],BX
POP ES
MOV AL,10110101B           ;T2 16 位 MODE2 BCD
MOV DX,0406H
OUT DX,AL
MOV DX,0404H
MOV AX,1000H               ;clk = 100kHz 1000 分频 out = 100Hz = 0.01s
OUT DX,AL
MOV AL,AH                  ;高字节
OUT DX,AL
LOP:CALL KEY
CALL DISP
JMP LOP

DISP PROC                  ;显示子程序
MOV BL,M_SEC
AND BX,000FH               ;毫秒个位
MOV SI,BX
MOV AL,SITUATION[SI]       ;毫秒个位段码
MOV BL,M_SEC
AND BX,00F0H               ;毫秒十位
MOV CL,4
SHR BX,CL
MOV SI,BX
MOV AH,SITUATION[SI]       ;毫秒十位段码
MOV DX,0000H               ;毫秒输出地址
OUT DX,AX

MOV BL,SEC
AND BX,000FH
MOV SI,BX
MOV AL,SITUATION[SI]
MOV BL,SEC
AND BX,00F0H
MOV CL,4
SHR BX,CL

```

```

MOV SI, BX
MOV AH, SITUATION[SI]
MOV DX, 0200H           ;秒输出地址
OUT DX, AX
RET
DISP ENDP

KEY PROC NEAR
MOV DX, 0600H
IN AL, DX
TEST AL, 01H
JNZ EXITKEY
CALL DELAY              ;消抖
MOV DX, 0600H
IN AL, DX
TEST AL, 01H
JNZ EXITKEY
INC STATE
WAITKEY: IN AL, DX
TEST AL, 01H
JZ WAITKEY              ;等待按键释放
CMP STATE, 03H
JB EXITKEY
MOV STATE, 0
EXITKEY: RET
KEY ENDP

 $f_{osc} = 5\text{MHz}, T = 0.2\mu\text{s}, T_d \approx 20\text{ms}$ 
DELAY PROC NEAR        ;延时子程序
PUSH BX
PUSH CX
MOV BX, 1              ;4
DEL1: MOV CX, 5882      ;4
DEL2: LOOP DEL2         ;17/5
DEC BX                 ;2
JNZ DEL1               ;16/4
POP CX
POP BX
RET
DELAY ENDP

```



```

NMI_SERVICE:
    PUSH AX
    CMP STATE,0H           ;STATE = 0,时钟清 0
    JE  CLEAR
    CMP STATE,01H         ;STATE = 1,时钟走时
    JE  START
    CMP STATE,02H         ;STATE = 2,时钟停止
    JE  STOP
    JMP EXIT
CLEAR:MOV M_SEC,0
    MOV SEC,0
    JMP EXIT
START:MOV AL,M_SEC
    ADD AL,1              ;毫秒加 1
    DAA
    MOV M_SEC,AL
    CMP M_SEC,00H
    JNE EXIT
    MOV M_SEC,0
    MOV AL,SEC
    ADD AL,1              ;秒加 1
    DAA
    MOV SEC,AL
    CMP SEC,60H
    JB  EXIT
    MOV SEC,0
    JMP EXIT
STOP:
EXIT:POP AX
    IRET
. DATA
STATE      DB 0;0_CLEAR,1_START,2_STOP
M_SEC      DB 00H
SEC        DB 00H
SITUATION  DB 3FH,06H,5BH,4FH,66H,6DH,7DH,07H,7FH,6FH;共阴极
SIT_END = $
END

```

4.9.5 用 8253A 定时/计数器设计可调时电子钟

用 8253A 可编程定时/计数器设计可调时电子钟，设有两个按键：一个用于调时，一个

用于调分。其电路原理如图 4-44 所示。8253A 可编程定时/计数器的输入时钟为 100kHz，设定定时/计数器 0 的计数初值为 100，工作在方式 3，即方波发生器，其输出的 1kHz 方波作定时/计数器 2 的时钟。定时/计数器 2 的初值设为 1000，工作在方式 2，即每隔 1s 输出负脉冲，取反后用做不可屏蔽中断的中断申请信号。

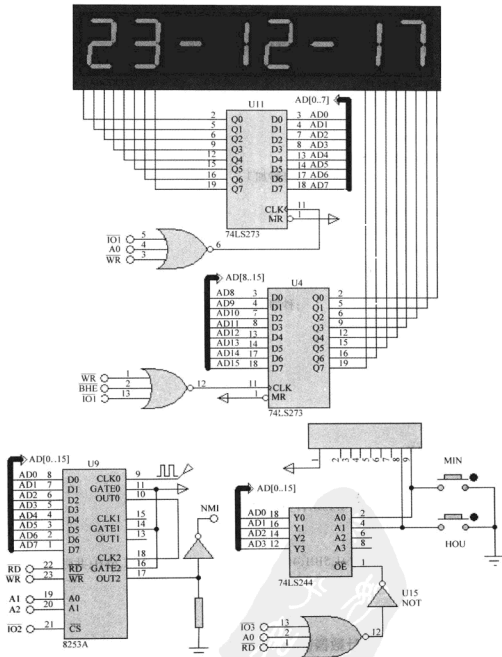


图 4-44 用 8253A 定时/计数器设计可调电子钟的电路原理

参考程序:

```

.MODEL      SMALL
.8086
.STACK
.CODE
.STARTUP
NMI_INIT: PUSH ES          ;NMI 不可屏蔽中断向量表初始化
        XOR AX,AX
        MOV ES,AX
        MOV AL,02H
        XOR AH,AH
        SHL AX,1
        SHL AX,1
        MOV SI,AX
        MOV AX,OFFSET NMI_SERVICE
        MOV ES:[SI],AX
        INC SI
        INC SI
        MOV BX,CS
        MOV ES:[SI],BX
        POP ES
        ;定时/计数器初始化
        MOV AL,00110111B          ;T0 16 位 MODE3 BCD
        MOV DX,0406H              ;控制口地址
        OUT DX,AL
        MOV DX,0400H              ;T0 地址
        MOV AX,0100H              ;100kHz 100 分频 1kHz
        OUT DX,AL
        MOV AL,AH                  ;高字节
        OUT DX,AL
        MOV AL,10110101B          ;T2 16 位 MODE2 BCD
        MOV DX,0406H
        OUT DX,AL
        MOV DX,0404H
        MOV AX,1000H              ;1kHz 1000 分频 1Hz
        OUT DX,AL
        MOV AL,AH                  ;高字节
        OUT DX,AL
        LOOP0:
        CALL KEY

```

```

CALL DISP
JMP LOOP0

NMI_SERVICE:                                ;中断服务程序
    PUSH AX
    MOV AL,SEC
    ADD AL,1
    DAA
    MOV SEC,AL
    CMP SEC,60H
    JB  EXIT
    MOV SEC,0
    MOV AL,MIN
    ADD AL,1
    DAA
    MOV MIN,AL
    CMP MIN,60H
    JB  EXIT
    MOV MIN,0
    MOV AL,HOU
    ADD AL,1
    DAA
    MOV HOU,AL
    CMP HOU,24H
    JB  EXIT
    MOV HOU,0
EXIT:POP AX
    IRET
DISP  PROC  NEAR
    MOV AL,0FFH                                ;不显示
    MOV DX,0201H
    OUT DX,AL
    MOV BL,SEC
    AND BX,000FH
    MOV SI,BX
    MOV AL,SITUATION[SI]                        ;段码
    MOV DX,0200H
    OUT DX,AL
    MOV AL,0FEH                                ;秒个位
    MOV DX,0201H

```



```
OUT DX,AL
CALL DELAY
MOV AL,0FFH           ;不显示
MOV DX,0201H
OUT DX,AL
MOV BL,SEC
AND BX,00F0H
MOV CL,4
SHR BX,CL
MOV SI,BX
MOV AL,SITUATION[SI]  ;段码
MOV DX,0200H
OUT DX,AL
MOV AL,0FDH           ;秒十位
MOV DX,0201H
OUT DX,AL
CALL DELAY
MOV AL,0FFH           ;不显示
MOV DX,0201H
OUT DX,AL
MOV AL,40H            ;段码
MOV DX,0200H
OUT DX,AL
MOV AL,0FBH           ;秒个位
MOV DX,0201H
OUT DX,AL
CALL DELAY
MOV AL,0FFH           ;不显示
MOV DX,0201H
OUT DX,AL
MOV BL,MIN
AND BX,000FH
MOV SI,BX
MOV AL,SITUATION[SI]  ;段码
MOV DX,0200H
OUT DX,AL
MOV AL,0F7H           ;分个位
MOV DX,0201H
OUT DX,AL
CALL DELAY
```

```
MOV AL,0FFH           ;不显示
MOV DX,0201H
OUT DX,AL
MOV BL,MIN
AND BX,00F0H
MOV CL,4
SHR BX,CL
MOV SI,BX
MOV AL,SITUATION[SI]   ;段码
MOV DX,0200H
OUT DX,AL
MOV AL,0EFH           ;分十位
MOV DX,0201H
OUT DX,AL
CALL DELAY
MOV AL,0FFH           ;不显示
MOV DX,0201H
OUT DX,AL
MOV AL,40H             ;段码
MOV DX,0200H
OUT DX,AL
MOV AL,0DFH           ;秒个位
MOV DX,0201H
OUT DX,AL
CALL DELAY
MOV AL,0FFH           ;不显示
MOV DX,0201H
OUT DX,AL
MOV BL,HOU
AND BX,000FH
MOV SI,BX
MOV AL,SITUATION[SI]   ;段码
MOV DX,0200H
OUT DX,AL
MOV AL,0BFH           ;时个位
MOV DX,0201H
OUT DX,AL
CALL DELAY
MOV AL,0FFH           ;不显示
MOV DX,0201H
```




```

OUT DX,AL
MOV BL,HOU
AND BX,00F0H
MOV CL,4
SHR BX,CL
MOV SI,BX
MOV AL,SITUATION[SI]      ;段码
MOV DX,0200H
OUT DX,AL
MOV AL,07FH                ;时十位
MOV DX,0201H
OUT DX,AL
CALL DELAY
RET
DISP      ENDP

KEY       PROC NEAR
MOV DX,0600H
IN  AL,DX
TEST AL,01H
JNZ  NEXTHOU
CALL DISP      ;消抖
CALL DISP
CALL DISP
MOV DX,0600H
IN  AL,DX
TEST AL,01H
JNZ  NEXTHOU
MOV AL,MIN
ADD AL,1        ;分调整
DAA
MOV MIN,AL
CMP MIN,60H
JB  NEXTHOU
MOV MIN,0
NEXTHOU:MOV DX,0600H
IN  AL,DX
TEST AL,02H
JNZ  EXITKEY
CALL DISP      ;消抖

```



```

CALL DISP
CALL DISP
MOV DX,0600H
IN AL,DX
TEST AL,02H
JNZ EXITKEY
MOV AL,HOU
ADD AL,1
DAA ;时调整
MOV HOU,AL
CMP HOU,24H
JB NEXTHOU
MOV HOU,0
EXITKEY:RET
KEY ENDP

DELAY PROC NEAR ;定时子程序
PUSH BX
PUSH CX
MOV BX,1
LP1: MOV CX,469
LP2: LOOP LP2
DEC BX
JNZ LP1
POP CX
POP BX
RET
DELAY ENDP

. DATA
SEC DB 00H
MIN DB 00H
HOU DB 23H
SITUATION DB 3FH,06H,5BH,4FH,66H,6DH,7DH,07H,7FH,6FH,40H;共阴极
SIT_END = $
END

```

4.9.6 用 8253A 定时/计数器作方波发生器

设 8253A 定时/计数器的地址为 04A0H、04A2H、04A4H、04A6H，CLK0 的输入时钟为

750kHz, 利用 8253A 的定时/计数器 0, 16 位二进制计数方式, 输出一个周期为 0.05s (20Hz) 的连续方波 (方式 3)。定时/计数器 0 的输出 OUT0 作为 CLK1 和 CLK2 时钟, OUT1 和 OUT2 分别连接 LED, 使连接 OUT1 的 LED 以 2s 的周期闪烁 (即每隔 1s 取反), 定时初值为 40 分频, 使连接 OUT2 的 LED 以 4s 的周期闪烁 (即每隔 2s 取反), 定时初值为 80 分频。设置 8253A 的定时/计数器 1 以 16 位二进制计数, 方式 3; 定时/计数器 2 以 8 位 BCD 码计数, 方式 3。其电路原理如图 4-45 所示。

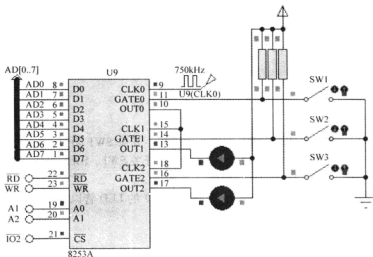


图 4-45 用 8253A 定时/计数器作方波发生器的电路原理

1) 参考程序。

```
.MODEL SMALL
```

```
.8086
```

```
.STACK
```

```
.CODE
```

```
.STARTUP
```

```
MOV    DX, 0406H    ; 控制寄存器
MOV    AL, 36H      ; 计数器 0, 方式 3
OUT    DX, AL
MOV    DX, 0400H
MOV    AL, 7CH
OUT    DX, AL
MOV    AL, 92H
OUT    DX, AL      ; 计数值 927CH
MOV    DX, 0406H
MOV    AL, 76H      ; 计数器 1, 16 位二进制计数, 方式 3
OUT    DX, AL
MOV    DX, 0402H
MOV    AL, 28H
```

```

OUT    DX, AL
MOV    AL, 0          ; 计数值 0028H = 40;  $0.05 \times 40 = 2s$ 
OUT    DX, AL
MOV    DX, 0406H
MOV    AL, 97H        ; 计数器 2, 仅读写低 8 位, 方式 3, BCD 码计数
OUT    DX, AL
MOV    DX, 0404H
MOV    AL, 80H        ; 计数值 (80) BCD;  $0.05 \times 80 = 4s$ 
OUT    DX, AL

```

```
JMP $
```

```
. DATA
```

```
END
```

2) 练习。

① 开关 SW1、SW2、SW3 都断开有何实验现象; 开关 SW1 闭合, SW2、SW3 断开有何现象; 开关 SW1、SW3 断开, SW2 闭合有何现象; 开关 SW1、SW2 断开, SW3 闭合有何现象; 并说明为什么?

② 调整定时/计数器 2 的定时常量, 使连接 OUT2 的 LED 管以 1s 的周期闪烁。

4.10 ADC0809 模/数转换器

4.10.1 查询方式读取 ADC0809 模/数转换结果

用 ADC0809 模/数转换器连续采集 1 路模拟信号, 用 2 位 LED 数码管显示转换结果, 用 74LS244 作 EOC 信号的输入接口, 根据 EOC 信号状态判断转换是否结束。其电路原理如图 4-46 所示。

参考程序:

```

.MODEL    SMALL
.8086
.STACK
.CODE
.STARTUP
AGAIN:    MOV DX, 0400H          ; 0809 地址
OUT DX, AL
MOV DX, 0600H                  ; 244 地址
WAIT0:    IN  AL, DX
          TEST AL, 01H
          JZ  WAIT0
          MOV DX, 0400H          ; 0809 地址
          IN  AL, DX             ; 读取转换结果
          MOV AH, AL

```

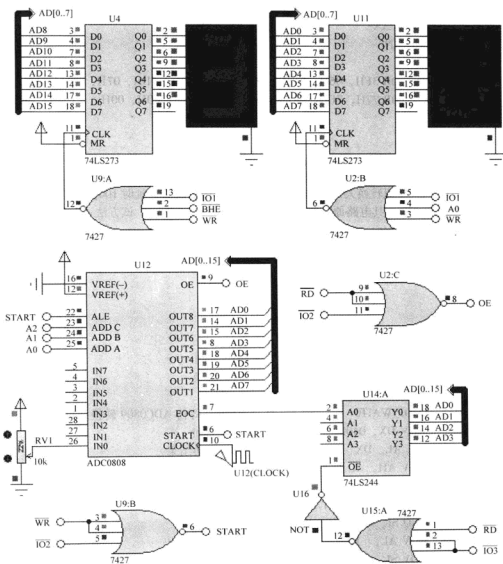


图 4-46 查询方式读取 ADC0809 模/数转换结果的电路原理

MOV SI, AX

MOV BX, OFFSET SITUATION

AND SL. 000FH

MOV AL, [BX] [SI] ; 低字节

MOV SI, AX

MOV CL, 12

SHR SL CL

MOV AH, [BX] [SI] ; 高字节

```

MOV DX, 0200H          ; 273 地址
OUT DX, AX             ; 显示结果
JMP AGAIN

. DATA
SITUATION DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 6FH
           DB 77H, 7CH, 58H, 5EH, 79H, 71H, 40H, 00H      ; 共阴极
END

```

4.10.2 延时方式读取 ADC0809 模/数转换结果

ADC0809 一次模拟转换大约用时 $100\mu\text{s}$ ，每次启动转换后，延时 $100\mu\text{s}$ 等待模拟转换结束，再读取转换结果。其电路原理同第 4.10.1 节如图 4-46 所示，该方法可节省 74LS244 输入接口。

参考程序：

```

. MODEL SMALL
. 8086
. STACK
. CODE
. STARTUP
AGAIN:  MOV DX, 0400H          ; 0809 地址
        OUT DX, AL           ; 启动转换
        MOV CX, 0080H
WAIT0:  LOOP WAIT0            ; 延时，等待 ADC0809 转换结束
        MOV DX, 0400H        ; 0809 地址
        IN AL, DX            ; 读取转换结果
        MOV AH, AL
        MOV SI, AX
        MOV BX, OFFSET SITUATION
        AND SI, 000FH
        MOV AL, [BX][SI]     ; 低字节段码
        MOV SI, AX
        MOV CL, 12
        SHR SI, CL
        MOV AH, [BX][SI]    ; 高字节段码
        MOV DX, 0200H        ; 273 地址
        OUT DX, AX           ; 显示结果
        JMP AGAIN

. DATA
SITUATION DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH
           DB 6FH, 77H, 7CH, 58H, 5EH, 79H, 71H, 40H, 00H      ; 共阴极
END

```

4.10.3 中断方式读取 ADC0809 模/数转换结果

ADC0809 每次 A/D 转换结束, EOC 引脚由低电平变为高电平, 将此信号作为 A/D 转换结束的中断申请信号, 在中断服务程序中读取 A/D 转换结果, 并启动下一次的 A/D 转换。电路原理基本上同第 4.10.1 节如图 4-46 所示, ADC0809 模/数转换器 EOC 引脚连线到 8086CPU 的 NMI 中断入口引脚, 其余不变。

参考程序:

```
.MODEL SMALL
.8086
.STACK
.CODE
.STARTUP
NMI_ INIT: PUSH ES                ; NMI 中断向量初始化
          XOR AX, AX
          MOV ES, AX
          MOV AL, 02H
          XOR AH, AH
          SHL AX, 1
          SHL AX, 1
          MOV SI, AX
          MOV AX, OFFSET NMI_SERVICE
          MOV ES: [SI], AX
          INC SI
          INC SI
          MOV BX, CS
          MOV ES: [SI], BX
          POP ES
          MOV DX, 0400H            ; 0809 地址
          OUT DX, AL               ; 启动转换
          JMP $

NMI_ SERVICE:
          MOV DX, 0400H            ; 0809 地址
          IN AL, DX                ; 读取转换结果
          MOV AH, AL
          MOV SI, AX
          MOV BX, OFFSET SITUATION
          AND SI, 000FH
          MOV AL, [BX] [SI]        ; 低字节段码
          MOV SI, AX
          MOV CL, 12
```

```

SHR SI, CL
MOV AH, [BX] [SI]          ; 高字节段码
MOV DX, 0200H              ; 273 地址
OUT DX, AX                 ; 显示结果
MOV DX, 0400H              ; 0809 地址
OUT DX, AL                 ; 启动转换

EXIT: IRET
. DATA
SITUATION DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 6FH
          DB 77H, 7CH, 58H, 5EH, 79H, 71H, 40H, 00H          ; 共阴极
END

```

4.10.4 中断方式读取 ADC0809 多路模/数转换结果

用 ADC0809 模/数转换器连续采集 8 路模拟信号，用 1 位 LED 数码管显示当前通道号，用 2 位 LED 数码管显示当前通道的转换结果，转换结束信号 EOC 用作 NMI 不可屏蔽中断的中断申请信号，在通道服务程序中读取前一个通道的 A/D 转换结果并启动下一个通道开始 A/D 转换。其电路原理如图 4-47 所示。

参考程序：

```

. MODEL SMALL
. 8086
. STACK
. CODE
. STARTUP
NMI_ INIT: PUSH ES
          XOR AX, AX          ; NMI 不可屏蔽中断变量表初始化
          MOV ES, AX
          MOV AL, 02H
          XOR AH, AH
          SHL AX, 1
          SHL AX, 1
          MOV SI, AX
          MOV AX, OFFSET NMI_SERVICE
          MOV ES: [SI], AX
          INC SI
          INC SI
          MOV BX, CS
          MOV ES: [SI], BX
          POP ES
          MOV TDH, 0          ; A/D 通道号

```

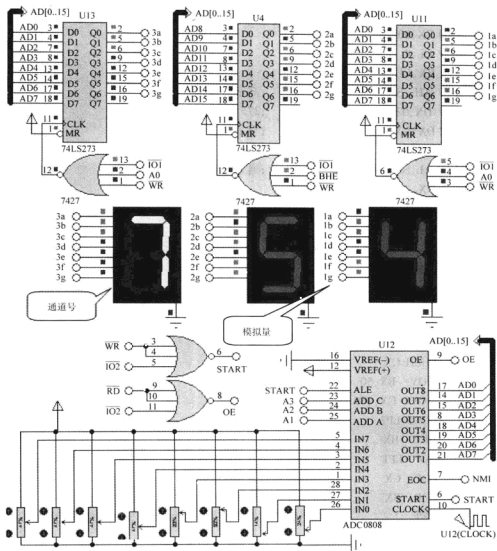



图 4-47 中断方式读取 ADC0809 多路模拟/数转换结果的电路原理

MOV ADC ADRESS, 0400H : A/D 通道 0 地址

MOV DX, ADC_ADDRESS

OUT DX, AL ; 启动转换

；8 路转换分时显示，第 1 位 LED 数码管显示通道号

：第2、3位LED数码管以十六进制显示转换结果

DISP0: MOV SI, 0 ; SI 通道号

```
DISP1:  MOV BL, ADC_RESULT[SI]
```

AND BX, 000FH

MOV AL, SITUATION[BX] ; 低字节段码

```

MOV BL,ADC_RESULT[SI]
AND BX,00F0H
MOV CL,4
SHR BX,CL
MOV AH,SITUATION[BX]      ;高字节段码
MOV DX,0200H              ;273 地址
OUT DX,AX                 ;显示结果
MOV DX,0
MOV AL,SITUATION[SI]      ;通道号段码
OUT DX,AL
CALL DELAY
INC SI
CMP SI,8
JNZ DISP1
JMP DISPO

```

DELAY PROC NEAR ;延时子程序

```

        PUSH BX
        PUSH CX
        MOV  BX,50
LP1:    MOV  CX,469
LP2:    LOOP LP2
        DEC  BX
        JNZ  LP1
        POP  CX
        POP  BX
        RET

```

DELAY ENDP

;中断服务程序

```

NMI_SERVICE:MOV DX,ADC_ADDRESS      ;当前 A/D 通道
            IN  AL,DX                ;读取转换结果
            PUSH SI
            MOV SI,TDH
            MOV ADC_RESULT[SI],AL    ;保存 A/D 转换结果
            POP SI
            ADD ADC_ADDRESS,2        ;下一通道地址
            INC TDH                  ;下一通道
            CMP TDH,8                ;8 个通道都扫描完否
            JNZ NEXT_AD              ;未描述完则启动下一通道

```

```

MOV TDH,0 ;从 0 通道开始重新采样
MOV ADC_ADRESS,0400H ;0 通道地址
NEXT_AD: MOV DX,ADC_ADRESS
OUT DX,AL ;启动转换
EXIT: IRET
.DATA
TDH DW 0
ADC_ADRESS DW 0400H
ADC_RESULT DB 8 DUP(0)
SITUATION DB 3FH,06H,5BH,4FH,66H,6DH,7DH,07H,7FH,6FH
DB 77H,7CH,58H,5EH,79H,71H,40H,00H ;共阴极
END

```

4.10.5 数据线方式选择 ADC0809 模/数转换通道

若 ADC0809 模/数转换器的通道选择端 ADDA、ADDB、ADDC 直接连接到 CPU 的地址总线 A0、A1、A2 低 3 位地址线，该方法占用的 I/O 地址较多，每一个模拟输入端对应一个口地址，8 个模拟输入端占用 8 个口地址。图 4-48 中，ADC0809 的通道选择输入端 ADDA、ADDB、ADDC 分别连接在了数据总线 D0、D1、D2 低 3 位，通过数据线输出一个控制字作为模拟通道的选择信号。

参考程序：

```

.MODEL SMALL
.8086
.STACK
.CODE
.STARTUP
MOV TD,0 ;通道号
AGAIN: MOV DX,0400H ;0809 芯片地址
MOV AL,TD
OUT DX,AL ;启动转换
MOV CX,0080H
WAIT0: LOOP WAIT0 ;延时等待 A/D 转换结束
MOV DX,0400H ;0809 地址
IN AL,DX ;读取转换结果
MOV AH,AL ;暂存转换结果
MOV SI,AX
MOV BX,OFFSET SITUATION
AND SI,000FH
MOV AL,[BX][SI] ;低字节段码

```

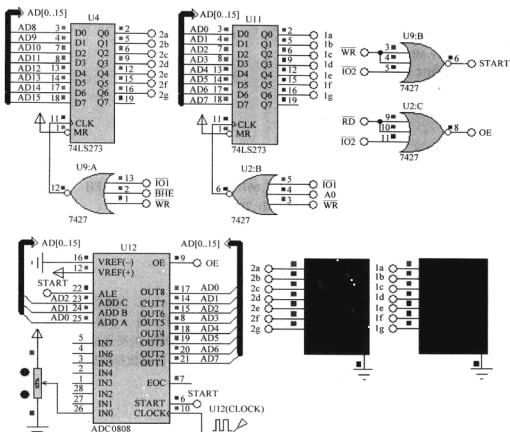


图 4-48 数据线方式选择 ADC0809 模/数转换通道的电路原理

```

MOV SI,AX
MOV CL,12
SHR SI,CL
MOV AH,[BX][SI] ;高字节段码
MOV DX,0200H ;273 地址
OUT DX,AX ;显示结果
JMP AGAIN

```

```

.DATA

```

```

TD DB 0

```

```

SITUATION DB 3FH,06H,5BH,4FH,66H,6DH,7DH,07H,7FH,6FH

```

```

DB 77H,7CH,58H,5EH,79H,71H,40H,00H ;共阴极

```

```

END

```

4.10.6 ADC0809 多路模拟量采集

用 ADC0809 模/数据转换器采集 8 路模拟量，并将每一路的数字量用 LCD 输出，LCD

的第1行显示通道号,在第2行相对位置显示该通道的数字量,其电路原理如图4-49所示。本例中所用LCD为字符型液晶模块,采用单线RS-232串行接口,利用8251A可编程串行接口发送待显示字符数据。

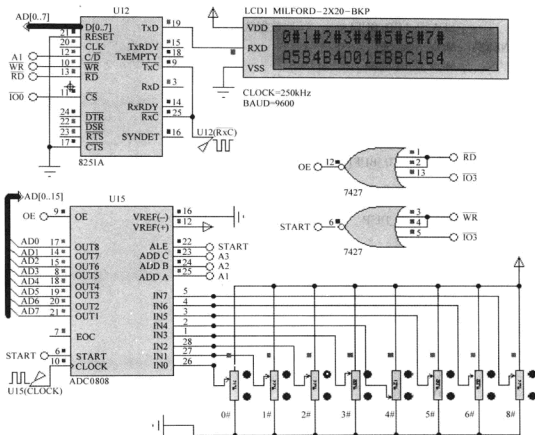


图 4-49 ADC0809 多路模拟量采集的电路模拟量

参考程序:

MODEL SMALL

8086

STACK

CODE

STARTUP

MOV DX,0002H

MOV AL,4DH

OUT DX_AL

MOV AL,07H

OUT DX,AL

:8251A 控制口地址送 DX

:1 位停止位、无校验、8 位数据、倍频 $\times 1$

:设置操作命令控制字

```

CALL DELAY          ;延时
MOV  CMD_BUF,80H    ;第 1 行
CALL WRCMD
CALL DELAY          ;延时
LEA  SI,LCD_BUF1    ;取数据区偏移地址
NEXT: MOV AL,[SI]
      MOV C_BUF,AL
      CALL PUTCTOLCD
      MOV CX,500
      LOOP $
      INC SI          ;修改数据区地址指针
      CMP SI,LCD_BUF1_END  是否到字符串尾
      JB  NEXT
LOP:  CALL ADC0809
      CALL LCD_DISP
      JMP LOP

```

```

LCD_DISP  PROC
      MOV CMD_BUF,0C0H
      CALL WRCMD
      CALL DELAY
      LEA SI,LCD_BUF2    ;取数据区偏移地址
NEXT1:  MOV AL,[SI]
      MOV C_BUF,AL
      CALL PUTCTOLCD
      MOV CX,500
      LOOP $
      INC SI          ;修改数据区地址指针
      CMP SI,LCD_BUF2_END  ;是否到字符串尾
      JB  NEXT1
      RET

```

```

LCD_DISP  ENDP

```

```

DELAY  PROC  NEAR          ;延时子程序
      PUSH BX
      PUSH CX
      MOV BX,200          ;4
DEL11:  MOV CX,120          ;4
DEL21:  LOOP DEL21        ;17/5

```

```

DEC BX                                ;2
JNZ DEL11                             ;16/4
POP CX
POP BX
RET
DELAY   ENDP

PUTCTOLCD PROC
    MOV DX,0002H                      ;设置 8251A 控制口地址
WAIT0:  IN AL,DX                      ;读取状态字
    AND AL,01H                        ;检测 TxRDY 位
    JZ WAIT0                          ;发送器未准备好,等待
    MOV DX,0000H                      ;设置 8251A 数据口地址
    MOV AL,C_BUF                      ;取数据
    OUT DX,AL                         ;将数据传送给另一台计算机
    MOV DX,0002H                      ;设置 8251A 控制口地址
WAIT1:  IN AL,DX                      ;读取状态字
    AND AL,04H                        ;检测 TxEMPTY 位
    JZ WAIT1                          ;发送器不空,等待
    RET
PUTCTOLCD ENDP

WRCMD   PROC                          ;发送 LCD 操作命令
    MOV C_BUF,0FEH
    CALL PUTCTOLCD
    MOV CX,500                        ;延时等待
    LOOP $
    MOV AL,CMD_BUF
    MOV C_BUF,AL
    CALL PUTCTOLCD
    RET
WRCMD   ENDP

ADC0809 PROC
    MOV TDH,0                         ;从 0 通道开始采样
    MOV ADC_ADRESS,0600H              ;0 通道地址
NEXT_AD: MOV DX,ADC_ADRESS
    OUT DX,AL                         ;启动转换

```

```

MOV CX,0080H
WAIT0: LOOP WAIT0           ;延时,等待 ADC0809 转换结束
MOV DX,ADC_ADRESS         ;当前 A/D 通道
IN AL,DX                   ;读取转换结果
MOV SI,TDH
MOV ADC_RESULT[SI],AL      ;保存 A/D 转换结果
MOV BH,0
MOV BL,AL
AND BX,00F0H
MOV CL,4
SHR BX,CL
MOV AH,ASC[BX]             ;高字节
SHL SI,1
MOV DI,SI
MOV LCD_BUF2[DI],AH
INC DI
MOV BL,AL
AND BX,000FH
MOV AL,ASC[BX]             ;低字节
MOV LCD_BUF2[DI],AL        ;存入显示缓冲区
ADD ADC_ADRESS,2           ;下一通道地址
INC TDH                     ;下一通道
CMP TDH,8                   ;8 个通道都扫描完否
JNZ NEXT_AD                ;没有启动下一通道
RET

ADC0809 ENDP

. DATA
C_BUF DB ?                 ;LCD 待发字符
CMD_BUF DB ?               ;LCD 待发命令字
TDH DW 0                    ;当前 0809 通道号
ADC_ADRESS DW ?            ;当前 0809 通道地址
ADC_RESULT DB 8 DUP (?)    ;ADC0809 转换结果
LCD_BUF1 DB '0#1#2#3#4#5#6#7#'
LCD_BUF1_END = $
LCD_BUF2 DB 16 DUP(?)
LCD_BUF2_END = $
ASC DB '01234567890ABCDEF'
END

```



4.11 DAC0832 数/模转换器

4.11.1 DAC0832 单缓冲方式输出三角波

采用单缓冲方式，通过 DAC0832 数/模转换器产生单极性三角波，并用虚拟示波器观察波形。

DAC0832 数/模转换器内部有二级锁存寄存器，根据要求采用单缓冲方式，即经一级输入寄存器锁存。假设用第一级锁存，第二级直通，那么第二级的控制端 WR_2 和 $XFER$ 应处于有效低电平状态，使第二级锁存器一直处于打开状态。第一级锁存具有锁存工能的条件是 ILE 、 CS 、 WR_1 都要满足有效电平。为减少控制线数，使 ILE 一直处于高电平有效状态，缓冲锁存信号使用 WR_1 和 CS 端。其电路原理如图 4-50 所示。

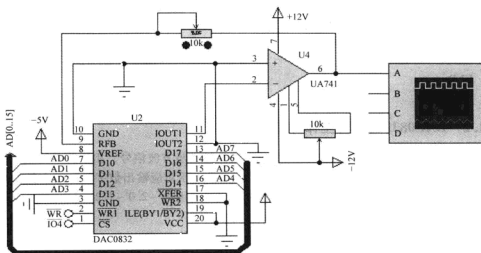


图 4-50 DAC0832 单缓冲方式输出三角波的电路原理

参考程序：

```
.MODEL SMALL
.8086
.STACK
.CODE
.STARTUP
AGAIN: MOV AL,0
      MOV DX,0800H      ;0832 地址
AA1:  OUT DX,AL
      CALL DELAY        ;延时
      INC AL
      CMP AL,0FFH      ;三角波是否到电压最高点
```

```

JNZ AA1
AA2: OUT DX,AL
CALL DELAY
DEC AL ;三角波是否到电压最低点
CMP AL,00H
JNZ AA2
JMP AA1
DELAY PROC NEAR ;延时程序,控制三角波周期
MOV BX,2
LP1: MOV CX,469
LP2: LOOP LP2
DEC BX
JNZ LP1
RET
DELAY ENDP
DATA
END

```

4.11.2 DAC0832 双缓冲工作方式

利用 DAC0832 数/模转换器产生两个不同极性的同步方波信号，相位关系如图 4-51 所示。图 4-51 所示，上面的波形为单极性方波，高低电压对应输出数字量的最大值为 0FFH 和最小值为 0，下面的波形为双极性波形，输出电压范围为 -2.0 ~ 1.2V，模拟电压所对应的数字量计算方法为：

$$D_x = (U_x - U_2) \frac{FFH}{U_1 - U_2}$$

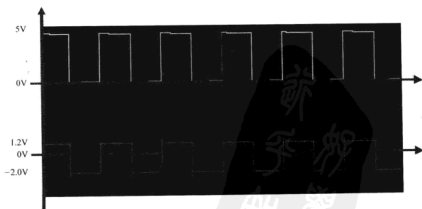


图 4-51 同步方波形的相位关系

其中, U_1 为上限电压, U_2 为下限电压, U_x 为待输出电压。1.2V 对应的数字量为 BCH, -2.0V 对应的数字量为 19H。

在双缓冲方式下, 需要执行两条输出指令: 第一条输出指令打开 DAC0832 的第一级输入寄存器, 第二条输出指令打开第二级 DAC 寄存器。第二条输出指令中数据无意义, 此指令仅使 XFER 引脚有效, 打开 DAC 寄存器。其电路原理如图 4-52 所示。

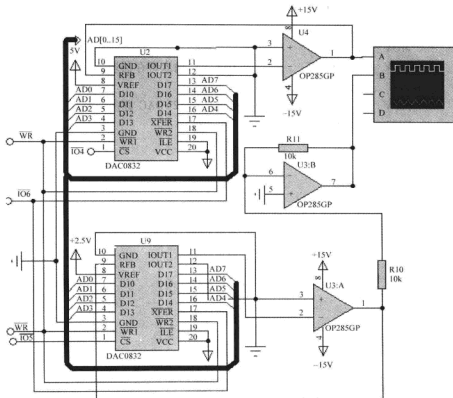


图 4-52 DAC0832 双缓冲工作方式的电路原理

参考程序:

```
.MODEL SMALL
```

```
.8086
```

```
.STACK
```

```
.CODE
```

```
.STARTUP
```

```
AA1: MOV AL,0
```

```
MOV DX,0800H ;0832(1)输入寄存器地址
```

```
OUT DX,AL
```

```
MOV AL,19H
```

```
MOV DX,0A00H ;0832(2)输入寄存器地址
```

```

OUT DX,AL
MOV DX,0C00H          ;0832(1)(2)DAC 寄存器地址
OUT DX,AL
CALL DELAY
MOV AL,0FFH
MOV DX,0800H          ;0832(1)输入寄存器地址
OUT DX,AL
MOV AL,0BCH
MOV DX,0A00H          ;0832(2)输入寄存器地址
OUT DX,AL
MOV DX,0C00H          ;0832(1)(2)DAC 寄存器地址
OUT DX,AL
CALL DELAY
JMP AA1
DELAY PROC NEAR
MOV BX,500
LP1:  MOV CX,469
LP2:  LOOP LP2
      DEC BX
      JNZ LP1
      RET
DELAY ENDP
DATA
END

```

4.11.3 DAC0832 直通工作方式

采用直通方式，利用 DAC0832 产生单极性锯齿波，其电路原理如图 4-53 所示。

由于采用直通方式，DAC0832 的第一级输入寄存器和第二级的 DAC 寄存器一直处于打开状态，因此要求 ILE 接高电平，CS、WR1、WR2、XFER 接地。

DAC0832 采用直通方式，CPU 输出的数据可直接传送到 DAC0832 的 8 位 D/A 转换器进行转换。在这种情况下，如果还是把 DAC0832 的数据输入端直接连在 CPU 数据总线上，数据总线上的任何数据都会导致 D/A 变换输出，这在实际工程中是不允许的。因此，DAC0832 的数据输入端不能直接连在 CPU 的数据总线上，来自 CPU 的数据总线上的数据必须经过锁存后才能传送到 DAC032 的数据输入端。本例将 DAC0832 的数据输入端连接到可编程并行接口芯片 8255A 的 PA 口，通过 8255A 将来自 CPU 的数据锁存。

锯齿波上升部分，采用数据值加 1 的方法，使输出数据由 00H 变化到 0FFH。在下降时，由 0FFH 自动加 1 变化为 00H。

参考程序：


```

DELAY    ENDP
.DATA
END

```

4.12 电动机控制

4.12.1 用 DAC0808 数/模转换器实现电动机调速

用 8 个按键控制电动机的转速，将转速分成 8 个等级，SW1 设为停止键，SW1 按键按下或没有按键按下时电动机停止，SW2 按键按下时电动机转速最低，SW7 按键对应的电动机转速最高。其电路原理如图 4-54 所示。

参考程序：

```

.MODEL    SMALL
.8086
.STACK
.CODE
.STARTUP
AGAIN:MOV  DX,0400H      ;244 地址
      IN   AL,DX         ;读入开关状态
      MOV  DX,0200H      ;273 地址(数/模转换数字量)
      TEST AL,01H        ;测试 SW1 按键是否按下
      JZ   SPEED0
      TEST AL,02H        ;测试 SW2 按键是否按下
      JZ   SPEED1
      TEST AL,04H        ;测试 SW3 按键是否按下
      JZ   SPEED2
      TEST AL,08H        ;测试 SW4 按键是否按下
      JZ   SPEED3
      TEST AL,10H        ;测试 SW5 按键是否按下
      JZ   SPEED4
      TEST AL,20H        ;测试 SW6 按键是否按下
      JZ   SPEED5
      TEST AL,40H        ;测试 SW7 按键是否按下
      JZ   SPEED6
      TEST AL,80H        ;测试 SW8 按键是否按下
      JZ   SPEED7
SPEED0:MOV  AL,0
      OUT  DX,AL         ;输出数字量
AGAIN1:CALL DELAY        ;等待 D/A 转换结束

```

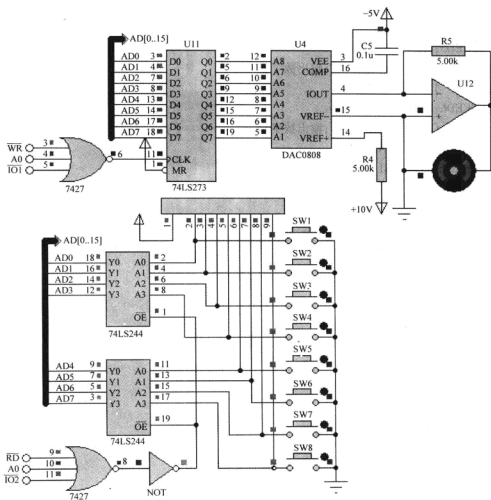


图 4-54 用 DAC0808 数/模转换器实现电动机调速的电路原理

```

JMP AGAIN
SPEED1:MOV AL,35
OUT DX,AL ;输出数字量
JMP AGAIN1
SPEED2:MOV AL,70
OUT DX,AL ;输出数字量
JMP AGAIN1
SPEED3:MOV AL,105
OUT DX,AL ;输出数字量
JMP AGAIN1
SPEED4:MOV AL,140

```

```

        OUT  DX,AL      ;输出数字量
        JMP  AGAIN1
SPEED5:MOV  AL,175
        OUT  DX,AL      ;输出数字量
        JMP  AGAIN1
SPEED6:MOV  AL,210
        OUT  DX,AL      ;输出数字量
        JMP  AGAIN1
SPEED7:MOV  AL,255
        OUT  DX,AL      ;输出数字量
        JMP  AGAIN1
;fosc = 5MHz, T = 0.2μs, Td ≈ 1ms;满转速
DELAY    PROC  NEAR    ;延时子程序
        PUSH BX
        PUSH CX
        MOV BX,1        ;4
DEL1:    MOV CX,295      ;4
DEL2:    LOOP DEL2       ;17/5
        DEC BX           ;2
        JNZ DEL1        ;16/4
        POP CX
        POP BX
        RET
        DELAY ENDP
END

```

4.12.2 直流电动机正反转控制

点动 SW1 按键控制直流电动机正转，点动 SW2 按键控制直流电动机反转，点动 SW3 按键电动机停止，在进行相应操作时，对应 LED 将被点亮。其电路原理如图 4-55 所示。

当 A 点为低电平时，Q3、Q2 截止，Q7、Q1 导通，电动机左端为高电平；当 B 点为高电平时，Q8、Q4 截止，Q6、Q5 导通，电动机右端为低电平。因此在 A 点为 0，B 点为 1 时，电动机正转。

当 A 点为高电平时，Q3、Q2 导通，Q7、Q1 截止，电动机左端为低电平；当 B 点为低电平时，Q8、Q4 导通，Q6、Q5 截止，电动机右端为高电平。因此，在 A 点为 1，B 点为 0 时，电动机反转。

当 A、B 两点同为高电平或同为低电平时，电动机两端电位相等，电动机停止。

参考程序：

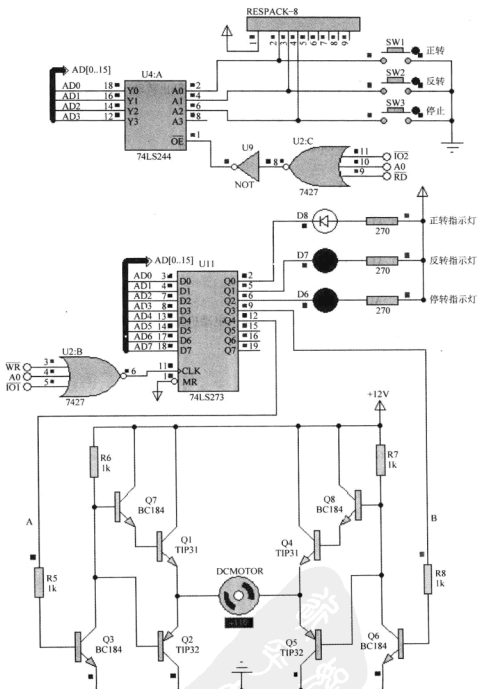


图 4-55 直流电动机正反转控制的电路原理

```

.MODEL      SMALL
.8086
.STACK
.CODE
.STARTUP

    MOV AL,11111011B      ;电动机停止转动
    MOV DX,0200H
    OUT DX,AL
AGAIN:  MOV DX,0400H      ;244 地址
        IN  AL,DX         ;读入开关状态
        TEST AL,01H       ;正转
        JZ  CLOCKWISE
        TEST AL,02H       ;反转
        JZ  UNCLOCKWISE
        TEST AL,04H
        JZ  STOP
        JMP AGAIN
CLOCKWISE:MOV AL,11101110B ;电动机正转
        MOV DX,0200H
        OUT DX,AL
        JMP AGAIN
UNCLOCKWISE:MOV AL,11110101B ;电动机反转
        MOV DX,0200H
        OUT DX,AL
        JMP AGAIN
STOP:   MOV AL,11111011B   ;电动机停止转动
        MOV DX,0200H
        OUT DX,AL
        JMP AGAIN

.DATA
END

```

4.12.3 步进电动机正反转控制

点动 SW1 按键控制步进电动机正转，点动 SW2 按键控制步进电动机反转，点动 SW3 按键控制步进电动机停止，在进行相应操作时，对应 LED 将被点亮。其电路原理如图 4-56 所示。

设步进电动机空载时满转速为 360r/min，则每秒 6 转，1 转用时 1/6s，1/8 转用时 1/48ms 即约 20ms。在实际工程中有负载情况下，应注意避免失步，即启动时速度应逐渐增大。四相步进电动机的 3 种励磁方式见表 4-3。

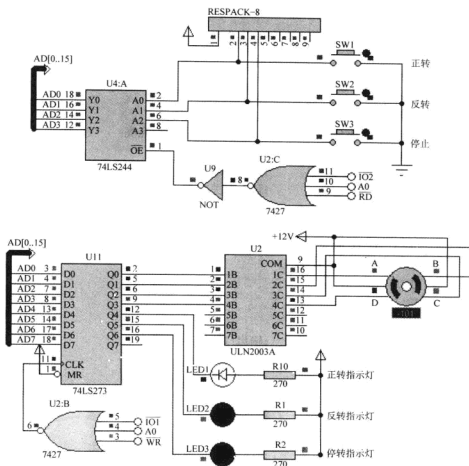


图 4-56 步进电动机正反转控制的电路原理

表 4-3 四相步进电动机的 3 种励磁方式

单 4 拍					双 4 拍					8 拍				
STEP	A	B	C	D	STEP	A	B	C	D	STEP	A	B	C	D
1	1	0	0	0	1	1	1	0	0	1	1	0	0	0
2	0	1	0	0	2	0	1	1	0	2	1	1	0	0
3	0	0	1	0	3	0	0	1	1	3	0	1	0	0
4	0	0	0	1	4	1	0	0	1	4	0	1	1	0
1	1	0	0	0	1	1	1	0	0	5	0	0	1	0
2	0	1	0	0	2	0	1	1	0	6	0	0	1	1
3	0	0	1	0	3	0	0	1	1	7	0	0	0	1
4	0	0	0	1	4	1	0	0	1	8	1	0	0	1

1) 参考程序。

.MODEL SMALL

.8086

.STACK

.CODE

.STARTUP

MOV DX,0200H

MOV AL,0B3H ;电动机停止,指示灯点亮

OUT DX,AL

AGAIN: MOV DX,0400H ;244 地址

IN AL,DX ;读入开关状态

TEST AL,01H ;正转

JZ CLOCKWISE

TEST AL,02H ;反转

JZ UNCLOCKWISE

JMP AGAIN

;电动机正转

CLOCKWISE: MOV SI,0

LOP0: MOV DX,0200H ;输出口地址

MOV AL,FFW[SI]

OUT DX,AL

CALL DELAY ;延时

MOV DX,0400H ;244 地址

IN AL,DX ;读入开关状态

TEST AL,02H ;反转按键是否按下

JZ UNCLOCKWISE

TEST AL,04H ;停止按键是否按下

JZ STOP

INC SI

CMP SI,8

JB LOP0

JMP CLOCKWISE

;电动机反转

UNCLOCKWISE: MOV SI,0

LOP1: MOV DX,0200H ;输出口地址

MOV AL,REV[SI]

OUT DX,AL

CALL DELAY ;延时

MOV DX,0400H ;244 地址

IN AL,DX ;读入开关状态

```

TEST AL,01H          ;正转按键是否按下
JZ  CLOCKWISE
TEST AL,04H          ;停止按键是否按下
JZ  STOP
INC SI
CMP SI,8
JB  LOP1
JMP UNCLOCKWISE
;电动机停止
STOP:  MOV DX,0200H
      MOV AL,0B3H      ;电动机停止,指示灯点亮
      OUT DX,AL
      JMP AGAIN

;fosc = 5MHz, T = 0.2μs, Td ≈ 25ms ;满转速
DELAY  PROC  NEAR      ;延时子程序
      PUSH BX
      PUSH CX
      MOV BX,25         ;4
DEL1:  MOV CX,295       ;4
DEL2:  LOOP DEL2        ;17/5
      DEC BX            ;2
      JNZ DEL1          ;16/4
      POP CX
      POP BX
      RET
DELAY  ENDP
. DATA
;正转
FFW  DB  069H,068H,06CH,064H,066H,062H,063H,061H
;反转
REV  DB  051H,053H,052H,056H,054H,05CH,058H,059H
END

```

2) 练习。修改程序,使步进电动机正转2圈90°,再反转45°。

4.13 8279 键盘显示接口

4.13.1 利用8279 键盘显示接口显示键号

8279 键盘显示接口可连接64个行列式按键,本实例中仅画出了4行8列共32个键。如

图 4-57 所示, 当有按键输入时, 键号值显示在 8 位 LED 数码管的左端, 再有按键输入时, 仍显示在最左端, 原先输入的键号值左移, 即模拟计算器的数字输入方式, 当按下大于 F 的键号时, LED 数码管显示 “Error”。

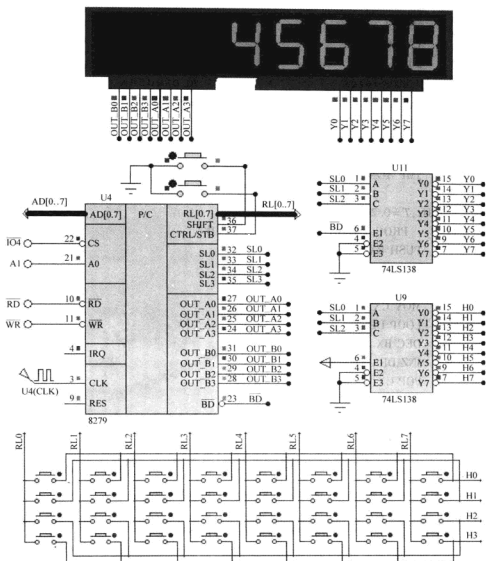


图 4-57 利用 8279 键盘显示接口显示键号的电路原理

1) 参考程序。

.MODEL SMALL

.8086

.STACK

```

.CODE
.STARTUP
    PORT EQU 0800H
INIT_8279: MOV DX, PORT + 2          ;8279 命令/状态口地址
            MOV AL, 02H              ;8 位, 左入; 编码扫描键盘, 2 键封锁
            OUT DX, AL
            MOV AL, 34H              ;时钟 2 分频
            OUT DX, AL
            MOV AL, 0D7H             ;清除显示 RAM, 清 0
            OUT DX, AL
WAIT1:    IN  AL, DX
            AND AL, 80H
            JNZ WAIT1               ;第 7 位 Dv, 若为 1, 表示显示器不可用; 若为
                                     ; 0, 表示准备就绪

LOP: CALL DISP
KEY: MOV DX, PORT + 2              ;8279 命令/状态口地址
     IN  AL, DX
     AND AL, 0FH
     JZ  KEY                       ;有键按下
     MOV AL, 40H                   ;读 FIFO 键盘缓冲区
     OUT DX, AL
     MOV DX, PORT                  ;8279 数据口地址
     IN  AL, DX
     AND AL, 3FH
     MOV KEY_TEMP, AL              ;暂存键号
     SUB AL, 10H                   ;非数字键则跳转
     JAE ERROR

     MOV AL, DISP_BUF + 1          ;显示数码左移
     MOV DISP_BUF, AL
     MOV AL, DISP_BUF + 2          ;显示数码左移
     MOV DISP_BUF + 1, AL
     MOV AL, DISP_BUF + 3          ;显示数码左移
     MOV DISP_BUF + 2, AL
     MOV AL, DISP_BUF + 4          ;显示数码左移
     MOV DISP_BUF + 3, AL
     MOV AL, DISP_BUF + 5          ;显示数码左移

```

```

MOV DISP_BUF + 4, AL
MOV AL, DISP_BUF + 6           ;显示数码左移
MOV DISP_BUF + 5, AL
MOV AL, DISP_BUF + 7           ;显示数码左移
MOV DISP_BUF + 6, AL
MOV AL, KEY_TEMP               ;恢复键号
MOV DISP_BUF + 7, AL
JMP LOP

ERROR: MOV BYTE PTR DISP_BUF + 0, 10H ;消隐
MOV BYTE PTR DISP_BUF + 1, 10H ;消隐
MOV BYTE PTR DISP_BUF + 2, 10H ;消隐
MOV BYTE PTR DISP_BUF + 3, 11H ;E
MOV BYTE PTR DISP_BUF + 4, 12H ;R
MOV BYTE PTR DISP_BUF + 5, 12H ;R
MOV BYTE PTR DISP_BUF + 6, 13H ;O
MOV BYTE PTR DISP_BUF + 7, 12H ;R
CALL DISP

KEY1: MOV DX, PORT + 2           ;8279 命令/状态口地址
IN AL, DX
AND AL, 0FH
JZ KEY1                         ;有键按下
MOV AL, 40H ;
OUT DX, AL
MOV DX, PORT                   ;8279 数据口地址
IN AL, DX
AND AL, 3FH
MOV KEY_TEMP, AL               ;暂存键号
SUB AL, 10H                    ;非数字键则跳转
JAE ERROR

MOV BYTE PTR DISP_BUF + 0, 10H ;消隐
MOV BYTE PTR DISP_BUF + 1, 10H ;消隐
MOV BYTE PTR DISP_BUF + 2, 10H ;消隐
MOV BYTE PTR DISP_BUF + 3, 10H ;消隐
MOV BYTE PTR DISP_BUF + 4, 10H ;消隐
MOV BYTE PTR DISP_BUF + 5, 10H ;消隐
MOV BYTE PTR DISP_BUF + 6, 10H ;消隐
MOV AL, KEY_TEMP               ;恢复键号
MOV DISP_BUF + 7, AL

```


JMP LOP

DISP PROC NEAR

MOV DX,PORT + 2 ;8279 命令/状态口地址
MOV AL,90H ;8 位,左入;编码扫描键盘,2 键封锁

OUT DX,AL

LEA SI,DISP_BUF

MOV DX,PORT ;8279 数据口地址

MOV CX,8

LEA BX,TAB1

DLO: MOV AL,[SI]

XLAT

OUT DX,AL

INC SI

LOOP DLO

RET

DISP ENDP

. DATA

KEY_TEMP DB ?

DISP_BUF DB 8 DUP (10H) ;显示缓冲区

TAB1 DB 3FH,06H,5BH,4FH,66H,6DH,7DH,07H ;共阴极段码表 0 ~ F

DB 7FH,6FH,77H,7CH,39H,5EH,79H,71H

DB 00H ;消隐

DB 79H,31H,5CH ;E,r,o 段码

END

2) 8279 调试窗口。暂停程序执行,右键单击 8279 仿真芯片,在弹出的快捷菜单中,勾选最下面的选项“Keyboard FIFO_Display RAM”,单步或断点执行,观察 8279 内部寄存器数据变化。

4.13.2 利用 8279 键盘显示接口显示时钟

利用 8279 键盘显示接口显示“12-59-50”格式的电子钟,其电路原理同图 4-57。

参考程序:

. MODEL SMALL

. 8086

. STACK

. CODE

. STARTUP

PORT EQU 0800H

INIT_8279:MOV DX,PORT + 2 ;8279 命令/状态口地址



```

MOV AL,00H                ;8 位,左入;编码扫描键盘,2 键封锁
OUT DX,AL
MOV AL,34H                ;时钟 2 分频
OUT DX,AL
MOV AL,0D7H               ;清除显示 RAM,清 0
OUT DX,AL
WAIT1: IN AL,DX
      AND AL,80H
      JNZ WAIT1           ;第 7 位 Dv,若为 1,表示显示器不可用;若为
                          ; 0,表示准备就绪
LOP:CALL DISP
      CALL DELAY

MOV BX,OFFSET DISP_BUF
INC BYTE PTR [BX+7]       ;秒个位加 1
CMP BYTE PTR [BX+7],0AH
JNE LOP
MOV BYTE PTR [BX+7],0
INC BYTE PTR [BX+6]       ;秒十位加 1
CMP BYTE PTR [BX+6],06H
JNE LOP
MOV BYTE PTR [BX+6],0

INC BYTE PTR [BX+4]       ;分个位加 1
CMP BYTE PTR [BX+4],0AH
JNE LOP
MOV BYTE PTR [BX+4],0
INC BYTE PTR [BX+3]       ;分十位加 1
CMP BYTE PTR [BX+3],06H
JNE LOP
MOV BYTE PTR [BX+3],0

INC BYTE PTR [BX+1]       ;时个位加 1
CMP BYTE PTR [BX],02H
JE HOUR4
CMP BYTE PTR [BX+1],0AH

```

```

JNE LOP
MOV BYTE PTR [BX+1],0
INC BYTE PTR [BX] ;时十位加1
JMP LOP
HOUR4: CMP BYTE PTR [BX+1],04H
JNE LOP
MOV BYTE PTR [BX+1],0 ;时个位清0
MOV BYTE PTR [BX],0 ;时十位清0
JMP LOP

DELAY PROC NEAR ;延时子程序
PUSH BX
PUSH CX
MOV BX,200
LP1: MOV CX,469
LP2: LOOP LP2
DEC BX
JNZ LP1
POP CX
POP BX
RET
DELAY ENDP

DISP PROC NEAR
MOV DX,PORT+2 ;8279 命令/状态口地址
MOV AL,90H ;8 位,左入;编码扫描键盘,2 键封锁
OUT DX,AL
LEA SI,DISP_BUF
MOV DX,PORT ;8279 数据口地址
MOV CX,8
LEA BX,TAB1
D10: MOV AL,[SI]
XLAT
OUT DX,AL
INC SI
LOOP D10
RET

```



```
DISP ENDP
```

```
. DATA
```

```
;时 - 分 - 秒
```

```
DISP_BUF DB 02H,03H,10H,05H,09H,10H,05H,07H
```

```
TAB1 DB 3FH,06H,5BH,4FH,66H,6DH,7DH,07H ;共阴极段码表 0~F
```

```
DB 7FH,6FH,77H,7CH,39H,5EH,79H,71H
```

```
DB 40H
```

```
; -
```

```
END
```

4.14 存储体扩展

如图 4-58 所示, 扩展两片 6264 存储体, U3 为奇地址存储体, U4 为偶地址存储体, 两芯片的片选接 4-16 译码器的输出 M1, 设未用地址信号为 0, 则图 4-58 中两存储体的地址范围为 10000H ~ 13FFFH。编程实现将 100 个字 'AB' 送到 10000H 地址开始的连续存储单元。

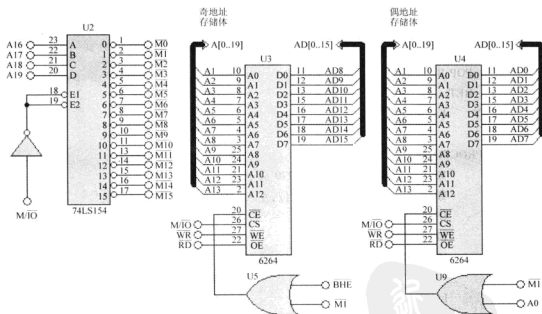


图 4-58 存储体扩展的电路原理

1) 参考程序。

```
. MODEL SMALL
```

```
. 8086
```

```
. STACK
```

```

.CODE
.STARTUP
AGAIN:
    MOV AX,1000H
    MOV DS,AX
    MOV BX,0
    MOV AX,'AB'
    MOV CX,100
LOP:  MOV [BX],AX
    ADD BX,2
    LOOP LOP
    JMP $

```

```

.DATA
END

```

2) 6264 数据调试窗口。暂停, 右键单击 6264 仿真芯片, 在弹出的快捷菜单中, 勾选最下面的选项“MEMORY CONTENTS”, 在最后一指令 `JMP $` 处设置断点, 单步或全速执行观察两片 6264 窗口的数据化。

3) 练习。再添加两片 6264 使其地址范围为 20000H ~ 23FFFH, 编程实现将 100 个字‘CD’ 送到 20000H 地址开始的连续存储单元。

4.15 8259A 可编程中断控制器

8259A 可编程中断控制器的每个输入控制 1 只 LED, 若有中断输入时对应的 LED 状态取反一次。如图 4-59 所示, 当 `IR0` 有中断申请时, `LED0` 状态取反; 当 `IR1` 有中断申请时, `LED1` 状态取反; 其他类同。

需要注意的是, 本实例虽然能从现象上说明每一个中断的发生, 但程序没有真实地体现出 8259A 的中断功能, `IR0 ~ IR7` 所有仿真中断申请都使用了相同的中断类型号 0 (实际芯片中断类型号应该是 `ICW2 + 0 ~ 7`, 且每一中断有对应的中断矢量)。本实例中断矢量表仅初始化了 0 号中断向量, 进入同一个中断服务程序后再根据 `ISR` 中断服务寄存器的值转入对应的分支。

参考程序:

```

.MODEL SMALL
.8086
.STACK
.CODE
.STARTUP
START:
    CLI

```

;CPU 关中断




```
MOV [DI],AX
ADD DI,2
MOV AX,CS
MOV [DI],AX
POP DS
STI                                ;CPU 开中断

LOP: MOV DX,0201H                ;主任务实现 8 只 LED 闪烁
MOV AL,55H
OUT DX,AL
CALL DELAY
MOV AL,0AAH
OUT DX,AL
CALL DELAY
JMP LOP

INT_SERVER: CLI                    ;中断服务程序
PUSH DX
PUSH AX
MOV DX,0400H
MOV AL,0BH                        ;OCW3 读 ISR 命令字
OUT DX,AL
IN AL,DX
MOV AH,AL                        ;暂存 ISR 中断服务寄存器的状态

IRQ0: MOV AL,AH
AND AL,01H
JZ IRQ1
MOV DX,0200H
XOR VALUE_273,01H                ;D0 位取反
MOV AL,VALUE_273
OUT DX,AL
JMP EXIT_INT

IRQ1: MOV AL,AH
AND AL,02H
JZ IRQ2
MOV DX,0200H
```

```
XOR    VALUE_273,02H      ;D1 位取反
MOV     AL,VALUE_273
OUT     DX,AL
JMP     EXIT_INT
IRQ2:   MOV     AL,AH
        AND     AL,04H
        JZ      IRQ3
        MOV     DX,0200H
        XOR     VALUE_273,04H    ;D2 位取反
        MOV     AL,VALUE_273
        OUT     DX,AL
        JMP     EXIT_INT
IRQ3:   MOV     AL,AH
        AND     AL,08H
        JZ      IRQ4
        MOV     DX,0200H
        XOR     VALUE_273,08H    ;D3 位取反
        MOV     AL,VALUE_273
        OUT     DX,AL
        JMP     EXIT_INT
IRQ4:   MOV     AL,AH
        AND     AL,10H
        JZ      IRQ5
        MOV     DX,0200H
        XOR     VALUE_273,10H    ;D4 位取反
        MOV     AL,VALUE_273
        OUT     DX,AL
        JMP     EXIT_INT
IRQ5:   MOV     AL,AH
        AND     AL,20H
        JZ      IRQ6
        MOV     DX,0200H
        XOR     VALUE_273,20H    ;D5 位取反
        MOV     AL,VALUE_273
        OUT     DX,AL
        JMP     EXIT_INT
IRQ6:   MOV     AL,AH
```




```

    AND    AL,40H
    JZ     IRQ7
    MOV    DX,0200H
    XOR    VALUE_273,40H      ;D6 位取反
    MOV    AL,VALUE_273
    OUT    DX,AL
    JMP    EXIT_INT
IRQ7: MOV    AL,AH
    AND    AL,80H
    JZ     EXIT_INT
    MOV    DX,0200H
    XOR    VALUE_273,80H      ;D7 位取反
    MOV    AL,VALUE_273
    OUT    DX,AL

EXIT_INT: MOV    DX,0400H
    MOV    AL,20H
    OUT    DX,AL              ;OCW2,发EOI清ISR
    POP    AX
    POP    DX
    IRET                      ;中断返回

DELAY    PROC    NEAR        ;延时子程序
    PUSH    BX
    PUSH    CX
    MOV     BX,200
LP1:     MOV    CX,469
LP2:     LOOP   LP2
    DEC     BX
    JNZ     LP1
    POP     CX
    POP     BX
    RET
DELAY    ENDP
. DATA
VALUE_273 DB 00H
    END

```



第5章 高级C语言程序设计

C语言是非常流行的一种编程语言，具有可读性强、编程简单、效率高、程序调试方便等优点，可实现汇编语言的大部分功能，生成的目标代码质量高且执行速度较快。因此，工程上多数都用C语言代替汇编语言。

5.1 开关检测与状态显示

利用74LS244 简单输入接口检测开关状态，利用74LS273 输出接口输出开关状态，其电路原理如图5-1所示。

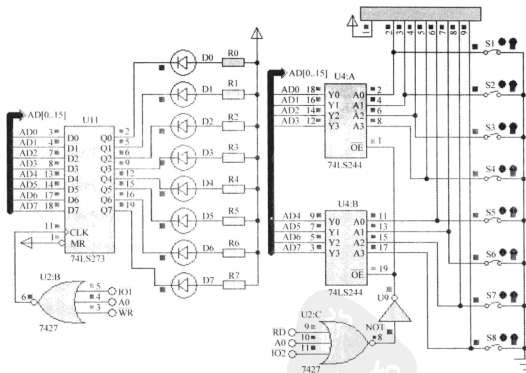


图 5-1 开关检测与状态显示的电路原理

Proteus 仿真模型是在没有 IBM PC BIOS 或者 MS - DOS 存在的情况下运行程序，它们不能在标准的 RTL (Run - Time Libraries) 下被编译，因为标准编译器的 RTL 利用的是 BIOS 和 MS - DOS 的函数调用。为了实现应用系统的 RTL (Run - Time Libraries)，需要

编写驱动系统低层硬件的输入/输出函数。程序执行时由入口点调用C语言程序中的main函数。

1) 参考程序。

程序入口点“RTL ASM”(Run Time Library)汇编程序文件

```
.MODEL SMALL
```

```
.8086
```

```
.CODE
```

```
EXTRN _MAIN:PROC
```

```
.STARTUP
```

```
CALL NEAR PTR _MAIN
```

```
ENDLESS;
```

```
JMP ENDLESS
```

```
.DATA
```

```
PUBLIC _ACRTUSED ; 16 或 32 位 DOS 编译器和 main() 函数启动参数
```

```
_ACRTUSED = 9876H ; FUNNY VALUE NOT EASILY MATCHED IN SYMDEB
```

```
.STACK
```

```
END
```

C 语言程序“MAIN.C”文件

```
void outp (unsigned int addr, char data)
```

```
//输出一字节到 I/O 端口
```

```
{_asm
```

```
{mov dx, addr
```

```
mov al, data
```

```
out dx, al
```

```
}
```

```
}
```

```
char inp(unsigned int addr)
```

```
//从 I/O 端口输入一字节
```

```
{char result;
```

```
_asm
```

```
{mov dx, addr
```

```
in al, dx
```

```
mov result, al
```

```
}
```

```
return result;
```

```
}
```

```
void main(void)
```

```
{
```

```
while (1)
```

PDG

```

{ char button_state;
  button_state = inp(0x0400); //输入
  outp(0x0200, button_state); //输出
}
}

```

批处理文件“BUILD. BAT”

```

ml /Zd /Zi /Zf -c RTL.ASM
dmc -g -ms -0 -c -o main.obj main.c
link /CO /NOD /DEB /DEBUGB main.obj + rtl.obj

```

上述批处理文件“BUILD. BAT”是使用 Digital Mars C/C++ 编译器的批处理文档，同时需要 MASM32 编译器对汇编程序文件进行汇编。其他的 C 语言编译器批处理命令格式请参阅 Proteus 帮助文档。在使用 Digital Mars C/C++ 编译器前需要在 Windows 环境变量 PATH 中添加编译器“dmc.exe”和连接器“link.exe”的搜索路径“C:\DM\BIN”。

2) 实验步骤。

① 新建工作目录“D:\Proteus_C\5_1”，单击【开始】→【程序】→【附件】→【C:\命令提示符】转至 DOS 目录，执行 cd proteus_c、cd 5_1 命令转至新建工作目录，如图 5-2 所示。

```

C:\Documents and Settings\Administrator>cd proteus_c
D:\>cd proteus_c
D:\Proteus_C>cd 5_1
D:\Proteus_C\5_1>edit rtl.asm

```

图 5-2 执行 DOS 命令转至工作目录

② 执行“edit rtl.asm”打开 edit 文本编辑器（或用记事本等其他编辑器），输入上述的 RTL ASM 汇编源程序并以 rtl.asm 作为文件名存盘于当前工作目录。

③ 同步步骤②，输入上述的“MAIN.C”文档并以 main.c 作为文件名存盘至当前工作目录。

④ 同步步骤②，输入上述的“BUILD. BAT”文档并以 build.bat 作为文件名存盘至当前工作目录。至此在当前目录下已建立工程中的 3 个文件：rtl.asm、main.c 和 build.bat。

⑤ 如图 5-3 所示，在 DOS 当前工作目录下执行 build 批处理命令，编译上述源程序，若源程序中有错误，则会提示错误信息，修改对应文档源程序中的错误，执行 build 批处理后重新编译连接，直到没有编译连接错误。打开工作目录会看到目录下生成了 3 个文件 rtl.obj、main.obj 和 main.exe。其中，rtl.obj 是 rtl.asm 的目标文件，main.obj 是 main.c 的目标文件，main.exe 是将 rtl.obj 和 main.obj 连接后生成的可执行文件。

⑥ 复制“5_1_C 语言基础”下原理图“244_273_IO.dsn”至当前工作目录并打开原理图，编辑 8086 元器件属性，修改 Program File 为“main.exe”。全速执行并按下开关观察程序执行效果，或单步执行调试程序。

```
D:\Proteus_CNS_1>build
D:\Proteus_CNS_1>cd ..\Zn\Zd\Zf\ZF && Rtl.asm
Microsoft (R) Macro Assembler Version 6.14.8446
Copyright (C) Microsoft Corp 1981-1997. All rights reserved.

Assembling: RTL.BSM
D:\Proteus_CNS_1>doc -O g -ng -H c:\c\dms\INCLUDE main.obj main.c
main.c
D:\Proteus_CNS_1>link /CO /NOD /DEF DEF00GE /STACK:1024 main.obj rtl.obj
D:\Proteus_CNS_1>
```

图 5-3 执行 build 编译连接批处理

5.2 LCD1602 电子钟

用 LCD1602 字符液晶显示器设计一个可调小时和分钟的电子钟, 其电路原理如图 5-4 所示。按“设置”键第 1 行显示“SEG <>”, 指示要调整的对象(小时或分钟), “加”键用于调整, 调整后按“保存”键退出。

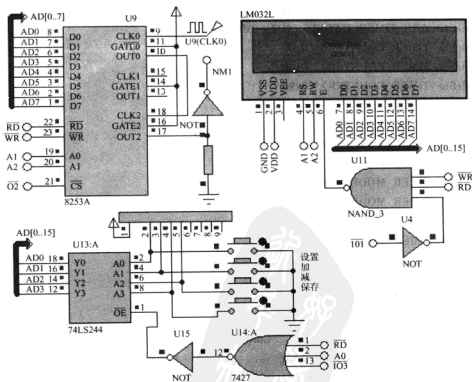


图 5-4 LCD1602 电子钟的电路原理

参考程序:

;rtl.asm 源程序同 5.1 节的源程序

;main.c 程序

```
#define IO1    0x0200
```

```
#define IO2    0x0400
```

```
#define IO3    0x0600
```

```
#define ADR_244 IO3
```

```
//LCD Registers addresses
```

```
#define LCD_CMD_WR    (IO1 + 0x00)
```

```
#define LCD_DATA_WR    (IO1 + 0x02)
```

```
#define LCD_BUSY_RD    (IO1 + 0x04)
```

```
#define LCD_DATA_RD    (IO1 + 0x06)
```

```
/* 可编程定时/计数器接口 8253A */
```

```
#define ADR_TIMER_CONTROL (IO2 + 0x06)
```

```
#define ADR_TIMER_DATA0 (IO2 + 0x00)
```

```
#define ADR_TIMER_DATA1 (IO2 + 0x02)
```

```
#define ADR_TIMER_DATA2 (IO2 + 0x04)
```

```
#define TIMER_COUNTER0 0x00
```

```
#define TIMER_COUNTER1 0x40
```

```
#define TIMER_COUNTER2 0x80
```

```
#define TIMER_LATCH 0x00
```

```
#define TIMER_LSB 0x10
```

```
#define TIMER_MSB 0x20
```

```
#define TIMER_LSB_MSB 0x30
```

```
#define TIMER_MODE0 0x00
```

```
#define TIMER_MODE1 0x02
```

```
#define TIMER_MODE2 0x04
```

```
#define TIMER_MODE3 0x06
```

```
#define TIMER_MODE4 0x08
```

```
#define TIMER_MODE5 0x09
```

```
#define TIMER_BCD 0x01
```

```
unsigned char    Str1[ ] = "    Current Time";
```

```
unsigned char    Str2[ ] = "Set          ";
```

```
unsigned char    HMS_String[ ] = "12:30:00 ";
```

```
unsigned char    Hour = 12, Minute = 30, Second = 0;
```

```
unsigned char Settime;
unsigned char Change_H_or_M = 1;

void outp(unsigned int addr, char data)
//输出一字节到 I/O 端口
{
    _asm
    {
        mov dx, addr
        mov al, data
        out dx, al
    }
}

char inp(unsigned int addr)
//从 I/O 端口输入一字节
{
    char result;
    _asm
    {
        mov dx, addr
        in al, dx
        mov result, al
    }
    return result;
}

//设置中断矢量表
void set_int(unsigned char int_no, void * service_proc)
{
    _asm
    {
        push es
        xor ax, ax
        mov es, ax
        mov al, int_no
        xor ah, ah
        shl ax, 1
        shl ax, 1
        mov si, ax
        mov ax, service_proc
        mov es:[si], ax
        inc si
        inc si
        mov bx, cs
        mov es:[si], bx
        pop es
    }
}
```



```
    }  
}  
//中断处理函数  
void interrupt_far nmi_handler( void )  
{  
    if( ++Second == 60 )  
    {  
        Second = 0;  
        if( ++Minute == 60 )  
        {  
            Minute = 0;  
            if( ++Hour == 24 )  
            { Hour = 0; Minute = 0; Second = 0; }  
        }  
    }  
}  
  
void setup_nmi( void )  
//在中断矢量表添加 2 号中断矢量  
{ set_int( 0x02, ( void * )&nmi_handler );  
//设置 8253A 定时/计数器  
    outp( ADR_TIMER_CONTROL, TIMER_COUNTER0 | TIMER_MODE3 | TIMER_LSB_  
MSB | TIMER_BCD );  
    outp( ADR_TIMER_DATA0, 0x00 );  
    outp( ADR_TIMER_DATA0, 0x01 );  
    outp( ADR_TIMER_CONTROL, TIMER_COUNTER2 | TIMER_MODE2 | TIMER_LSB_  
MSB | TIMER_BCD );  
    outp( ADR_TIMER_DATA2, 0x00 );  
    outp( ADR_TIMER_DATA2, 0x10 );  
}  
  
//LCD 忙等待  
void WaitForEnable( void )  
{  
    unsigned char result;  
    do {  
        result = inp( LCD_BUSY_RD );  
    } while( result & 0x80 );  
}
```



```
//LCD 写命令
void LcdWriteCommand(unsigned char cmd)
{
    WaitForEnable();
    outp(LCD_CMD_WR,cmd);
}

//LCD 写数据
void LcdWriteData(char data )
{
    WaitForEnable();
    outp(LCD_DATA_WR,data);
}

void LcdReset(void)
{
    LcdWriteCommand(0x38);
    LcdWriteCommand(0x0c);
    LcdWriteCommand(0x06);
    LcdWriteCommand(0x01);
}

void lcd_pos(unsigned char pos)
{
    LcdWriteCommand(pos|0x80);
}

void lcd_out(unsigned char *row)
{
    unsigned char *p;
    p = row;
    while( *p! = '\0')
    {
        LcdWriteData(*p);
        p++;
    }
}

void delay(unsigned int x)
{
    unsigned char i;
    while(x--)
```

```
    { for(i = 0; i < 120; i + + ) { } }
}
```

//时分秒转换为字符串

```
void hms_str(unsigned char h,unsigned char m,unsigned char s)
```

```
{
    HMS_String[4] = h/10 + '0';
    HMS_String[5] = h%10 + '0';
    HMS_String[7] = m/10 + '0';
    HMS_String[8] = m%10 + '0';
    HMS_String[10] = s/10 + '0';
    HMS_String[11] = s%10 + '0';
}
```

//调整时间

```
void Change_Time()
```

```
{
    Settime = 0;
    if( inp( ADR_244) != 0x0f)
    {
        lcd_pos( 0x00);
        lcd_out( Str2);
        Settime = 1;
    }
    while( Settime)
    {
        if( ! ( inp( ADR_244) & 0x01))
        {
            while( ! ( inp( ADR_244) & 0x01)) { }
            Change_H_or_M = ! Change_H_or_M;
            if( ! Change_H_or_M)
            { Str2[4] = ' '; Str2[5] = '-'; Str2[7] = '<'; Str2[8] = '>'; }
            else
            { Str2[4] = '<'; Str2[5] = '>'; Str2[7] = ' '; Str2[8] = ' '; }
            lcd_pos( 0x00); lcd_out( Str2);
        }
        if( ! ( inp( ADR_244) & 0x02))
        {
            while( ! ( inp( ADR_244) & 0x02)) { }

```

```
if( Change_H_or_M == 1 )
{ if( ++ Hour == 24 ) Hour = 0; }
else
{ if( ++ Minute == 60 ) Minute = 0; }
}

if( ! ( inp( ADR_244 ) & 0x04 ) )
{
while( ! ( inp( ADR_244 ) & 0x04 ) ) {}
if( Change_H_or_M == 1 )
{ if( -- Hour == 0xff ) Hour = 23; }
else
{ if( -- Minute == 0xff ) Minute = 59; }
}

if( ! ( inp( ADR_244 ) & 0x08 ) )
{
while( ! ( inp( ADR_244 ) & 0x08 ) ) {}
lcd_pos( 0x00 );
lcd_out( Str1 );
Settime = 0;
}

hms_str( Hour, Minute, Second );
lcd_pos( 0x40 );
lcd_out( HMS_String );
}

}

void main( void )
{
setup_nmi( );
LcdReset( );
lcd_pos( 0x00 );
lcd_out( Str1 );
while( 1 )
{
hms_str( Hour, Minute, Second );
lcd_pos( 0x40 );
lcd_out( HMS_String );
delay( 100 );
Change_Time( );
}
```

;build. bat 程序同 5.1 节

5.3 LCM12864 万年历

本例用 LCM12864 液晶显示屏同时显示数字和汉字，设计一个万年历时钟（见图 5-5），要求时钟调整时能反白显示，设置键用于选择调整年、月、日、时和分，加减键用于调整，调整后按“保存”键保存。

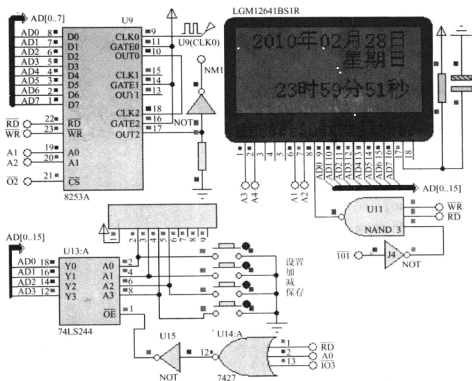


图 5-5 LCM12864 万年历的电路原理

参考程序：

;rtl. asm 源程序同 5.1 节的源程序

;main. h 头文件

#define uchar unsigned char

#define IO3 0x0600

#define ADR_244 IO3

//键盘输入接口

```

#define uint unsigned int
char Reverse_Display = 0;           //反显
extern char Sec_Flag;               //秒中断标志
uchar MonthsDays[] = {0,31,0,31,30,31,30,31,31,30,31,30,31};
uchar DateTime[7] = {50,59,23,28,02,0,10};
char Adjust_Index = -1;             //年月日时分选择标志
uchar H_Offset = 10, V_Page_Offset = 0; //显示位置
uchar DATE_TIME_WORDS[] = {        //取模方式: 阴码、逆向、列行式
    0x40,0x20,0x10,0x0C,0xE3,0x22,0x22,0x22,
    0xFE,0x22,0x22,0x22,0x22,0x02,0x00,0x00,
    0x04,0x04,0x04,0x04,0x07,0x04,0x04,0x04,
    0xFF,0x04,0x04,0x04,0x04,0x04,0x04,0x00,/* "年",0 */
    .....//“月日时分秒”略
};

uchar WEEKDAY_WORDS[] = {
    0x00,0x00,0x00,0xFE,0x42,0x42,0x42,0x42,
    0x42,0x42,0x42,0xFE,0x00,0x00,0x00,0x00,
    0x00,0x00,0x00,0x3F,0x10,0x10,0x10,0x10,
    0x10,0x10,0x10,0x3F,0x00,0x00,0x00,0x00,/* "日",0 */
    .....//“一二三四五六”略
};

uchar DIGITS[] = {
    0x00,0xE0,0x10,0x08,0x08,0x10,0xE0,0x00,
    0x00,0x0F,0x10,0x20,0x20,0x10,0x0F,0x00,/* "0",0 */
    .....//“123456789”略
};

;main.c 文件
#include "main.h"
//判断是否为闰年
uchar isLeapYear(uint y)
//是否闰年函数
{
    return(y%4 == 0 && y%100 != 0) || (y%400 == 0);
}

void RefreshWeekDay()

```

//计算星期几函数

```
{
    uint i,d,w=5;
    for(i=2000;i<2000+DateTime[6];i++)
    {
        d=isLeapYear(i)?366:365;
        w=(w+d)%7;
    }
    d=0;
    for(i=1;i<DateTime[4];i++)d+=MonthsDays[i];
    d+=DateTime[3];
    DateTime[5]=(w+d)%7+1;
}
```

void GetTime()

```
{
    if(Sec_Flag)
    {
        Sec_Flag=0;
        if(++DateTime[0]==60)    //秒加1
        {
            DateTime[0]=0;
            if(++DateTime[1]==60)    //分加1
            {
                DateTime[1]=0;
                if(++DateTime[2]==24)    //时加1
                {
                    DateTime[2]=0;
                    //获取2月天数
                    MonthsDays[2]=isLeapYear(2000+DateTime[6])?29:28;
                    if(DateTime[3]<MonthsDays[DateTime[4]])DateTime[3]++; //天加1
                    else
                    {
                        DateTime[3]=1;
                        if(++DateTime[4]==12)    //月加1
                        {
                            DateTime[4]=0;
                            if(++DateTime[6]==99)DateTime[6]=0; //年加1
                        }
                    }
                }
            }
        }
    }
}
```

```
        RefreshWeekDay(); //刷新星期
    }
}

void DateTime_Adjust(char x)
//时间调整函数
{
    switch(Adjust_Index)
    {
        case 6:
            if(x == 1 && DateTime[6] < 99) DateTime[6] ++;
            if(x == -1 && DateTime[6] > 0) DateTime[6] --;
            //获取2月天数
            MonthsDays[2] = isLeapYear(2000 + DateTime[6]) ? 29 : 28;
            if(DateTime[3] > MonthsDays[DateTime[4]])
                DateTime[3] = MonthsDays[DateTime[4]];
            RefreshWeekDay();
            break;

        case 4:
            if(x == 1 && DateTime[4] < 12) DateTime[4] ++;
            if(x == -1 && DateTime[4] > 1) DateTime[4] --;
            MonthsDays[2] = isLeapYear(2000 + DateTime[6]) ? 29 : 28;
            if(DateTime[3] > MonthsDays[DateTime[4]])
                DateTime[3] = MonthsDays[DateTime[4]];
            RefreshWeekDay();
            break;

        case 3:
            MonthsDays[2] = isLeapYear(2000 + DateTime[6]) ? 29 : 28;
            if(x == 1 && DateTime[3] < MonthsDays[DateTime[4]]) DateTime[3] ++;
            if(x == -1 && DateTime[3] > 0) DateTime[3] --;
            RefreshWeekDay();
            break;

        case 2:
            if(x == 1 && DateTime[2] < 23) DateTime[2] ++;
            if(x == -1 && DateTime[2] > 0) DateTime[2] --;
            break;
    }
}
```

```

case 1:
    if( x == 1 && DateTime[1] < 59) DateTime[1] ++;
    if( x == -1 && DateTime[1] > 0) DateTime[1] --;
    break;

/* case 0:
    if( x == 1 && DateTime[0] < 59) DateTime[0] ++;
    if( x == -1 && DateTime[0] > 0) DateTime[0] --;
    break;
    */
}

}

void key()
//检测是否有键闭合
{
    if( ! ( inp( ADR_244) & 0x01) ) // '选择' 键闭合
    {
        while( ! ( inp( ADR_244) & 0x01) ) {} //等待键释放
        if( Adjust_Index == -1 || Adjust_Index == 1) Adjust_Index = 7;
        Adjust_Index --;
        if( Adjust_Index == 5) Adjust_Index = 4;
    }
    else
    if( ! ( inp( ADR_244) & 0x02) ) // ' + ' 键闭合
    {
        while( ! ( inp( ADR_244) & 0x02) ) {} DateTime_Adjust( 1 );
    }
    else
    if( ! ( inp( ADR_244) & 0x04) ) // ' _ ' 键闭合
    {
        while( ! ( inp( ADR_244) & 0x04) ) {} DateTime_Adjust( -1 );
    }
    else
    if( ! ( inp( ADR_244) & 0x08) ) // ' 确定 ' 键闭合
    {
        while( ! ( inp( ADR_244) & 0x08) ) {}

        Adjust_Index = -1;
    }
}

```



```

void disp()           //LCD 显示输出函数
{
    Reverse_Display = Adjust_Index = 6;
    Display_A_Char(V_Page_Offset, 16 + H_Offset, DIGITS + DateTime[6]/10 * 16); //年
    Display_A_Char(V_Page_Offset, 24 + H_Offset, DIGITS + DateTime[6] % 10 * 16);
    Reverse_Display = Adjust_Index = 4;
    Display_A_Char(V_Page_Offset, 48 + H_Offset, DIGITS + DateTime[4]/10 * 16); //月
    Display_A_Char(V_Page_Offset, 56 + H_Offset, DIGITS + DateTime[4] % 10 * 16);
    Reverse_Display = Adjust_Index = 3;
    Display_A_Char(V_Page_Offset, 80 + H_Offset, DIGITS + DateTime[3]/10 * 16); //日
    Display_A_Char(V_Page_Offset, 88 + H_Offset, DIGITS + DateTime[3] % 10 * 16);
    Reverse_Display = Adjust_Index = 5;
    Display_A_WORD(V_Page_Offset + 2, 96 + H_Offset, WEEKDAY_WORDS + (DateTime
[5] - 1) * 32); //星期
    Reverse_Display = Adjust_Index = 2;
    Display_A_Char(V_Page_Offset + 5, 16 + H_Offset, DIGITS + DateTime[2]/10 * 16); //时
    Display_A_Char(V_Page_Offset + 5, 24 + H_Offset, DIGITS + DateTime[2] % 10 * 16);
    Reverse_Display = Adjust_Index = 1;
    Display_A_Char(V_Page_Offset + 5, 48 + H_Offset, DIGITS + DateTime[1]/10 * 16); //分
    Display_A_Char(V_Page_Offset + 5, 56 + H_Offset, DIGITS + DateTime[1] % 10 * 16);
    Reverse_Display = Adjust_Index = 0;
    Display_A_Char(V_Page_Offset + 5, 80 + H_Offset, DIGITS + DateTime[0]/10 * 16); //秒
    Display_A_Char(V_Page_Offset + 5, 88 + H_Offset, DIGITS + DateTime[0] % 10 * 16);
}

void main()
{
    setup_nmi(); //设置中断矢量
    LedReset(); //LCD 复位
    //显示年的固定前两位
    Display_A_Char(V_Page_Offset, 0 + H_Offset, DIGITS + 2 * 16);
    Display_A_Char(V_Page_Offset, 8 + H_Offset, DIGITS);
    //显示固定的汉字:年月日时分秒
    Display_A_WORD(V_Page_Offset, 32 + H_Offset, DATE_TIME_WORDS + 0 * 32);
    Display_A_WORD(V_Page_Offset, 64 + H_Offset, DATE_TIME_WORDS + 1 * 32);
    Display_A_WORD(V_Page_Offset, 96 + H_Offset, DATE_TIME_WORDS + 2 * 32);
    Display_A_WORD(V_Page_Offset + 2, 64 + H_Offset, DATE_TIME_WORDS + 3 * 32);
}

```

```

Display_A_WORD( V_Page_Offset + 2,80 + H_Offset, DATE_TIME_WORDS + 4 * 32 );
Display_A_WORD( V_Page_Offset + 5,32 + H_Offset, DATE_TIME_WORDS + 5 * 32 );
Display_A_WORD( V_Page_Offset + 5,64 + H_Offset, DATE_TIME_WORDS + 6 * 32 );
Display_A_WORD( V_Page_Offset + 5,96 + H_Offset, DATE_TIME_WORDS + 7 * 32 );
RefreshWeekDay(); //计算星期几
while(1)
{
    GetTime();          //获取时间
    disp();             //显示输出
    key();              //检测闭合键
}
}

```

;nmi. c 文件(NMI 中断矢量及可编程定时/计数器 8253A 初始化)

```

#define uchar unsigned char
#define IO2    0x0400    //8253A 地址
/* 可编程定时/计数器 8253A 端口地址 */
#define ADR_TIMER_CONTROL (IO2 + 0x06)
#define ADR_TIMER_DATA0   (IO2 + 0x00)
#define ADR_TIMER_DATA1   (IO2 + 0x02)
#define ADR_TIMER_DATA2   (IO2 + 0x04)
#define TIMER_COUNTER0    0x00
#define TIMER_COUNTER1    0x40
#define TIMER_COUNTER2    0x80
#define TIMER_LATCH       0x00
#define TIMER_LSB         0x10
#define TIMER_MSB         0x20
#define TIMER_LSB_MSB     0x30
#define TIMER_MODE0       0x00
#define TIMER_MODE1       0x02
#define TIMER_MODE2       0x04
#define TIMER_MODE3       0x06
#define TIMER_MODE4       0x08
#define TIMER_MODE5       0x09
#define TIMER_BCD         0x01
uchar    Sec_Flag;        //秒中断标志

```

//设置中断矢量表

```
void set_int(unsigned char int_no, void *service_proc)
```

```

    | _asm
    | {
    |     push es
    |     xor ax, ax
    |     mov es, ax
    |     mov al, int_no
    |     xor ah, ah
    |     shl ax, 1
    |     shl ax, 1
    |     mov si, ax
    |     mov ax, service_proc
    |     mov es:[si], ax
    |     inc si
    |     inc si
    |     mov bx, cs
    |     mov es:[si], bx
    |     pop es
    | }
    |
    |
    | //中断处理函数
    | void interrupt_far nmi_handler( void)
    | {
    |     Sec_Flag = 1;
    | }
    |
    | void setup_nmi(void)
    |     //在中断矢量表添加 2 号中断矢量
    |     | set_int(0x02, (void *) &nmi_handler);
    |     //设置 8253A 定时/计数器
    |     outp( ADR_TIMER_CONTROL,      TIMER_COUNTER0 |  TIMER_MODE3 |
    |     TIMER_LSB_MSB|TIMER_BCD);
    |     outp(ADR_TIMER_DATA0, 0x00);
    |     outp(ADR_TIMER_DATA0, 0x01);
    |     outp( ADR_TIMER_CONTROL,      TIMER_COUNTER2 |  TIMER_MODE2 |
    |     TIMER_LSB_MSB|TIMER_BCD);
    |     outp(ADR_TIMER_DATA2, 0x00);
    |     outp(ADR_TIMER_DATA2, 0x10);
    | }

```

```
;lcm12864. h 头文件
#define uchar unsigned char
#define IO1    0x0200
//LCD Registers addresses
#define CS1 0x08
#define CS2 0x10
#define LCD_DIS          0x3f
#define LCD_START_ROW    0xc0
#define LCD_PAGE          0xb8
#define LCD_COL           0x40
#define LCD_CMD_WR      (IO1 + 0x00 + CS1 + CS2)
#define LCD_DATA_WR     (IO1 + 0x02)
#define LCD_BUSY_RD     (IO1 + 0x04)
#define LCD_DATA_RD     (IO1 + 0x06)
```

;lcm12864. c 文件

```
#include "lcm12864. h"
```

```
extern char Reverse_Display;
```

//输出一字节到 I/O 端口

```
void outp(unsigned int addr, char data)
```

```
{_asm
    | mov dx, addr
      mov al, data
      out dx, al
    |
}
```

//从 I/O 端口输入一字节

```
char inp(unsigned int addr)
```

```
{char result;
_asm
    | mov dx, addr
      in al, dx
      mov result, al
    |
    return result;
}
```

```
//LCD 忙等待
void WaitForEnable( void)
{
    uchar result;
    do{
        result = inp( LCD_BUSY_RD );
    } while( result&0x80 );
}

//LCD 写命令
void LcdWriteCommand( uchar cmd)
{
    WaitForEnable();

    outp( LCD_CMD_WR,cmd );
}

//LCD 写数据
void LcdWriteData( unsigned char data,uchar cs)
{
    WaitForEnable();
    if( cs ) outp( LCD_DATA_WR|CS1,data );
    else outp( LCD_DATA_WR|CS2,data );
}

void LcdReset( void) //LCD 复位
{
    LcdWriteCommand( LCD_DIS );
    LcdWriteCommand( LCD_START_ROW );
    LcdWriteCommand( LCD_PAGE );
    LcdWriteCommand( LCD_COL );
}

void Common_Show(uchar P,uchar L,uchar W,uchar *r)
{
    uchar i;
    if( L < 64 )
    {
        LcdWriteCommand( LCD_PAGE|P );
```

```

LCDWriteCommand( LCD_COL + L );
if( L + W < 64 )
{
    for( i = 0; i < W; i ++ )
        { if( ! Reverse_Display ) LCDWriteData( r[ i ], 1 ); else LCDWriteData( ~ r[ i ], 1 ); }
    else
    {
        for( i = 0; i < 64 - L; i ++ ) { if( ! Reverse_Display ) LCDWriteData( r[ i ], 1 );
            else LCDWriteData( ~ r[ i ], 1 ); }
        LCDWriteCommand( LCD_PAGE + P );
        LCDWriteCommand( LCD_COL );
        for( i = 64 - L; i < W; i ++ )
            { if( ! Reverse_Display ) LCDWriteData( r[ i ], 0 ); else LCDWriteData( ~ r[ i ], 0 ); }
        }
    else
    {
        LCDWriteCommand( LCD_PAGE + P );
        LCDWriteCommand( LCD_COL + L - 64 );
        for( i = 0; i < W; i ++ )
            { if( ! Reverse_Display ) LCDWriteData( r[ i ], 0 ); else LCDWriteData( ~ r[ i ], 0 ); }
        }
}

void Display_A_Char( uchar P, uchar L, uchar * M )
{
    Common_Show( P, L, 8, M );
    Common_Show( P + 1, L, 8, M + 8 );
}

void Display_A_WORD( uchar P, uchar L, uchar * M )
{
    Common_Show( P, L, 16, M );
    Common_Show( P + 1, L, 16, M + 16 );
}

void Display_A_WORD_String( uchar P, uchar L, uchar C, uchar * M )
{
    uchar i;

```

```

for(i=0;i<C;i++)
    Display_A_WORD(P,L+i*16,M+i*32);
}

;build.bat 批处理文件
ml /Zm /Zd /Zi /Zf -c RTL.ASM
dmc -0 -g -ms -0 -c -Ic:\dm\INCLUDE lcm12864.obj lcm12864.c
dmc -0 -g -ms -0 -c -Ic:\dm\INCLUDE nmi.obj nmi.c
dmc -0 -g -ms -0 -c -Ic:\dm\INCLUDE main.obj main.c
link/CO/NOD/DEB/DEBUGB /STACK:1024main.obj + lcm12864.obj + nmi.obj + rtl.obj

```

5.4 LCD2002 显示 ADC0809 八路模拟转换结果

利用字符液晶显示器 LCD2002 显示 ADC0809 八路模拟转换结果, 其电路原理如图 5-6 所示。ADC0809 采用延时方式等待转换结束, 通道号通过数据线低 3 位 AD0 ~ AD2 输出。

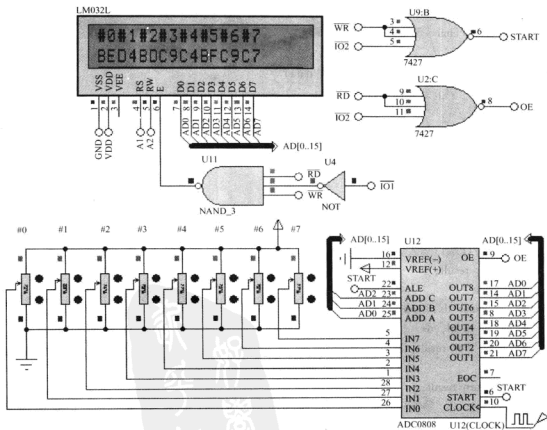


图 5-6 LCD2002 显示 ADC0809 八路模拟转换结果的电路原理

参考程序:

;rtl.asm 源程序同 5.1 节的源程序

;main.c 文件

```
#define IO1    0x0200
```

```
#define IO2    0x0400
```

```
#define ADC0809_ADR  IO2                                //模/数转换器 ADC0809 片选地址
```

//LCD 寄存器地址

```
#define LCD_CMD_WR    (IO1 + 0x00)                    //写 LCD 命令字寄存器地址
```

```
#define LCD_DATA_WR    (IO1 + 0x02)                    //写 LCD 数据字寄存器地址
```

```
#define LCD_BUSY_RD    (IO1 + 0x04)                    //读 LCD 状态字寄存器地址
```

```
#define LCD_DATA_RD    (IO1 + 0x06)                    //读 LCD 数据字寄存器地址
```

```
unsigned char    Str1[] = "#0#1#2#3#4#5#6#7"; //LCD 第 1 行显示缓冲区
```

```
unsigned char    Str2[] = "                "; //LCD 第 2 行显示缓冲区
```

```
unsigned char    temp;
```

```
void outp(unsigned int addr, char data)
```

//输出一字节到 I/O 端口

```
{_asm
```

```
|mov dx, addr
```

```
    mov al, data
```

```
    out dx, al
```

```
|
```

```
}
```

```
char inp(unsigned int addr)
```

//从 I/O 端口输入一字节

```
{char result;
```

```
_asm
```

```
|mov dx, addr
```

```
    in al, dx
```

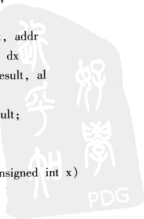
```
    mov result, al
```

```
|
```

```
    return result;
```

```
}
```

```
void delay(unsigned int x)
```




```
//延时
{ unsigned char i;
  while(x -- )
    | for(i = 0; i < 120; i ++ ) | | |
  |

//LCD 忙等待
void WaitForEnable( void )
{
    unsigned char result;
    do{
        result = inp( LCD_BUSY_RD );
        | while( result&0x80 );
    }

//LCD 写命令
void LcdWriteCommand( unsigned char cmd )
{
    WaitForEnable( );
    outp( LCD_CMD_WR, cmd );
}

//LCD 写数据
void LcdWriteData( char data )
{
    WaitForEnable( );
    outp( LCD_DATA_WR, data );
}

void LcdReset( void )
//LCD 初始化
{
    LcdWriteCommand( 0x38 );
    LcdWriteCommand( 0x0c );
    LcdWriteCommand( 0x06 );
    LcdWriteCommand( 0x01 );
}

void led_pos( unsigned char pos )
//LCD 字符显示位置
{
    LcdWriteCommand( pos!0x80 );
```

```

}

void lcd_out(unsigned char *p)
//输出显示的字符串
{
    while( *p!= '\0')
    {
        LcdWriteData( *p);
        p + +;
    }
}

void main( void)
{
    unsigned char i;
    LcdReset();           //LCD 复位
    while(1)
    {
        for(i=0;i <= 7;i + +)      //共 8 个通道
        {
            outp( ADC0809_ADR,i);    //启动 i 通道
            delay(4);                //等待转换完成
            temp = inp( ADC0809_ADR); //读取转换结果
            //将每个通道 AD 转换结果 temp 的值转换为 2 位十六进制字符存入 str2 显示缓冲区
            if( temp%16 <= 9&&temp%16 >= 0) Str2[ 2 * i + 1] = temp%16 + '0';           //
HEX 低半字节 0 ~ 9
            if( temp%16 <= 0x0F&&temp%16 >= 0x0A) Str2[ 2 * i + 1] = temp%16 + 55; //
HEX 低半字节 A ~ F
            if( temp/16 <= 9&&temp/16 >= 0) Str2[ 2 * i] = temp/16 + '0';           //
HEX 高半字节 0 ~ 9
            if( temp/16 <= 0x0F&&temp/16 >= 0x0A) Str2[ 2 * i] = temp/16 + 55; //
HEX 高半字节 A ~ F
        }
        lcd_pos(0x00);           //LCD 第 1 行行首地址
        lcd_out( Str1);          //输出显示第 1 行字符串,通道号
        lcd_pos(0x40);           //LCD 第 2 行行首地址
        lcd_out( Str2);          //输出显示第 2 行字符串,每个通道的 AD 转换结果
        delay(100);
    }
}

```

;build.bat 批处理文件

```
ml /Zm /Zd /Zi /Zf -c RTL.ASM
```

```
dmc -O -g -ms -O -c -lc:\dm\INCLUDE main.obj main.c
```

```
link /CO /NOD /DEB /DEBUGB /STACK:1024 main.obj + rtl.obj
```



参考文献

- [1] 李文英, 等. 微机原理与接口技术 [M]. 北京: 清华大学出版社, 2001.
- [2] 李学礼. 基于 Proteus 的 8051 单片机实例教程 [M]. 北京: 电子工业出版社, 2008.
- [3] 彭伟. 单片机 C 语言程序设计实训 100 例—基于 8051 + Proteus 仿真 [M]. 北京: 电子工业出版社, 2008.
- [4] 郑晓薇. 汇编语言 [M]. 北京: 机械工业出版社, 2009.
- [5] 尹建华. 微型计算机原理与接口技术 [M]. 2 版. 北京: 高等教育出版社, 2008.
- [6] 熊江, 杨风年, 成运. 微机系统与接口技术 [M]. 武汉: 武汉大学出版社, 2007.
- [7] 李继灿. 微型计算机系统与接口 [M]. 北京: 清华大学出版社, 2005.
- [8] 杨季文. 80X86 汇编语言程序设计教程 [M]. 北京: 清华大学出版社, 2009.
- [9] 周明德, 陶龙芳. 微机原理与应用实验 [M]. 北京: 中央广播电视大学出版社, 1998.



读者信息反馈表

感谢您购买《微机原理与接口技术实验——基于 Proteus 仿真》一书。为了更好地为您服务，有针对性地为您提供图书信息，方便您选购合适图书，我们希望了解您的需求和对我们教材的意见和建议，愿这小小的表格为我们架起一座沟通的桥梁。

姓 名		所在单位名称	
性 别		所从事工作（或专业）	
通信地址			邮 编
办公电话		移动电话	
E-mail			
1. 您选择图书时主要考虑的因素：（在相应项前面画√） （ ） 出版社 （ ） 内容 （ ） 价格 （ ） 封面设计 （ ） 其他 2. 您选择我们图书的途径（在相应项前面画√） （ ） 书目 （ ） 书店 （ ） 网站 （ ） 朋友推介 （ ） 其他			
希望我们与您经常保持联系的方式： ☐ 电子邮件信息 ☐ 定期邮寄书目 ☐ 通过编辑联络 ☐ 定期电话咨询			
您关注（或需要）哪些类图书和教材：			
您对我社图书出版有哪些意见和建议（可从内容、质量、设计、需求等方面谈）：			
您今后是否准备出版相应的教材、图书或专著（请写出出版的专业方向、准备出版的时间、出版社的选择等）：			

非常感谢您能抽出宝贵的时间完成这张调查表的填写并回寄给我们，我们愿以真诚的服务回报您对机械工业出版社技能教育分社的关心和支持。

请联系我们——

地 址：北京市西城区百万庄大街 22 号 机械工业出版社技能教育分社

邮 编：100037

社长电话：(010) 88379080 88379083 68329397（带传真）

E-mail: jnfs@mail.machineinfo.gov.cn